

Bangladesh University of Engineering and Technology

Department of Computer Science & Engineering

Database Question Bank

CSE 303 · CSE 215 · Database

Year Coverage: 2010–11 to 2024–25

Topics: Database Fundamentals, Relational Model, ER Model,
SQL & Query Languages, Normalization, Storage & Indexing,
Query Processing, Transaction Management, NoSQL & Big Data

Authors & Contributors

Md Ratul Hasan (2405009) — Question Curation & Organisation
— $\text{\LaTeX}$ Typesetting

NotebookLM — Research & Extraction

Claude AI — $\text{\LaTeX}$ Typesetting

Gemini — Daigram Generation

Table of Contents

Part I — Database Fundamentals & Database Design

- ▷ **S1.** Database Fundamentals
 - Concepts of Database Systems
 - Relational Model
- ▷ **S2.** Database Design
 - Entity-Relationship (ER) Model
 - ER Diagrams
 - Database Schema Design

Part II — Query Languages

- ▷ **S3.** Query Languages
 - SQL
 - Relational Algebra
 - Joins
 - Constraints, Assertions & Triggers

Part III — Normalization

- ▷ **S4.** Normalization
 - Functional Dependencies
 - Normal Forms (1NF, 2NF, 3NF, BCNF, 4NF)
 - Anomalies & Decomposition

Part IV — Storage, Indexing & Query Processing

- ▷ **S5.** Storage and Indexing
 - Data Storage Architecture & RAID
 - Primary Index
 - Secondary & Bitmap Index
 - B⁺ Tree Index
 - Hash Index
- ▷ **S6.** Query Processing
 - Query Processing
 - Query Optimization

Part V — Transaction Management & Advanced Topics

- ▷ **S7.** Transaction Management
 - Transactions & ACID Properties
 - Concurrency Control
 - Deadlock
 - Recovery Systems

▷ S8. Advanced Topics

· Distributed & Parallel Databases

BUET · CSE 215 · Database

Part I

Database Fundamentals & Database Design

Database Fundamentals & Database Design

S1 — Database Fundamentals

Concepts of Database Systems

Q1. Draw the diagram of 2-tier and 3-tier database architectures. (5 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer

1. Two-Tier Database Architecture

In a **2-tier architecture**, the system is divided into two distinct layers: the client and the database server. The client machine contains both the user interface and the application business logic, while the database system runs on the server.

Client Tier (Tier 1)
GUI & Application Business Logic
(Fat Client)

↕ Direct Network Connection
(e.g., ODBC / JDBC API)

Database Tier (Tier 2)
Database Management System (DBMS)
(Data Storage & Access)

Properties & Characteristics:

- Working Principle:** The client application directly establishes a connection to the database server using APIs like ODBC or JDBC. Queries are generated by the client and sent directly to the database.
- Advantages:**
 - Simple to design, develop, and deploy.
 - Faster response times for small user groups due to the lack of an intermediate layer.
- Disadvantages:**
 - Scalability Issue:** Performance degrades significantly as the number of concurrent users increases (each requires a dedicated database connection).
 - Security Risk:** The database is directly exposed to client applications.
 - Maintenance:** Application logic changes require updates on every individual client machine.

2. Three-Tier Database Architecture

A **3-tier architecture** decouples the user interface, business logic, and database management into three separate layers. This is the standard architecture for modern web applications.

Presentation Tier (Tier 1)
User Interface (Web Browser / App)
(Thin Client)

↕ Network Protocol
(e.g., HTTP / HTTPS)

Application Tier (Tier 2)
Business Logic & Query Generation
(Web / App Server)

↕ Database Protocol
(e.g., TCP/IP, SQL Access)

Database Tier (Tier 3)
Database Management System (DBMS)
(Data Storage)

Properties & Characteristics:

- Working Principle:** The client sends a request to the application server. The application server evaluates the business rules, forms the necessary database queries, executes them against the database server, and finally sends the processed result back to the client.

5

- **Advantages:**
 - **High Scalability:** Connection pooling at the application layer allows thousands of clients to share a few database connections.
 - **Improved Security:** The database is shielded from direct client access. The application layer acts as a secure gateway.
 - **Maintainability:** Business logic updates are made centrally on the application server without altering client software.
- **Disadvantages:**
 - More complex to design, configure, and maintain.
 - Slight network overhead due to the intermediate routing through the application layer.

3. Comparison Summary

Feature	2-Tier Architecture	3-Tier Architecture
Client Type	Fat client (contains application logic)	Thin client (primarily presentation)
Scalability	Poor (difficult to scale beyond 100 users)	Excellent (handles thousands of users)
Security	Low (direct database access)	High (database hidden behind app server)
Maintenance	High effort (update every client device)	Low effort (centralized updates)
Use Case	Legacy desktop software, internal enterprise tools	Web applications, cloud services, e-commerce

Q2. Why do we use data abstraction? Describe the different data abstraction levels. (8 marks) [CSE 215 | L-2/T-1 | 2024–25]

Answer

1. Definition and Purpose of Data Abstraction

Data Abstraction is the process of hiding the complex, underlying physical storage details of a database from the end-users and applications, presenting them only with the relevant structural view of the data.

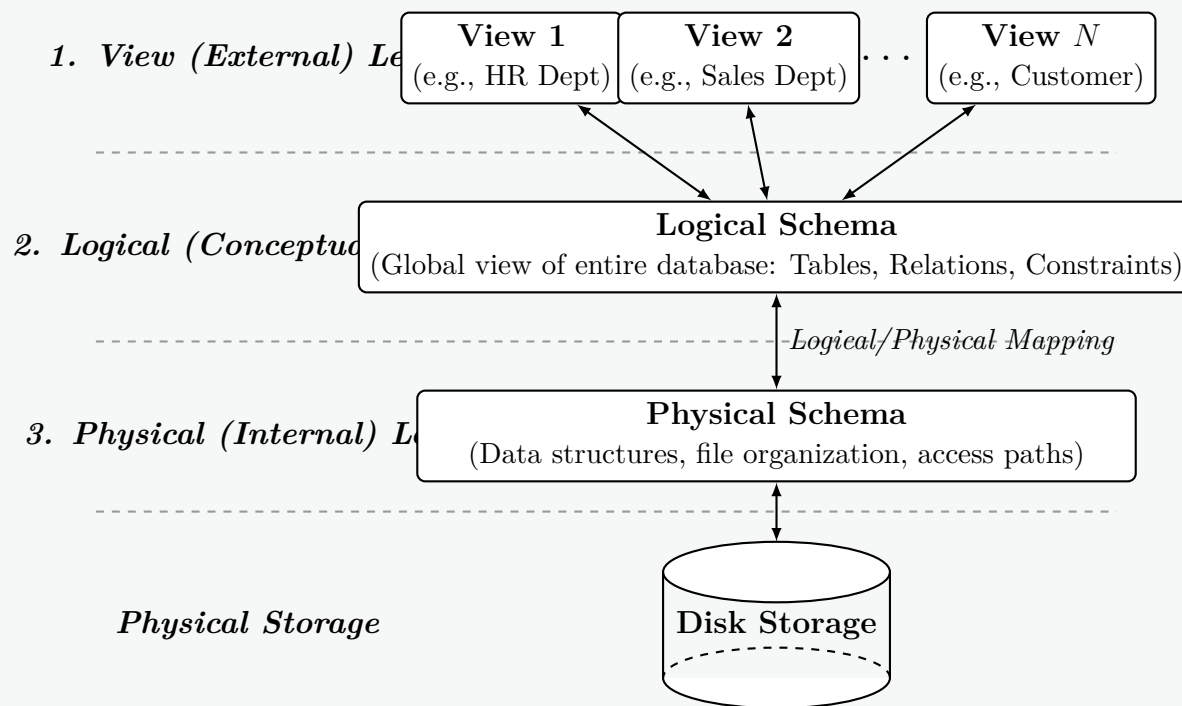
In a Database Management System (DBMS), highly complex data structures (like B+ trees, hash tables, and linked lists) are used to store and retrieve data efficiently. Because most database users are not trained to interact with such structures, the DBMS implements data abstraction.

Why do we use Data Abstraction?

- **Complexity Hiding:** End-users and application developers can interact with the database without needing to understand disk blocks, byte offsets, or indexing mechanisms.
- **Security & Authorization:** By restricting what specific users can see (via views), the DBMS prevents unauthorized access to sensitive information.
- **Data Independence:** The ability to change the schema at one level of the database system without having to change the schema at the next higher level.
- **Ease of Access:** It simplifies querying processes by allowing users to use high-level declarative languages like SQL.

2. Architecture of Data Abstraction

Data abstraction is typically implemented using the **Three-Schema Architecture** (also known as the ANSI/SPARC architecture), which divides the database into three distinct levels.



3. Detailed Description of Abstraction Levels

(a) Physical (Internal) Level:

- **Definition:** The lowest level of abstraction. It describes *how* the data is actually stored in the underlying storage mediums (HDDs/SSDs).
- **Working Principle:** It defines data structures (like B+ trees, Hash maps), file organizations (sequential, indexed), compression techniques, and encryption. This level is managed by the Database Administrator (DBA).
- **Example:** Storing a 'Customer' record as a contiguous block of 256 bytes at a specific memory address, utilizing an ISAM (Indexed Sequential Access Method) index.

(b) Logical (Conceptual) Level:

- **Definition:** The middle level of abstraction. It describes *what* data is stored in the database and the relationships existing among those data.
- **Working Principle:** It represents the entire database logically. Programmers and DBAs work at this level using Data Definition Language (DDL). There is no hardware awareness at this level.
- **Example:** Defining a relational table 'Customer(ID, Name, Account_Balance)' with a primary key on 'ID' and a foreign key linking to a 'Transactions' table.

(c) View (External) Level:

- **Definition:** The highest level of abstraction. It describes only *part* of the entire database.
- **Working Principle:** A database can have multiple views tailored for different user groups. It simplifies interaction by hiding data that is not relevant to the user and masking sensitive information.
- **Example:** A bank teller viewing a customer's 'Name' and 'Account_Balance', while the conceptual level also holds the customer's 'Encrypted_Password' and 'SSN', which the teller is completely unaware of.

Q3. Why does it yield current values when derived attributes are not stored in the database? Discuss with an example. **(10/7 marks)**
 [CSE 215 | L-2/T-1 | 2023–24] [CSE 303 | L-3/T-1 | 2014–15]

Answer

1. Definition and Concept

A **Derived Attribute** is an attribute in a database whose value is not permanently stored in the physical storage. Instead, its value is dynamically calculated (or derived) from one or more related **base attributes** (which are physically stored) or system functions (such as the current system time) whenever the attribute is queried.

2. Why Derived Attributes Yield Current Values

Derived attributes yield the most current and accurate values because of their **on-the-fly computation** mechanism. The underlying reasons include:

- **Runtime Calculation:** The DBMS evaluates the derived value at the exact moment the query is executed. It does not read a stale value from disk.
- **Dependency on Current State:** The derivation formula relies on the present state of the base attributes. If a base attribute changes, the derived attribute instantly reflects this change.

- **Integration with System State:** Many derived attributes depend on the system clock (e.g., `CURRENT_DATE`). As time progresses, the calculation naturally factors in the new time, ensuring the result is always up to date.
- **Elimination of Update Anomalies:** If a dynamic value (like age or years of service) were stored statically, a background job would be required to update every record periodically. Skipping this update would lead to stale, incorrect data. Derivation completely avoids this synchronization issue.

3. Illustrative Example

Consider a university database tracking students.

- **Base Attribute (Stored):** `Date_of_Birth` (e.g., '2002-05-15').
- **Derived Attribute (Not Stored):** `Age`.

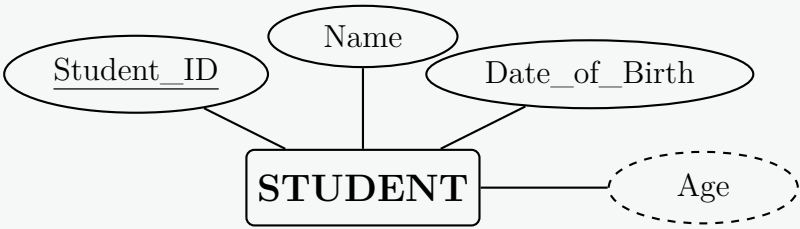
If `Age` were physically stored as '20' in the year 2022, it would become incorrect (stale) by the year 2024. However, by treating `Age` as a derived attribute, the DBMS computes it dynamically using the formula:

$$\text{Age} = \text{Current_Date} - \text{Date_of_Birth}$$

Because `Current_Date` advances automatically, executing this query in 2026 will accurately and mathematically yield an age of 24, without a single write operation being made to the database since the student's insertion.

4. ER Diagram Representation

In an Entity-Relationship (ER) model, derived attributes are denoted by a **dashed ellipse**. The base attributes are denoted by a standard solid ellipse.



5. Implementation (SQL View or Query)

In standard relational databases, derived attributes are typically implemented using expressions in a `SELECT` statement or encapsulated within a **View**.

```
-- Creating a view to continuously expose the derived 'Age' attribute
CREATE VIEW Student_Current_Profile AS
SELECT
    Student_ID,
    Name,
    Date_of_Birth,
    -- Derivation calculation happens here on the fly
    FLOOR(DATEDIFF(DAY, Date_of_Birth, CURRENT_DATE) / 365.25) AS Age
FROM
    STUDENT;
```

6. Comparison Summary

Property	Stored Attribute	Derived Attribute
Storage	Occupies physical disk space.	Does not occupy disk space.
Accuracy over time	Becomes stale if not updated continuously.	Always accurate; dynamically computes current state.
Performance	Faster retrieval (simple disk read).	Slower retrieval (requires CPU computation time).
Redundancy	Introduces redundancy if logically dependent on other attributes.	Completely eliminates logical redundancy.

Q4. What are the requirements of an ideal DBMS? Define entity and attribute in the context of database with suitable example(s). (10 marks)

CSE 215 | L-2/T-2 | 2018–19

Answer

1. Requirements of an Ideal DBMS

An ideal Database Management System (DBMS) must provide a robust, secure, and efficient environment for storing and retrieving data. The primary requirements include:

- **Data Integrity and Consistency:** The system must enforce constraints (e.g., primary keys, foreign keys, domain constraints) to ensure data remains accurate and valid across all tables, even after multiple transactions.
- **Concurrency Control:** It must allow multiple users to access and modify data simultaneously without causing data corruption or inconsistencies, typically achieved by enforcing ACID (Atomicity, Consistency, Isolation, Durability) properties.
- **Data Security and Authorization:** The DBMS must restrict unauthorized access, ensuring that only authenticated users with specific privileges can view or manipulate sensitive data.
- **Crash Recovery and Resilience:** In the event of a hardware or software failure, the system must be able to restore the database to a consistent state without losing any committed data.
- **Data Independence:** The architecture should allow changes in the physical storage layer without affecting the logical schema (Physical Data Independence), and changes in the logical schema without rewriting application programs (Logical Data Independence).
- **Performance and Scalability:** It must efficiently process complex queries over massive datasets and scale seamlessly as the volume of data and the number of concurrent users increase.
- **High-Level Query Language:** It should provide a declarative interface (like SQL) allowing users to retrieve data easily without needing to understand complex physical access paths.

2. Definition of Entity and Attribute

In the context of database design and the Entity-Relationship (ER) model:

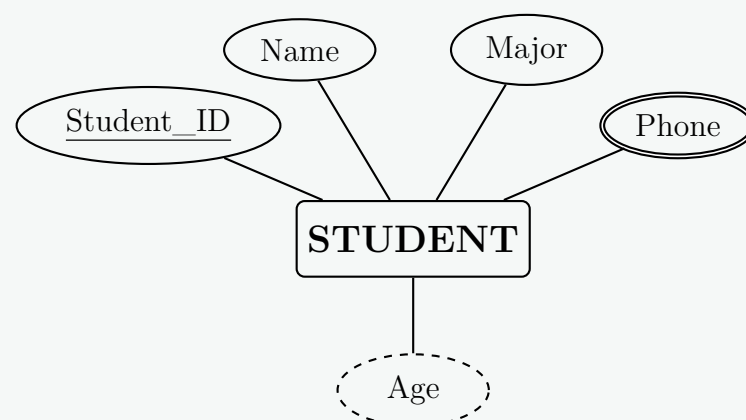
- **Entity:** An entity is a distinguishable real-world object, concept, or event that exists independently and about which data needs to be collected and stored. Entities are mapped to **Tables** (or relations) in a relational database.
- **Attribute:** An attribute is a characteristic, property, or trait that describes an entity. It holds specific pieces of information about the entity. Attributes are mapped to **Columns** in a relational database.

3. Suitable Example and ER Diagram

Let us consider a university database environment:

- **Entity:** STUDENT (represents a physical person).
- **Attributes of STUDENT:**
 - **Student_ID:** A *key attribute* (uniquely identifies the student).
 - **Name:** A *simple attribute*.
 - **Phone:** A *multivalued attribute* (a student can have multiple phone numbers).
 - **Age:** A *derived attribute* (calculated from Date of Birth).

The following ER Diagram visually represents the **STUDENT** entity and its associated attributes based on standard ER notation:



4. Comparison Table (Entity vs. Attribute)		
Feature	Entity	Attribute
Definition	A distinct real-world object or concept.	A property or characteristic describing an object.
Relational Mapping	Becomes a Table .	Becomes a Column in that table.
ER Diagram Notation	Represented by a Rectangle .	Represented by an Ellipse .
Example	‘Car’, ‘Employee’, ‘Order’	‘Color’, ‘Salary’, ‘Order_Date’

Q5. What are bags in DBMS? How are they different from sets? Give examples of union, intersection and minus operation on bags to highlight the differences. (15 marks) [CSE 303 | L-3/T-1 | 2012–13]

Answer

1. Definition of Bags (Multisets) in DBMS

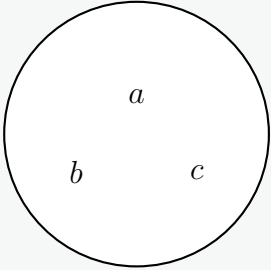
In the context of a Database Management System (DBMS), a **bag** (also known as a **multiset**) is an unordered collection of elements where **duplicate elements are permitted**.

While pure relational algebra is mathematically based on set theory (where duplicates are strictly prohibited), commercial database systems (like SQL) inherently operate on bags by default. This is because eliminating duplicates is a computationally expensive operation (requiring sorting or hashing). Therefore, unless a user explicitly requests duplicate removal (e.g., using the **DISTINCT** keyword), a DBMS will output a bag of tuples.

2. Difference Between Bags and Sets

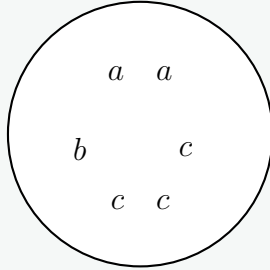
- Sets:** A set contains distinct elements. An element x either belongs to a set S or it does not. The cardinality (count) of any element in a set is at most 1.
- Bags:** A bag can contain multiple identical copies of the same element. An element x belongs to a bag B with a specific count or frequency, denoted as $C(x, B) \geq 0$.

Set



Unique elements only

Bag (Multiset)



Duplicates are permitted

3. Operations on Bags vs. Sets

Let $C(x, A)$ represent the number of occurrences of an element x in a bag A . Consider two bags:

- Bag $A = \{a, a, a, b, b, c\}$
- Bag $B = \{a, a, b, b, b, d\}$

Therefore, element counts are: $C(a, A) = 3$, $C(b, A) = 2$, $C(c, A) = 1$; and $C(a, B) = 2$, $C(b, B) = 3$, $C(d, B) = 1$.

A. Bag Union ($\cup$)

In a bag union, the count of an element in the result is the **sum** of its counts in the two bags.

$$C(x, A \cup B) = C(x, A) + C(x, B)$$

- Bag Result:** $A \cup B = \{a, a, a, a, a, b, b, b, b, b, c, d\}$
- Comparison to Set Union:** A strict set union of A and B would simply be $\{a, b, c, d\}$, completely ignoring frequencies.

B. Bag Intersection ($\cap$)

In a bag intersection, the count of an element in the result is the **minimum** of its counts in the two bags.

$$C(x, A \cap B) = \min(C(x, A), C(x, B))$$

- Bag Result:** $A \cap B = \{a, a, b, b\}$
- Explanation:** a appears three times in A and two times in B ; the minimum is two. b appears twice in A and three times in B ; the minimum is two. c and d do not appear in both, so their minimum count is zero.

10

C. Bag Difference / Minus (−)

In a bag difference, the count of an element in the result is the count in the first bag **minus** the count in the second bag, floored at zero (an element cannot have a negative count).

$$C(x, A - B) = \max(0, C(x, A) - C(x, B))$$

- **Bag Result:** $A - B = \{a, c\}$
- *Explanation:* For a : $\max(0, 3 - 2) = 1$. For b : $\max(0, 2 - 3) = 0$. For c : $\max(0, 1 - 0) = 1$.

4. Comparison Summary Table

Feature	Set Semantics	Bag (Multiset) Semantics
Duplicates	Strictly not allowed.	Allowed and maintained.
Union ($A \cup B$)	Element exists if it is in A or B .	Sum of occurrences: $C(A) + C(B)$.
Intersection ($A \cap B$)	Element exists if it is in both A and B .	Minimum of occurrences: $\min(C(A), C(B))$.
Difference ($A - B$)	Element exists if it is in A but not B .	Remaining occurrences: $\max(0, C(A) - C(B))$.
Performance	Slower (requires duplicate elimination).	Faster (simply streams tuples).

Q6. What are the key properties of a DBMS? (8 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Answer

1. Introduction

A Database Management System (DBMS) is designed to manage large bodies of information efficiently and securely. To achieve this, it possesses several key properties that distinguish it from traditional file-processing systems, making it the standard for modern data management.

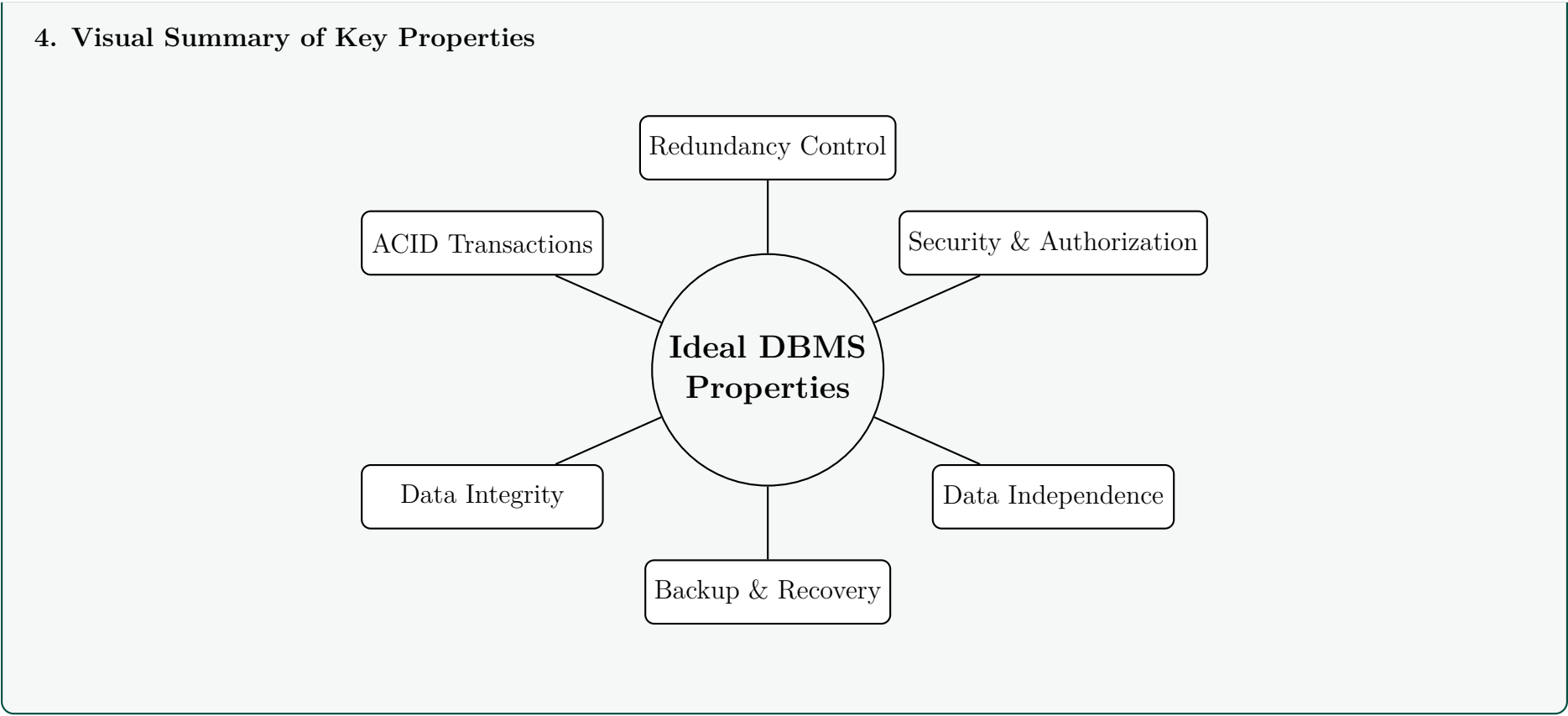
2. Fundamental Properties of a DBMS

- **Control of Data Redundancy:** Unlike flat-file systems where duplicate data is scattered across multiple locations, a DBMS integrates data into a central repository, minimizing duplication and preventing data inconsistency.
- **Data Security and Authorization:** A DBMS enforces security by ensuring that only authorized users can access or modify the data. It supports the creation of user accounts, roles, and granular access privilege configurations.
- **Data Independence:** The architecture provides a strict separation between application programs and the physical data storage.
 - *Logical Data Independence:* The ability to change the conceptual schema without altering application programs.
 - *Physical Data Independence:* The ability to change the underlying storage structure without altering the conceptual schema.
- **Concurrency Control:** When multiple users or programs attempt to access the same data simultaneously, the DBMS systematically manages these interactions (usually via locking mechanisms) to prevent data corruption.
- **Backup and Recovery:** A DBMS provides robust, automatic mechanisms to back up data and recover it to a consistent state in the event of hardware crashes or software failures.
- **Data Integrity:** The system enforces structural rules and constraints (such as primary keys, foreign keys, and domain constraints) to ensure all data stored is structurally accurate and functionally valid.

3. Transaction Properties (ACID)

At the core of a robust relational DBMS is its ability to process database transactions reliably. These required transactional properties are summarized by the **ACID** acronym:

- (a) **Atomicity:** Often called the "all-or-nothing" rule. A transaction is treated as a single indivisible logical unit of work; either all its operations are executed successfully, or none are.
- (b) **Consistency:** A transaction must reliably transition the database from one valid, consistent state to another, strictly adhering to all defined database rules and constraints.
- (c) **Isolation:** Concurrent transactions execute completely independent of one another. Intermediate, uncommitted states of one transaction are invisible to other concurrent transactions.
- (d) **Durability:** Once a transaction is successfully committed, its changes are permanently recorded in non-volatile storage and will survive any subsequent system failures or power losses.



Q7. Show the major components and their interactions in a DBMS. Briefly explain the functionalities of major components. **(15 marks)**
[CSE 303 | L-3/T-1 | 2011–12]

Answer

1. DBMS Architecture and Component Interactions

A Database Management System (DBMS) is highly complex and is divided into multiple modular components that interact to execute queries, manage storage, and ensure data integrity. The architecture is broadly categorized into the **Query Processor**, the **Storage Manager**, and the **Disk Storage**.

The diagram below illustrates the major components and the flow of instructions (interactions) from the user level down to the physical disk.

```
graph TD; DA[Database Admin] --> DDL[DDL Commands]; DA --> DDI[DDL Interpreter]; DA --> QEE[Query Evaluation Engine]; DA --> AIM[Authorization & Integrity Manager]; DA --> FM[File Manager]; DA --> DD[Data Dictionary]; DA --> I[Indices]; DA --> DF[Data Files]; EUA[End Users / Apps] --> DML[DML Queries]; EUA --> DMO[DML Compiler & Optimizer]; EUA --> QEE; EUA --> TM[Transaction Manager]; EUA --> BM[Buffer Manager]; EUA --> DF; subgraph QueryProcessor [Query Processor]; DDI; DMO; end; subgraph StorageManager [Storage Manager]; AIM; TM; end; subgraph DiskStorage [Disk Storage]; DD; I; DF; end; DDI --> QEE; DMO --> QEE; QEE --> AIM; QEE --> TM; AIM <--> FM; FM <--> DD; FM <--> I; TM <--> BM; BM <--> DF;
```

2. Functionalities of Major Components

The DBMS is divided into three primary functional blocks:

A. Query Processor

The query processor handles incoming queries, translates them into low-level instructions, and optimizes them for performance.

- **DDL Interpreter:** Parses Data Definition Language (DDL) statements (like `CREATE TABLE`) and records the definitions in the data dictionary.
- **DML Compiler and Query Optimizer:** Translates Data Manipulation Language (DML) statements (like `SELECT`, `UPDATE`) into an execution plan. The optimizer evaluates various execution strategies and selects the most efficient one based on cost (CPU, I/O).
- **Query Evaluation Engine:** Executes the low-level instructions generated by the DML compiler, coordinating with the storage manager to retrieve the required records.

B. Storage Manager

The storage manager acts as the interface between the low-level data on the disk and the application queries. It is responsible for efficient storage, retrieval, and updating of data.

- **Authorization and Integrity Manager:** Verifies that the user issuing the query has the correct permissions and ensures that the operation does not violate any database constraints (e.g., primary/foreign keys).
- **Transaction Manager:** Ensures that the database remains in a consistent state despite system failures. It controls concurrent access, ensuring transactions adhere to ACID properties.
- **File Manager:** Manages the allocation of space on disk storage and the data structures used to represent information stored on disk.
- **Buffer Manager:** Responsible for fetching data from disk storage into main memory (RAM) and deciding what data to cache to minimize expensive disk I/O operations.

C. Disk Storage (Data Structures)

This represents the physical layer where the information is permanently written.

- **Data Files:** The actual files residing on the disk that store the database records.
- **Data Dictionary (Metadata):** Stores data about the data, including table schemas, constraints, user permissions, and statistical data used by the query optimizer.
- **Indices:** Auxiliary data structures (such as B+ Trees or Hash tables) that provide fast access to data items based on key values, bypassing sequential scans.

Q8. Describe the key properties that give a DBMS edge over a file system. (14 marks)

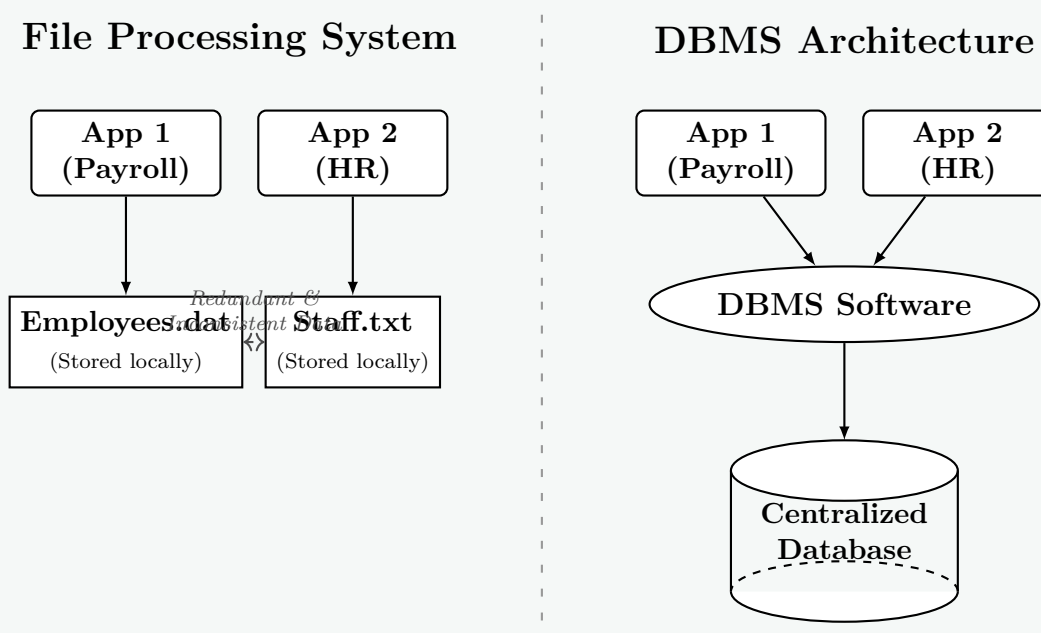
[CSE 215 | L-3/T-1 | 2010–11]

Answer

1. Introduction

Before the advent of Database Management Systems (DBMS), data was stored using traditional file-processing systems where each application maintained its own set of files. This approach led to severe limitations as data grew in size and complexity. A DBMS provides a centralized, robust architecture that overcomes these limitations, offering significant advantages in data handling, security, and performance.

2. Visual Comparison: Architecture



3. Key Properties That Give DBMS an Edge

The decisive edge of a DBMS over file-based systems stems from the following core properties:

- (a) **Data Abstraction and Independence:**
 - File System:* Application code is tightly coupled to the physical structure of the files. Adding a new field to a text file requires rewriting every application that reads it.
 - DBMS:* Provides logical and physical **data independence**. Users interact with a logical view of the data. The underlying storage mechanisms (like switching from HDDs to SSDs or altering indexing methods) can change without impacting the application code.
- (b) **Control of Data Redundancy and Inconsistency:**
 - File System:* Data is fragmented across multiple departments, leading to massive duplication (redundancy). If an employee’s address changes, it might be updated in the Payroll file but forgotten in the HR file (inconsistency).
 - DBMS:* Integrates and centralizes data via **normalization**. Duplication is minimized, ensuring that a single update reflects universally, maintaining global data consistency.
- (c) **High-Level Data Access (Query Languages):**
 - File System:* Extracting specific data requires writing custom procedural scripts (e.g., writing a Python script to parse a CSV and filter records).
 - DBMS:* Offers declarative query languages like **SQL**. A user merely describes *what* data is needed (e.g., `SELECT * FROM Employees WHERE Salary > 50000`), and the DBMS optimizer dynamically determines the most efficient way to fetch it using built-in access paths and indices.
- (d) **Enforcement of Data Integrity:**
 - File System:* Integrity constraints (e.g., "Age must be > 18" or "Department ID must exist") must be manually coded into every single application.
 - DBMS:* Constraints (Primary Keys, Foreign Keys, Domain constraints) are centralized within the database schema. The DBMS strictly rejects any transaction that violates these rules, guaranteeing valid data.
- (e) **Concurrency Control and ACID Transactions:**
 - File System:* Employs coarse-grained locking. If one user opens a file to edit a single line, the entire file is often locked, preventing concurrent access by others.
 - DBMS:* Provides fine-grained (row-level) locking and guarantees **ACID** (Atomicity, Consistency, Isolation, Durability) properties. Thousands of users can read and write simultaneously without interfering with each other (Isolation), and transactions are treated as indivisible units of work (Atomicity).
- (f) **Crash Recovery (Durability):**
 - File System:* A power failure mid-write can corrupt the file, requiring manual restoration from a backup.
 - DBMS:* Uses Write-Ahead Logging (WAL) and undo/redo mechanisms. If a crash occurs, the DBMS automatically recovers, rolling back incomplete transactions to ensure the database remains in a consistent state.

4. Comparison Summary

Feature	Traditional File System	DBMS
Data Redundancy	Very high (scattered duplicate data).	Minimized (centralized and normalized).
Data Independence	Low (applications depend on file structures).	High (abstracts physical storage).
Querying	Difficult (requires custom procedural code).	Easy (declarative languages like SQL).
Concurrency	Poor (file-level locking limits multi-user access).	Excellent (row-level locking and isolation protocols).
Data Integrity	Hard-coded in external applications.	Centralized enforcement via schema constraints.
Security	Coarse OS-level permissions (Read/Write file).	Granular role-based access control (RBAC).

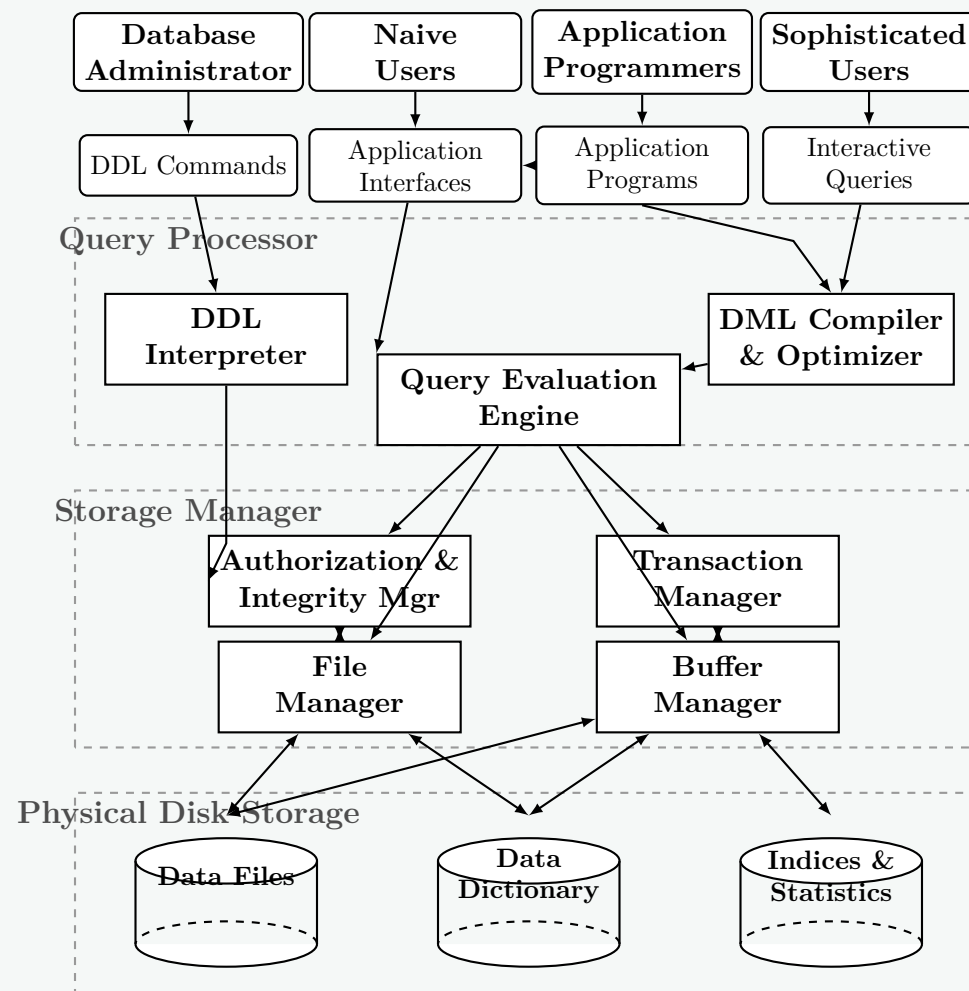
Answer

1. Overview of DBMS Architecture

A Database Management System (DBMS) is a complex software system tasked with managing, storing, and retrieving data efficiently and securely. To handle these responsibilities, the architecture of a DBMS is partitioned into distinct functional modules. The overall block diagram is primarily divided into three major layers:

- **Users and Interfaces:** The top layer where various users interact with the system.
- **Query Processor:** The brain of the DBMS that translates high-level user queries into low-level execution plans.
- **Storage Manager:** The interface between the query processor and the physical disk storage, responsible for efficient data handling and concurrency.

2. DBMS Block Diagram



3. Description of Major Components

A. Users and Interfaces

- **Database Administrator (DBA):** Interacts using DDL (Data Definition Language) to define the database schema, grant permissions, and configure storage.
- **Naive Users:** Interact through graphical application interfaces (e.g., banking tellers) without knowing underlying queries.
- **Application Programmers:** Write application code interacting with the database using embedded DML (Data Manipulation Language).
- **Sophisticated Users:** Use interactive query tools (like SQL Plus or pgAdmin) to directly request data.

B. Query Processor

- **DDL Interpreter:** Parses DDL statements and updates the system metadata (Data Dictionary) with schema definitions.
- **DML Compiler & Optimizer:** Translates DML queries into an evaluation plan. The **optimizer** determines the most efficient way to execute a query based on disk access costs and available indices.
- **Query Evaluation Engine:** Executes the low-level instructions generated by the compiler against the storage manager.

C. Storage Manager

- **Authorization & Integrity Manager:** Ensures the user has the required privileges to execute the query and checks that no integrity constraints (like primary/foreign keys) are violated.
- **Transaction Manager:** Ensures the database remains in a consistent state (ACID properties) despite system failures and manages concurrent transaction executions.

- **File Manager:** Manages the allocation of space on the disk storage and the data structures used to represent information stored on disk.
- **Buffer Manager:** Responsible for caching data in main memory (RAM). It decides what disk pages to bring into memory and what pages to evict, minimizing slow disk I/O.

D. Disk Storage (Data Structures)

- **Data Files:** Store the actual database records.
- **Data Dictionary (Metadata):** Stores data about the database structure, schemas, user permissions, and constraints.
- **Indices:** Specialized data structures (e.g., B+ Trees) that speed up the retrieval of specific records without sequentially scanning the entire data file.

Q10. Discuss ‘Schema versus Data’ and ‘DDL versus DML’. (7 marks)

[CSE 215 | L-3/T-1 | 2010–11]

Answer

1. Schema versus Data

In database design, it is critical to distinguish between the logical description of the database and the actual information it contains. This distinction is broadly referred to as **Schema** versus **Data** (or Instance).

- **Database Schema (Intension):** The overall design, structure, and logical blueprint of the database. It is defined during the database design phase and is expected to remain relatively static. It specifies the tables, attributes, data types, constraints, and relationships.
- **Database Data / Instance (Extension):** The actual collection of data stored in the database at a specific moment in time. The instance changes continuously as records are inserted, updated, or deleted during routine operations.

Analogy: A schema is like the architectural blueprint of a house (showing where the rooms and doors are), while the data/instance represents the actual furniture and people residing in the house at any given time.

Schema (Structure / Intension)

EMPLOYEE (Emp_ID INT, Name VARCHAR, Salary REAL)

Data / Instance (Records / Extension)

Emp_ID	Name	Salary
101	Alice	75000
102	Bob	68000
103	Carol	82000

Populates, operates over time

Feature	Database Schema	Database Data (Instance)
Definition	The logical structure and rules of the database.	The actual content stored within the structure.
Frequency of Change	Rarely changes (static).	Changes constantly (dynamic).
Metadata	Stored in the database catalog (Data Dictionary).	Stored in the data files on the disk.
Terminology	Intension.	Extension.

2. DDL versus DML

To interact with both the schema and the data, a DBMS provides specific categories of SQL languages: **Data Definition Language (DDL)** and **Data Manipulation Language (DML)**.

- **DDL (Data Definition Language):** Used by database designers and administrators to specify, modify, or drop the **Database Schema**. Execution of DDL statements directly affects the Data Dictionary and is typically auto-committed (cannot be rolled back in standard relational systems).
 - *Common Commands:* CREATE, ALTER, DROP, TRUNCATE.
- **DML (Data Manipulation Language):** Used by users and applications to retrieve, insert, delete, and modify the actual **Data (Instance)** stored within the tables. DML commands form the basis of transactional processing.
 - *Common Commands:* SELECT, INSERT, UPDATE, DELETE.

16

Example Code Demonstration

```
-- === DDL: Defining the Schema ===
CREATE TABLE Employee (
    Emp_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Salary DECIMAL(10,2)
);

-- === DML: Manipulating the Data / Instance ===
INSERT INTO Employee (Emp_ID, Name, Salary) VALUES (101, 'Alice', 75000);
UPDATE Employee SET Salary = 80000 WHERE Emp_ID = 101;
SELECT * FROM Employee;
```

Feature	DDL (Data Definition Language)	DML (Data Manipulation Language)
Target	Operates on the Schema (metadata).	Operates on the Data (instances/records).
Primary Users	Database Administrators (DBA), Database Designers.	End Users, Application Programs.
Transaction Control	Usually auto-committed (cannot be rolled back).	Controlled by transaction blocks (can be rolled back).
Effect	Alters the structure (e.g., adding a column).	Alters the values (e.g., changing a salary).

Q11. Explain the data dictionary schema for DBMS. List the advantages and disadvantages for storing a relational database as follows:

- (a) Store each relation in one file.
- (b) Store multiple relations in one file.

(10 marks) [CSE 215 | L-3/T-1 | 2010–11]

Answer

1. Data Dictionary Schema for DBMS

Definition: The data dictionary (also known as the system catalog) is an internal database maintained by the DBMS to store **metadata**—that is, "data about the data." It acts as a comprehensive directory detailing the logical and physical structure of the entire database system.

Working Principle: The DBMS itself accesses the data dictionary constantly. Before executing any query (DML or DDL), the Query Processor consults the data dictionary to verify that relations exist, attributes match, users have appropriate permissions, and to identify available indices for query optimization.

Contents of a Data Dictionary: To manage this metadata, the DBMS stores the data dictionary as a set of relational tables. The schema of these system tables typically includes:

- **Relation Information:** Names of all tables, views, their creators, and physical storage paths.
- **Attribute Information:** Names of attributes for each table, their data types, lengths, and constraints (e.g., NOT NULL).
- **Index Information:** Index names, the attributes they index, and the type of index (e.g., B+ tree, Hash).
- **Integrity Constraints:** Primary keys, foreign key references, and check constraints.
- **User and Authorization Data:** Usernames, passwords, and access privileges (Grants).
- **Statistical Data:** Number of tuples in a relation, distribution of values, used heavily by the Query Optimizer.

Visual Representation of a Simplified Data Dictionary Schema

RELATION_METADATA

<u>Rel_Name</u>	Creator	File_Path	Tuple_Count
-----------------	---------	-----------	-------------

ATTRIBUTE_METADATA

<u>Attr_Name</u>	<u>Rel_Name</u> (FK)	Data_Type	Length
------------------	----------------------	-----------	--------

INDEX_METADATA

<u>Index_Name</u>	<u>Rel_Name</u> (FK)	Attr_Name	Index_Type
-------------------	----------------------	-----------	------------

2. Storing the Relational Database: File Strategies	
A DBMS needs a mapping between logical relations (tables) and physical operating system files. Two primary strategies exist, each with specific trade-offs.	
Strategy 1: Store each relation in one separate file	
In this approach, every table in the database maps directly to its own isolated file on the disk (e.g., <code>Employee</code> maps to <code>employee.dat</code> , <code>Department</code> to <code>department.dat</code>).	
Each Relation in One File	
Advantages	• Simplicity: The mapping between the database conceptual schema and physical files is extremely straightforward. • OS-Level Security: The DBMS can leverage underlying Operating System file-level permissions to restrict access to specific tables. • Easy Maintenance: Backing up, copying, or moving individual tables is very simple (just copy the specific file). • No Mixed-Record Overhead: Because a file contains only one type of record (all the same length or structure), calculating record offsets and managing free space is mathematically simpler.
Disadvantages	• OS Bottlenecks: If a database contains thousands of tables, it results in thousands of files. Operating systems impose hard limits on the maximum number of simultaneously open file descriptors. • Join Performance Penalty: Executing a <code>JOIN</code> operation between two relations requires the OS to open, read, and buffer multiple separate files concurrently, increasing disk I/O seek times and overhead. • Internal Fragmentation: Small tables might waste disk space because the minimum allocation unit is a full OS disk block.
Strategy 2: Store multiple relations in one file (Clustering)	
In this approach, the DBMS consolidates multiple relations—especially those that are frequently accessed together—into a single large, contiguous physical file (often referred to as a multitable clustering file organization).	
Multiple Relations in One File	
Advantages	• Optimized Joins (Clustering): Related records from different tables can be stored physically adjacent on the same disk block. For example, a <code>Department</code> record and all its associated <code>Employee</code> records can be read in a single disk I/O fetch, dramatically speeding up <code>JOIN</code> queries. • Overcoming OS Limits: The DBMS uses very few OS files (sometimes just one massive file), completely avoiding OS limitations on the number of open file descriptors. • Space Efficiency: Better packing of blocks reduces internal fragmentation, as small tables share blocks with other tables.
Disadvantages	• High Complexity: The DBMS must build and maintain its own internal file system and buffer manager logic to track where different table records reside within the single file. • Complex Space Reclamation: When records of varying sizes from different tables are deleted, reclaiming the fragmented free space within the file is algorithmically difficult. • Sequential Scan Penalty: Performing a sequential scan on a single table (<code>SELECT * FROM Employee</code>) becomes much slower. The DBMS must read the entire massive file and explicitly filter out the intermingled records belonging to other tables.

Relational Model

Q1. Define super key, candidate key, and primary key with examples. (9 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Answer
1. Introduction and Running Example
In the relational data model, keys are fundamental constraints used to uniquely identify tuples (rows) within a relation (table) and to establish relationships between different relations. To illustrate these concepts, consider the following relation schema for a university system:

STUDENT(Roll_No, Reg_No, Email, Name, Age, Branch)

Assumption: Roll_No, Reg_No (Registration Number), and Email are strictly unique for every student. Two students may share the same Name, Age, or Branch.

2. Definitions and Mathematical Properties

A. Super Key

- **Definition:** A Super Key is a set of one or more attributes whose combined values uniquely identify a tuple within a relation.
- **Mathematical Property (Uniqueness):** Let R be a relation schema. A subset of attributes $K \subseteq R$ is a super key if the functional dependency $K \rightarrow R$ holds.
- **Examples:**
 - {Roll_No}
 - {Reg_No, Name}
 - {Email, Age, Branch}
 - {Roll_No, Reg_No, Email, Name, Age, Branch} (The trivial super key comprising all attributes).
- **Note:** Adding any arbitrary attribute to a super key results in another valid super key.

B. Candidate Key

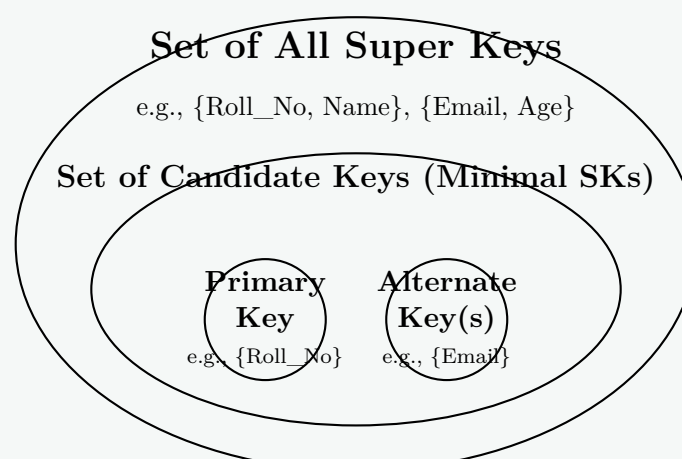
- **Definition:** A Candidate Key is a **minimal** super key. It is a super key such that no proper subset of its attributes also constitutes a super key.
- **Mathematical Property (Minimality):** K is a candidate key if K is a super key, and for any proper subset $K' \subset K$, K' is *not* a super key.
- **Examples:**
 - {Roll_No} (Minimal, removing it leaves the empty set).
 - {Reg_No}
 - {Email}
- *Why is {Reg_No, Name} NOT a Candidate Key?* Because its proper subset {Reg_No} is already a super key. Thus, it violates the minimality condition.

C. Primary Key

- **Definition:** The Primary Key is the specific candidate key selected by the database designer to principally identify tuples in the relation.
- **Properties:**
 - A relation can have **only one** primary key.
 - Its attributes cannot contain NULL values (Entity Integrity Constraint).
 - Its values should ideally remain immutable (unchanging) over time.
- **Example:** From the candidate keys {Roll_No}, {Reg_No}, and {Email}, the designer selects {**Roll_No**} as the Primary Key.
- **Note:** The unselected candidate keys ({Reg_No} and {Email}) are then referred to as **Alternate Keys**.

3. Visualizing the Relationship

The relationship between these keys can be accurately represented using set theory. All candidate keys are super keys, and the primary key is exactly one of the candidate keys.



4. Comparison Summary

Feature	Super Key	Candidate Key	Primary Key
Definition	Uniquely identifies a tuple.	A minimal super key.	The chosen candidate key.
Quantity	Many per relation.	One or more per relation.	Exactly one per relation.
Minimality	Not required.	Strictly required.	Strictly required.
Null Values	May contain NULLs (if other attributes ensure uniqueness).	Cannot contain NULLs (in practice).	Strictly prohibits NULLs.
Subset Relation	The superset of all keys.	A subset of Super Keys.	A subset of Candidate Keys.

Q2. Suppose we have two relations A and B with the same set of attributes. Relation A has n_a tuples and relation B has n_b tuples. For each of the following relational algebra expressions, give the minimum and maximum number of tuples that can appear in the result. Assume that the produced results are bags, not sets.

- (a) $A \cap B$
- (b) $\sigma_c(A) - B$
- (c) $A \cup B$
- (d) $A \bowtie_L B$, where $\bowtie_L$ is the left outer join

(8 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

Bag Semantics Overview

In relational algebra, standard operations often assume **set semantics** (where duplicates are eliminated). However, real-world Database Management Systems (DBMS) typically use **bag (multiset) semantics**, where duplicates are retained unless explicitly removed.

Let a tuple t appear x times in relation A (where $\sum x = n_a$) and y times in relation B (where $\sum y = n_b$). The output cardinality depends on how the specific bag operation computes the new frequency for each tuple.

1. Intersection ($A \cap B$)

- **Working Principle:** Under bag semantics, the intersection retains a tuple t exactly $\min(x, y)$ times.
- **Minimum Tuples: 0.** This occurs when relations A and B are completely disjoint (they share no tuples in common).
- **Maximum Tuples: $\min(n_a, n_b)$.** This occurs when all tuples of the smaller relation are entirely contained within the larger relation.

2. Selection and Difference ($\sigma_c(A) - B$)

- **Working Principle:** First, the selection $\sigma_c(A)$ filters A . The number of tuples generated by $\sigma_c(A)$ can range from 0 to n_a . Next, the bag difference $X - B$ computes the frequency of tuple t as $\max(0, \text{count}(t \in X) - y)$.
- **Minimum Tuples: 0.** This occurs if the selection condition c matches no tuples in A (yielding 0 tuples), OR if all tuples generated by $\sigma_c(A)$ also exist in B in equal or greater quantities, causing them to be entirely subtracted out.
- **Maximum Tuples: n_a .** This occurs if the selection condition c is true for all tuples in A (yielding n_a tuples), AND relations A and B are completely disjoint, meaning no tuples are subtracted by B .

3. Union ($A \cup B$)

- **Working Principle:** Under bag semantics, union does **not** eliminate duplicates (equivalent to **UNION ALL** in SQL). The frequency of a tuple t in the result is simply $x + y$.
- **Minimum Tuples: $n_a + n_b$.**
- **Maximum Tuples: $n_a + n_b$.**
- **Note:** Unlike set union (which can range from $\max(n_a, n_b)$ to $n_a + n_b$), bag union always yields exactly the sum of the cardinalities of the two input relations.

4. Left Outer Join ($A \bowtie_L B$)

- **Working Principle:** Because A and B have the *same set of attributes*, the join condition evaluates all attributes for equality (Natural Left Outer Join).
 - If a tuple in A finds matching tuples in B ($y > 0$), it produces $x \times y$ copies.
 - If it finds no match ($y = 0$), the Left Outer Join ensures the tuple from A is preserved exactly x times, padded with NULLs (though here, all attributes overlap).

Mathematically, the total tuples are $\sum(x \cdot \max(y, 1))$.

- **Minimum Tuples:** n_a . This occurs either when A and B are completely disjoint (every tuple in A gets preserved exactly once with no matches), or when every tuple in A matches exactly one tuple in B .
- **Maximum Tuples:** $\max(n_a, n_a \times n_b)$. If $n_b > 0$, the maximum is $n_a \times n_b$. This explosion occurs when every tuple in A is identical, and every tuple in B is also identical to the tuples in A , resulting in a full Cartesian product. If B is empty ($n_b = 0$), the maximum is simply n_a .

Summary Table

Relational Algebra Expression	Minimum Tuples	Maximum Tuples
$A \cap B$	0	$\min(n_a, n_b)$
$\sigma_c(A) - B$	0	n_a
$A \cup B$	$n_a + n_b$	$n_a + n_b$
$A \bowtie_L B$	n_a	$\max(n_a, n_a \cdot n_b)$

Q3. Should a primary key have any semantic meaning? Explain your answer. (6 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Answer

1. Direct Answer

Ideally, **no**. A primary key should generally **not** have any semantic (real-world business) meaning. In database design, keys are divided into two categories based on meaning:

- **Natural Key (Semantic):** An attribute that has a logical relationship to the business data (e.g., Social Security Number, Email Address, Vehicle Identification Number).
- **Surrogate Key (Meaningless):** An artificially generated attribute with no real-world meaning, created solely to identify a tuple uniquely (e.g., an Auto-Incrementing Integer, UUID/GUID).

Best practices in modern database design strongly favor the use of **Surrogate Keys** for primary keys, while applying **UNIQUE** constraints to Natural Keys.

2. Why Semantic Primary Keys are Problematic

Using a semantic attribute as a primary key introduces several risks to the stability and performance of a relational database:

- (a) **Mutability (Data Changes):** Semantic data is tied to business rules and human behavior, meaning it is subject to change. For example, if an **Email** is used as a primary key and the user changes their email address, the database must perform a cascading update to every foreign key in every related table (e.g., Orders, Logs, Profiles). A surrogate key, being meaningless, never needs to change.
- (b) **Format and Business Rule Changes:** Business rules evolve. A 9-digit alphanumeric product code might be expanded to 12 digits. If this code is a primary key, altering the data type or length across millions of foreign key references is an expensive and risky operation.
- (c) **Reusability:** Some semantic identifiers get recycled over time (e.g., phone numbers or invoice numbers). Reusing a primary key can lead to catastrophic data integrity issues where old, unrelated records accidentally link to new entities.
- (d) **Privacy and Security:** Primary keys are frequently exposed in URLs, log files, and API endpoints. Using sensitive semantic data (like an SSN or phone number) as a primary key leaks personally identifiable information (PII).
- (e) **Performance (Storage and Indexing):** Semantic keys are often long strings (like **VARCHAR(255)** for an email). String comparisons are significantly slower than integer comparisons. Furthermore, B+ Tree indexes built on large strings consume more memory and disk space. A surrogate key, typically a 4-byte or 8-byte integer, is highly optimized for fast **JOIN** operations.

3. Visualizing the Recommended Approach

Instead of forcing a semantic attribute to act as the primary key, database designers implement a surrogate primary key and designate the semantic attributes as **Alternate Keys** (using the **UNIQUE** constraint).

Suboptimal Design: Semantic Primary Key

User_Email (PK)	First_Name	Last_Name
-----------------	------------	-----------

Recommended Design: Meaningless Surrogate Key

User_ID (PK)	User_Email (UNIQUE)	First_Name	Last_Name
--------------	---------------------	------------	-----------

4. Comparison: Natural (Semantic) vs. Surrogate (Meaningless) Keys

Feature	Natural Key (Semantic)	Surrogate Key (Meaningless)
Origin	Exists in the real world (e.g., SSN).	System-generated (e.g., Auto-increment).
Immutability	Prone to change if business rules change.	100% Immutable. Never changes.
Performance	Often slow (e.g., long strings).	Very fast (e.g., 32/64-bit integers).
Coupling	Tightly couples DB structure to business rules.	Decouples structure from business logic.
Use Case	Enforce uniqueness (Alternate Key).	Primary Key / Foreign Key relationships.

Notes: While surrogate keys are generally preferred, there is one major exception: **Link Tables** (Junction Tables) used to resolve Many-to-Many relationships. In a link table (e.g., `Enrollment(Student_ID, Course_ID)`), the primary key should be the composite of the two foreign keys, which intrinsically carries semantic meaning regarding the relationship.

Q4. Describe with an example, the object identity and reference types in SQL. How can you process queries using reference type? (10 marks) [CSE 215 | L-3/T-1 | 2010–11]

Answer

1. Object Identity in SQL

In standard relational databases, tuples (rows) are identified purely by their values, specifically through a **Primary Key**. However, in **Object-Relational Database Management Systems (ORDBMS)** complying with the SQL:1999 standard and later, tuples can be treated as unique objects.

- **Object Identity (OID):** When a row is inserted into a “typed table” (a table built upon a user-defined complex data type), the DBMS assigns it a unique, system-generated, and immutable Object Identifier (OID).
- **Properties of OID:** Unlike primary keys, an OID has no real-world semantic meaning, does not depend on the object’s attribute values, and never changes even if all the data within the row is updated.

2. Reference Types (REF)

A **Reference Type** is a special data type in SQL that stores an OID. It acts essentially as a strongly-typed **pointer** to a specific object (row) in another typed table.

- **Working Principle:** Instead of using standard Foreign Keys that store copies of Primary Key values (which requires traditional joins), a REF attribute directly points to the memory/disk location of the referenced object, enabling highly efficient object traversal.
- **Scoping:** A reference type is usually constrained using a SCOPE clause to ensure it only points to objects within a specific target table.

3. Example Scenario: DDL for Objects and References

Consider a university system where an **Employee** works for a **Department**. First, we define the object types, and then we create the typed tables.

```
-- 1. Create the base User-Defined Types (UDT)
CREATE TYPE DepartmentType AS (
    Dept_No INT,
    Dept_Name VARCHAR(50)
```

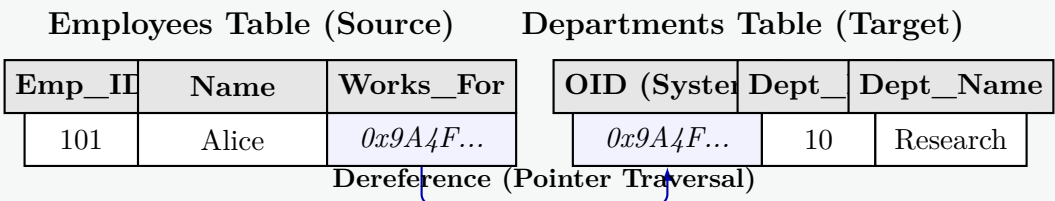
```
);

CREATE TYPE EmployeeType AS (
    Emp_ID INT,
    Name VARCHAR(50),
    Works_For REF(DepartmentType) -- The Reference Type
);

-- 2. Create Typed Tables
CREATE TABLE Departments OF DepartmentType (
    REF IS Dept_OID SYSTEM GENERATED, -- Explicitly generating OIDs
    PRIMARY KEY (Dept_No)
);

CREATE TABLE Employees OF EmployeeType (
    PRIMARY KEY (Emp_ID),
    SCOPE FOR Works_For IS Departments -- Restricting pointer target
);
```

4. Visualizing Object References



5. Query Processing Using Reference Types

To process queries involving reference types, standard JOIN operations can be entirely bypassed. Instead, object-relational SQL introduces **Path Expressions** and **Dereferencing**.

A. Dereferencing Operator (->)

The -> operator takes a REF value and retrieves the object it points to, allowing direct access to the target object’s attributes.

```
-- Retrieve employee names and the name of the department they work for.
-- Notice the absence of a JOIN clause.
SELECT e.Name,
       e.Works_For->Dept_Name AS Department_Name
FROM Employees e;
```

B. The Deref() Function

If the goal is to return the entire referenced object (tuple) rather than just a specific attribute, the Deref() function is utilized.

```
-- Retrieve the full Department object for the employee named Alice
SELECT Deref(e.Works_For) AS Dept_Object
FROM Employees e
WHERE e.Name = 'Alice';
```

6. Advantages of Object References

- **Performance (Implicit Joins):** Traversing pointers is computationally faster than performing hash-joins or sort-merge joins on standard foreign keys.
- **Simpler Query Syntax:** Complex, multi-table relationships can be queried cleanly using cascading path expressions (e.g., a->b->c) without cluttering the FROM clause.
- **Polymorphism:** References can be designed to point to an object of a super-type or any of its sub-types, enabling polymorphic queries.

Entity-Relationship (ER) Model

Q1. Differentiate between strong entity and weak entity by showing appropriate examples. How do you design them in a relational schema? (10 marks)

Answer

1. Distinction Between Strong and Weak Entities

In Entity-Relationship (ER) modeling, entities are categorized based on their ability to be uniquely identified strictly by their own attributes.

- Strong Entity:** An entity type that possesses a **Primary Key**. It has an independent existence in the database and does not rely on any other entity for its identification.
- Weak Entity:** An entity type that does **not** have sufficient attributes to form a primary key. It depends on a strong entity (called the **Identifying Owner**) for its existence and identification. It uses a **Partial Key** (or Discriminator) combined with the primary key of its owner to uniquely identify its instances.

Comparison Table

Feature	Strong Entity	Weak Entity
Primary Key	Has its own primary key.	Does not have a primary key; has a partial key.
Existence	Independent existence.	Existence depends on the identifying strong entity.
Relationship	Associated via standard relationships.	Associated via an identifying relationship .
Participation	Can have total or partial participation.	Always has total participation in the identifying relationship.
ER Diagram	Represented by a single rectangle.	Represented by a double rectangle.

2. Example and ER Diagram Representation

Example Scenario: Consider a company database storing information about EMPLOYEES and their DEPENDENTS (spouses, children).

- EMPLOYEE is a **Strong Entity** uniquely identified by Emp_ID.
- DEPENDENT is a **Weak Entity**. A dependent's name (Dep_Name) only identifies them uniquely *within* the context of a specific employee. It cannot uniquely identify a dependent across the entire company.

3. Design in a Relational Schema

To convert a strong and weak entity relationship into a relational database schema, apply the following mapping rules:

- Strong Entity Mapping:** Create a separate table for the strong entity. Its attributes become columns, and its key attribute becomes the **Primary Key (PK)**.
- Weak Entity Mapping:** Create a separate table for the weak entity. Include all of its own attributes. Then, add the Primary Key of the identifying strong entity as a **Foreign Key (FK)**.
- Weak Entity Primary Key:** The Primary Key of the weak entity table is a **Composite Key** formed by combining its Foreign Key and its Partial Key.

Relational Schema Diagram

** Note: The primary key of DEPENDENT is composite: (Emp_ID, Dep_Name).*

Exam Tip: When mapping a weak entity, you do **not** create a separate relation (table) for the identifying relationship (e.g., the "Has" diamond). The relationship is implicitly managed by the foreign key placed inside the weak entity's table.

Q2. Define weak entity set. Also give an example of a weak entity set. (5 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Answer

1. Definition

A **weak entity set** is an entity type that does not contain sufficient attributes to uniquely identify its own entities (i.e., it lacks a primary key of its own). To be uniquely identified, a weak entity must be associated with another entity set, known as the **identifying entity set** (or strong entity set).

2. Key Characteristics

- **Existence Dependency:** A weak entity cannot exist in the database without its corresponding strong entity. If the strong entity is deleted, all its associated weak entities must also be deleted (cascade delete).
- **Identifying Relationship:** The relationship associating the weak entity set with the strong entity set is called the identifying relationship. In an Entity-Relationship (ER) diagram, it is denoted by a **double diamond**.
- **Total Participation:** The weak entity set always exhibits total participation (denoted by a **double line**) in the identifying relationship, meaning every weak entity must be related to an identifying entity.
- **Partial Key (Discriminator):** Instead of a primary key, a weak entity set has a **partial key** (or discriminator). This is an attribute (or set of attributes) that uniquely identifies weak entities related to the *same* strong entity. It is denoted by a **dashed underline** in an ER diagram.
- **Primary Key Formation:** The actual primary key of a weak entity is a composite key formed by combining the primary key of the strong entity set with the weak entity set's partial key.

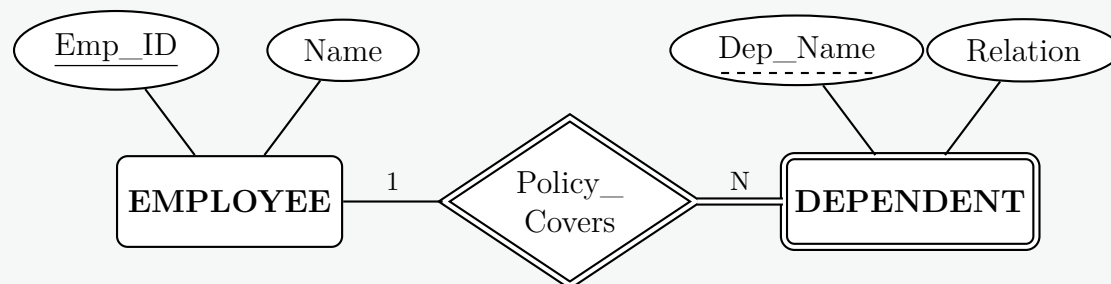
3. Example Scenario

Consider a company database that stores information about employees and their dependents (e.g., spouses, children) for health insurance purposes.

- **Strong Entity Set:** EMPLOYEE (uniquely identified by Emp_ID).
- **Weak Entity Set:** DEPENDENT. A dependent's name (Dep_Name) is not globally unique across the whole company, but it is unique for a specific employee.
- **Identifying Relationship:** Policy_Covers.

In this context, a dependent only exists in the database if their associated employee exists.

4. ER Diagram Representation



Exam Tip (Relational Mapping): When mapping this ER model to a relational database schema, you do not create a separate table for the identifying relationship. Instead, the resulting DEPENDENT table will simply have the composite primary key (Emp_ID, Dep_Name), where Emp_ID acts as a foreign key referencing the EMPLOYEE table.

Q3. Show an example of specialization hierarchy in the context of ERD with disjoint and overlapping subtypes. (9 marks) [CSE 303 | L-3/T-1 | 2014–15]

Answer

1. Specialization Hierarchy and Constraints

Specialization is the top-down design process of defining a set of subclasses (subtypes) from a single superclass (supertype) based on distinguishing characteristics. The entities in the subtypes inherit all attributes and relationships of the supertype.

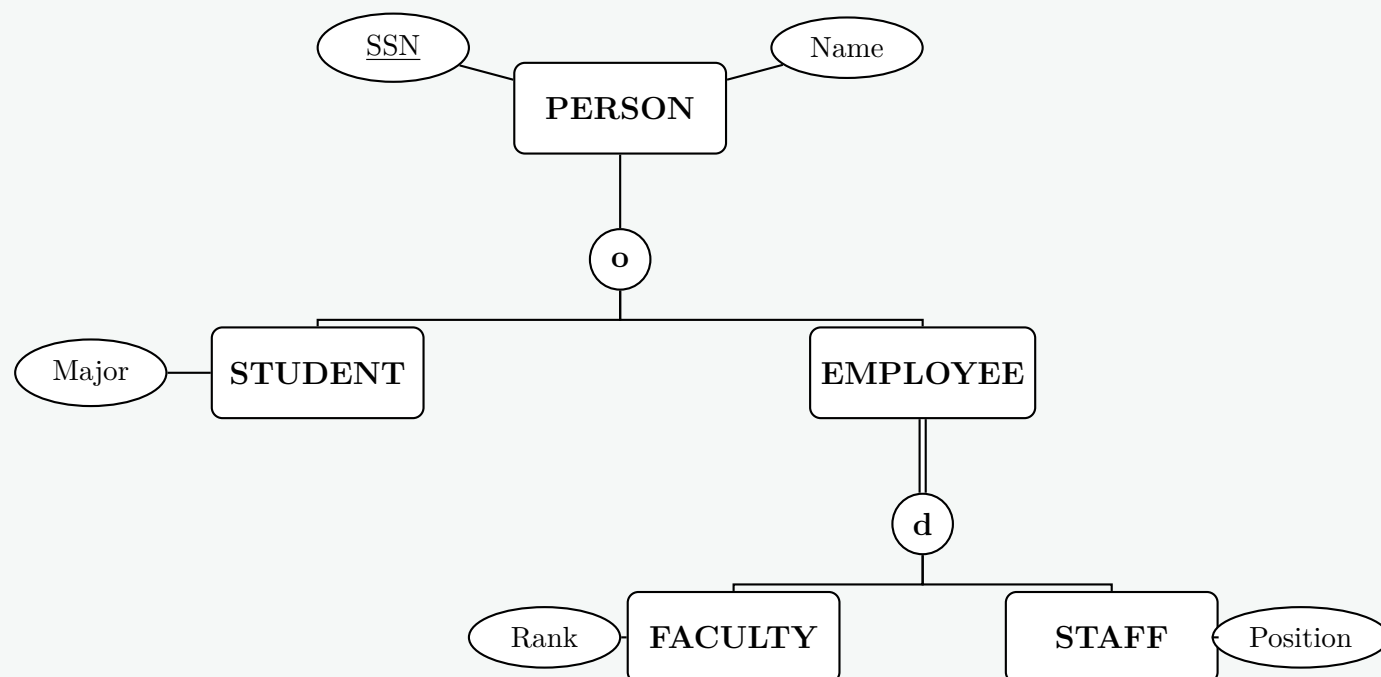
Specialization hierarchies are governed by **disjointness constraints**, which dictate whether an entity can belong to more than one subclass simultaneously:

- **Disjoint Subtypes (denoted by 'd'):** An entity instance of the supertype can belong to **at most one** subtype. The subtypes are mutually exclusive.
- **Overlapping Subtypes (denoted by 'o'):** An entity instance of the supertype can belong to **more than one** subtype simultaneously.

2. Example Scenario and ER Diagram

Consider a university database containing a general PERSON entity.

- A PERSON can be a STUDENT, an EMPLOYEE, or **both**. This is an **overlapping** specialization.
- An EMPLOYEE can be either a FACULTY member or a STAFF member, but **never both**. This is a **disjoint** specialization.
- Furthermore, every EMPLOYEE *must* be either FACULTY or STAFF (Total Specialization, denoted by a double line).



3. Key Observations from the Diagram

- Overlapping ('o'):** The circle containing 'o' connects PERSON to STUDENT and EMPLOYEE. Because it is overlapping, a specific SSN (e.g., a graduate teaching assistant) can exist simultaneously as a STUDENT and an EMPLOYEE.
- Disjoint ('d'):** The circle containing 'd' connects EMPLOYEE to FACULTY and STAFF. An employee is strictly categorized as one or the other; no individual can hold both a standard faculty and staff role concurrently in this schema.
- Completeness Constraints:**
 - The single line from PERSON to the 'o' circle indicates **Partial Specialization**. A PERSON might just be an Alumnus (not modeled here) and thus does not have to be a STUDENT or EMPLOYEE.
 - The double line from EMPLOYEE to the 'd' circle indicates **Total Specialization**. If a record exists in EMPLOYEE, it *must* be present in either the FACULTY or STAFF subclass.

Q4. Which considerations should be applied while choosing the primary key of a relation? Discuss with example how the multi-valued attribute problem in database design can be solved. **(12 marks)** [CSE 303 | L-3/T-1 | 2013–14]

Answer

1. Considerations for Choosing a Primary Key

A **Primary Key (PK)** is a candidate key chosen by the database designer to uniquely identify tuples (rows) within a relation. When multiple candidate keys exist, the following considerations should be applied to select the most appropriate primary key:

- **Uniqueness and Non-Nullability (Entity Integrity):** The chosen attribute(s) must guarantee uniqueness across all tuples and must never accept NULL values.
- **Irreducibility (Minimality):** If the key is composite (consisting of multiple attributes), no attribute can be removed without destroying the uniqueness property.
- **Immutability (Stability):** The value of a primary key should rarely, ideally never, change over time. Frequent updates to a primary key cause cascading updates in foreign keys, degrading performance and risking data anomalies.
- **Simplicity and Size:** A single-attribute key is preferable to a composite key. Additionally, shorter, numeric data types (e.g., INT) are heavily preferred over long strings (e.g., VARCHAR) because primary keys are heavily indexed, and smaller keys lead to faster search operations and lower storage overhead for the B+ tree indexes.
- **Security and Privacy:** Avoid using sensitive or personally identifiable information (PII) such as Social Security Numbers (SSN), email addresses, or phone numbers as primary keys.
- **Surrogate vs. Natural Keys:** If no natural candidate key meets the above criteria, a **surrogate key** (a system-generated, artificial identifier like an auto-incrementing integer) should be introduced.

2. Solving the Multi-valued Attribute Problem

Definition: A **multi-valued attribute** is an attribute that can store multiple distinct values for a single entity instance simultaneously. For example, an employee might have multiple phone numbers or possess multiple skills.

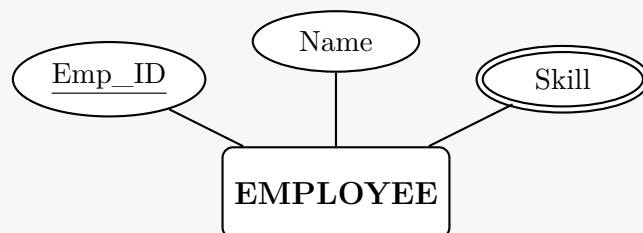
The Problem: Relational databases require relations to be in **First Normal Form (1NF)**, which strictly prohibits multi-valued attributes. Storing multiple values in a single column violates atomic domains, making data retrieval, querying, and updating highly inefficient and prone to anomalies.

The Solution: To resolve this in database design, the multi-valued attribute is removed from the original entity and placed into a **new, separate relation**.

- The new relation contains the multi-valued attribute itself.
- It also includes the primary key of the original entity as a **Foreign Key (FK)**.
- The **Primary Key** of this new relation is the composite of the original primary key and the multi-valued attribute.

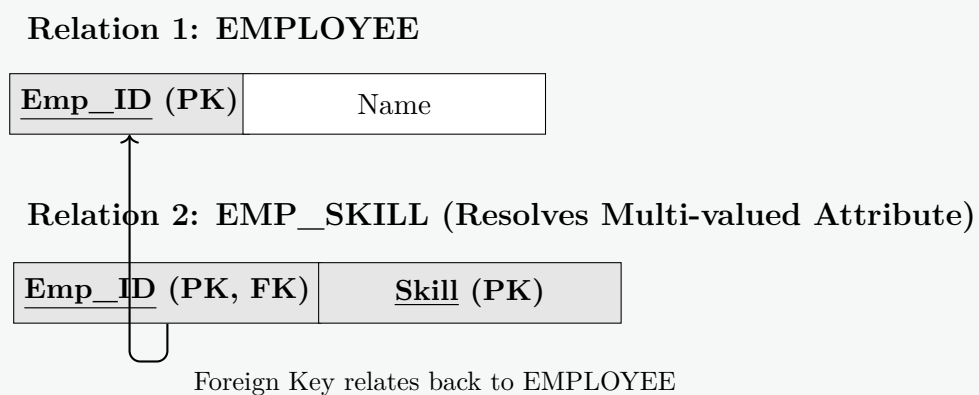
Example Scenario

Consider an **EMPLOYEE** entity that tracks the employee's ID, Name, and multiple **Skills**. In the ER Diagram, the multi-valued attribute **Skill** is represented by a double ellipse.



Relational Mapping (Decomposition)

To map this to a relational schema and conform to 1NF, we decompose the entity into two distinct tables: **EMPLOYEE** and **EMP_SKILL**.



Explanation of the Mapping:

- EMPLOYEE(Emp_ID, Name):** The original entity becomes a table, holding only the single-valued attributes. **Emp_ID** remains the Primary Key.
- EMP_SKILL(Emp_ID, Skill):** The multi-valued attribute **Skill** gets its own table. To link the skill to the correct employee, we import **Emp_ID** as a Foreign Key. Because one employee can have many skills, **Emp_ID** alone is not unique in this new table. Therefore, the Primary Key for **EMP_SKILL** is the composite key (**Emp_ID, Skill**).

Q5. What are the different elements of Entity-Relation Model? Briefly discuss them. **(15 marks)** [CSE 303 | L-3/T-1 | 2012–13]

Answer

Introduction to the ER Model

The **Entity-Relationship (ER) Model** is a high-level conceptual data model used in database design to represent the logical structure of a database. It abstracts the real-world system into a collection of basic objects and the associations among them. The three fundamental elements of the ER model are **Entities**, **Attributes**, and **Relationships**.

1. Entities and Entity Sets

An **entity** is a real-world object or concept that exists independently and can be uniquely identified (e.g., a specific student, a specific car). An **entity set** is a collection of entities of the same type.

- **Strong Entity:** An entity set that contains sufficient attributes to uniquely identify all its entities. It has a primary key and

is represented by a **single rounded rectangle**.

- **Weak Entity:** An entity set that does not have a primary key of its own and depends on another entity (the strong entity) for its existence. It is represented by a **double rounded rectangle**.

2. Attributes

Attributes are the properties or characteristics that describe an entity. They are represented by **ellipses**. Attributes can be classified into several types:

- **Simple Attribute:** Cannot be subdivided further (e.g., Gender, Salary).
- **Composite Attribute:** Can be divided into smaller subparts representing basic attributes (e.g., *Name* can be divided into *First Name*, *Middle Name*, and *Last Name*).
- **Single-valued Attribute:** Has only a single value for a particular entity (e.g., Date of Birth).
- **Multi-valued Attribute:** Can have multiple values for a single entity. Represented by a **double ellipse** (e.g., Phone Number, Skills).
- **Derived Attribute:** Its value is computed or derived from another related attribute or set of attributes. Represented by a **dashed ellipse** (e.g., *Age* derived from *Date of Birth*).
- **Key Attribute:** Uniquely identifies an entity within an entity set. Its name is **underlined**.

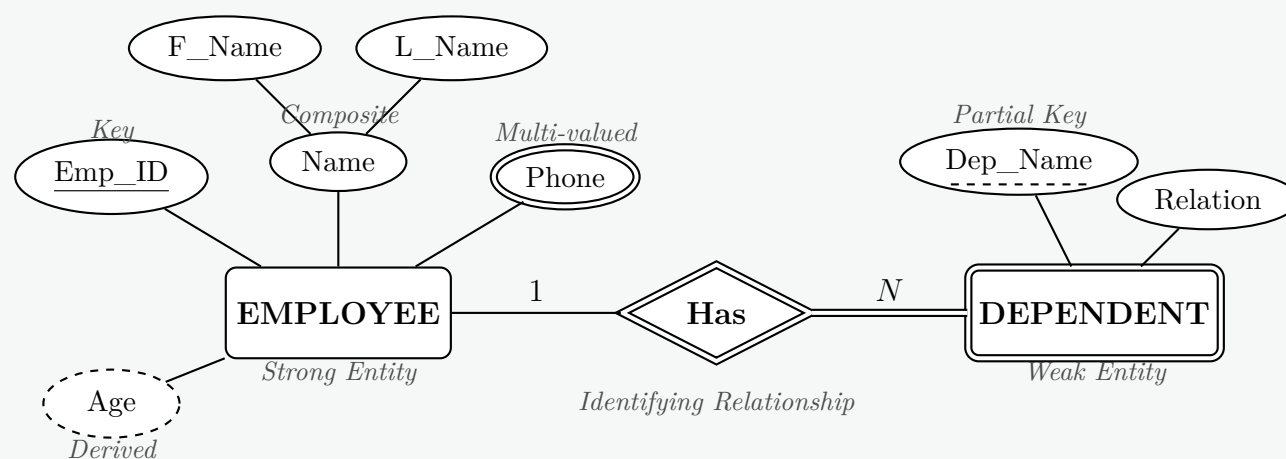
3. Relationships and Relationship Sets

A **relationship** is an association among two or more entities. A **relationship set** is a collection of similar relationships. They are represented by **diamonds**.

- **Degree of Relationship:** The number of participating entity types (e.g., Unary, Binary, Ternary).
- **Cardinality Ratio:** Expresses the maximum number of relationship instances an entity can participate in (e.g., 1 : 1 (One-to-One), 1 : *N* (One-to-Many), *M* : *N* (Many-to-Many)).
- **Participation Constraint:**
 - **Total Participation:** Every entity in the set must participate in at least one relationship instance. Represented by a **double line**.
 - **Partial Participation:** Only some entities in the set participate in the relationship. Represented by a **single line**.
- **Identifying Relationship:** The relationship associating a weak entity to its strong entity. Represented by a **double diamond**.

4. Visual Summary of ER Elements

The following Entity-Relationship diagram illustrates all the fundamental elements working together in a unified database schema.



Q6. What is a weak entity set? What is the requirement of a weak entity set? (10 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Answer

1. Definition of a Weak Entity Set

In the Entity-Relationship (ER) model, a **weak entity set** is an entity set that does not contain sufficient attributes to form a primary key of its own. Instead, it relies on another entity set—called the **strong entity set** (or owner entity set)—for its existence and identification.

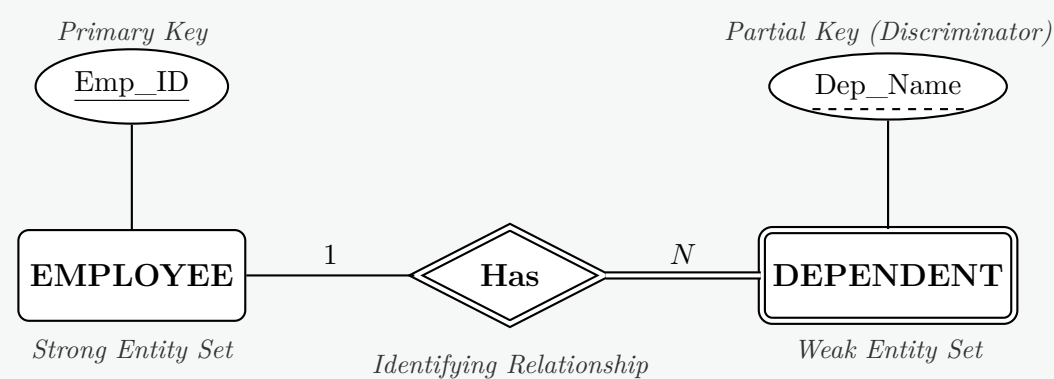
The primary key of a weak entity set is formed by a combination of the primary key of its strong entity set and its own **discriminator** (or partial key).

2. Requirements of a Weak Entity Set

- For a weak entity set to exist and function correctly in a database schema, the following requirements must be met:
- (a) **Existence Dependency (Owner Entity):** A weak entity cannot exist in the database without its corresponding strong entity. If the strong entity is deleted, all its associated weak entities must also be deleted (cascading delete).
 - (b) **Identifying Relationship:** The weak entity set must be associated with the strong entity set via a specific relationship known as the **identifying relationship**. In an ER diagram, this is represented by a double diamond.
 - (c) **Total Participation:** The weak entity set must exhibit **total participation** in the identifying relationship. This means every instance of the weak entity must participate in the relationship. It is represented by a double line connecting the weak entity to the identifying relationship.
 - (d) **Discriminator (Partial Key):** Although it lacks a full primary key, the weak entity set must possess a set of attributes that uniquely identifies its instances *among the instances related to a specific strong entity*. This attribute is called the partial key and is represented by a dashed underline in an ER diagram.

3. Visual Representation (ER Diagram)

Consider an organization where employees have dependents (e.g., children, spouse) for insurance purposes. A dependent cannot exist in the database without the employee, and two different employees might have dependents with the same name. Thus, **DEPENDENT** is a weak entity set owned by the **EMPLOYEE** strong entity set.



4. Comparison Summary

Feature	Strong Entity Set	Weak Entity Set
Primary Key	Has its own primary key.	Lacks a primary key; uses a partial key.
Existence	Exists independently.	Existence depends on a strong entity.
Representation	Single rounded rectangle.	Double rounded rectangle.
Relationship	Associated via regular relationships.	Associated via an identifying relationship (double diamond).

- Q7. Consider the following scenario of a hospital:
- Patients are treated in a single ward by the doctors assigned to them. Usually each patient will be assigned a single doctor, but in rare cases he/she will be assigned two doctors.
 - Healthcare assistants also attend to the patients; a number of these are associated with each ward.
 - Initially the system will be concerned solely with drug treatment. Each patient is required to take a variety of drugs a certain number of times per day and for varying lengths of time.
 - The system must record details concerning patient treatment and staff payment. Some staffs are paid on a part time basis and doctors and care assistants work varying amounts of overtime at varying rates (subject to grade).
 - The system will also need to track what treatments are required for which patients and when, and it should be capable of calculating the cost of treatment per week for each patient.

Define entities and their attributes for the hospital management system described above. (10 marks) [CSE 215 | L-3/T-1 | 2010–11]

Answer

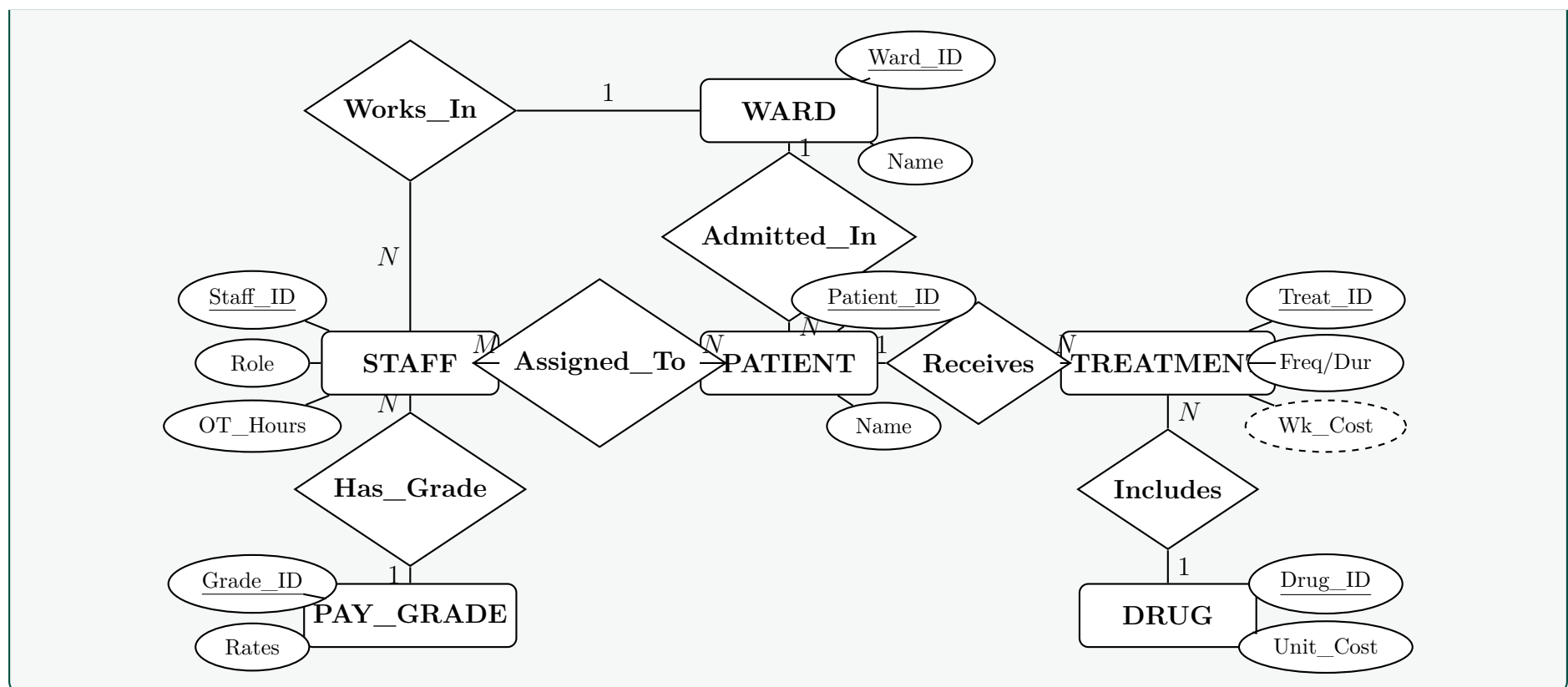
1. Entity Definitions and Attributes

Based on the hospital scenario provided, the system can be modeled using the following core entities and their attributes. **Primary Keys (PK)** are underlined and used to uniquely identify records, while **Foreign Keys (FK)** establish relationships between the entities.

- **WARD:** Represents the physical wards where patients are admitted.
 - **Ward_ID** (PK): Unique identifier for the ward.
 - **Ward_Name:** Name or designation of the ward.
 - **Capacity:** Maximum number of patients the ward can hold.
- **PATIENT:** Stores information regarding the individuals receiving care.
 - **Patient_ID** (PK): Unique identifier for the patient.
 - **Name:** Full name of the patient.
 - **Admission_Date:** Date the patient was admitted.
 - **Ward_ID** (FK): The specific ward to which the patient is admitted.
- **STAFF:** A generalized entity representing both doctors and healthcare assistants to streamline the payment and shift tracking system.
 - **Staff_ID** (PK): Unique identifier for the staff member.
 - **Name:** Full name of the staff member.
 - **Role:** Distinguishes between 'Doctor' and 'Healthcare Assistant'.
 - **Employment_Type:** Indicates if the staff is 'Full-time' or 'Part-time'.
 - **Overtime_Hours:** Tracks varying amounts of overtime worked.
 - **Grade_ID** (FK): Links to the pay grade for rate calculations.
- **PAY_GRADE:** Manages the varying payment rates subject to a staff member's grade.
 - **Grade_ID** (PK): Unique identifier for the pay grade.
 - **Basic_Hourly_Rate:** Standard rate of pay.
 - **Overtime_Rate:** Multiplier or specific rate for overtime hours.
- **DRUG:** Stores the inventory and pricing of treatments.
 - **Drug_ID** (PK): Unique identifier for the medication.
 - **Drug_Name:** Name of the drug.
 - **Unit_Cost:** The cost per single administration/pill of the drug.
- **TREATMENT:** An associative entity (or relationship entity) tracking the specific drug regimens assigned to patients.
 - **Treatment_ID** (PK): Unique identifier for the specific treatment plan.
 - **Patient_ID** (FK): The patient receiving the treatment.
 - **Drug_ID** (FK): The specific drug prescribed.
 - **Doctor_ID** (FK): The staff ID of the doctor who prescribed it.
 - **Frequency_Per_Day:** Number of times the drug must be taken daily.
 - **Duration_Days:** Varying length of time the drug is required.
 - **Weekly_Cost** (Derived): A calculated attribute resolving the system's need to compute treatment cost per week ($\text{Frequency_Per_Day} \times 7 \times \text{Unit_Cost}$).
- **PATIENT_DOCTOR_ASSIGNMENT:** Resolves the many-to-many relationship mapping between patients and their assigned doctors (as a patient can have 1 or 2 doctors).
 - **Patient_ID** (PK, FK)
 - **Staff_ID** (PK, FK)

2. Conceptual Entity-Relationship (ER) Diagram

The following diagram illustrates the core entities, their key attributes, and the cardinalities of their relationships based on the constraints described in the scenario.



ER Diagrams

Q1. You are the manager of the website development department of your organisation. Each website developed by your organization can be uniquely identified by a url. It also has a public host-ip address and a date of going live. Each website is developed by a team of developers. Each developer can be uniquely identified by an employee id. The relevant information for a developer is his/her name, address, highest educational qualification, salary and hire date. Each website is developed by a web-based framework which is associated with a particular programming language. Each framework and its associated programming language have a specific name and the development year. The other information relevant to a web framework is its author name.

Design an Entity-Relationship Diagram for your website development department to manage it conveniently. **(10 marks)** [CSE 215 | L-2/T-2 | 2018–19]

Answer

1. Entity Identification and Attributes

Based on the provided scenario for the website development department, we can identify four primary entities along with their respective attributes. **Primary Keys (PK)** are used to uniquely identify each record.

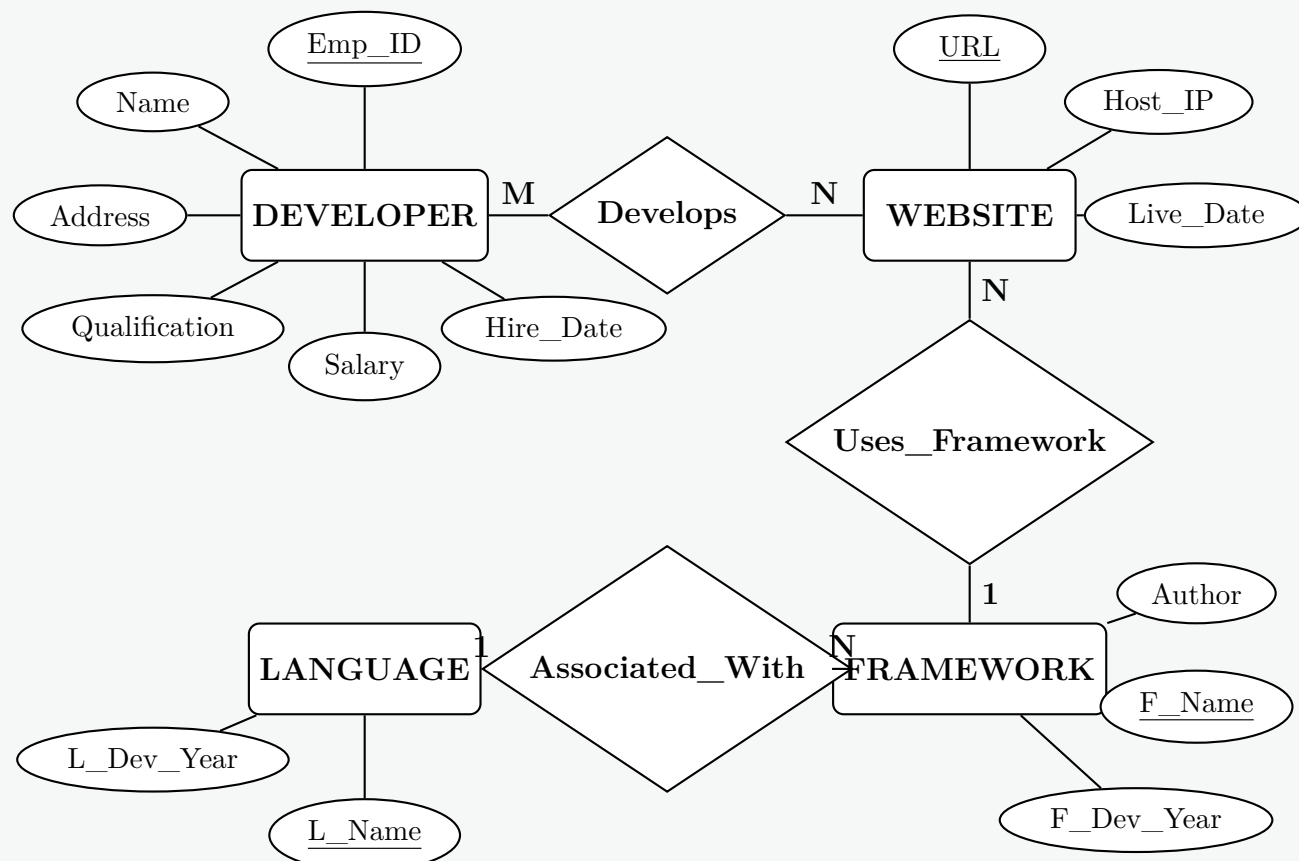
- **WEBSITE:** Stores details about the deployed projects.
 - **URL** (PK): The unique identifier for each website.
 - **Host_IP:** The public host-IP address.
 - **Live_Date:** The date the website went live.
- **DEVELOPER:** Stores professional and personal details of the staff.
 - **Emp_ID** (PK): The unique employee identifier.
 - **Name:** The full name of the developer.
 - **Address:** Residential address.
 - **Qualification:** Highest educational qualification.
 - **Salary:** Current compensation.
 - **Hire_Date:** Date of joining the organization.
- **FRAMEWORK:** Details the web-based frameworks utilized for development.
 - **F_Name** (PK): The specific name of the framework.
 - **F_Dev_Year:** The year the framework was developed.
 - **Author:** The author or creator of the framework.
- **LANGUAGE:** The programming language associated with the frameworks.
 - **L_Name** (PK): The specific name of the programming language.
 - **L_Dev_Year:** The development year of the language.

2. Relationships and Cardinalities

The entities interact through the following relationships:

- **Develops (DEVELOPER and WEBSITE):** A **Many-to-Many (M:N)** relationship. The scenario states a website is developed by a "team of developers" (meaning multiple developers per website), and a developer can generally work on multiple websites over time.
- **Uses_Framework (WEBSITE and FRAMEWORK):** A **Many-to-One (N:1)** relationship. Each website is developed using one specific web-based framework, but a single framework can be used to build many websites.
- **Associated_With (FRAMEWORK and LANGUAGE):** A **Many-to-One (N:1)** relationship. Each framework is associated with a *particular* (single) programming language, while a programming language (like Python or JavaScript) can have multiple frameworks associated with it (e.g., Django, Flask).

3. Entity-Relationship (ER) Diagram



Q2. You are designing a database for a university with the following requirements: Each student has a student id, name, address and contact name. Each course has a course number, name, credit hours, and the minimum level and term. Each student enrolls in several courses. University employees are either faculty members or office staff. Faculty members have a name, designation, address, contact number and room number. Office staff have a name, contact, address and hourly payment rate. In each semester (identified by a label e.g. "January 2019"), every faculty member conducts several courses; a single course can have many teachers. If a course is conducted by several teachers, one is assigned as coordinator.

- Write down the list of entity sets with their attributes.
- Draw the E/R diagram using appropriate arrows to indicate multiplicity. Show attributes on relations where applicable.

(20 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Answer

1. Entity Sets and their Attributes

Based on the scenario, we can identify the following entity sets and their attributes. A generalized superclass **EMPLOYEE** is introduced to hold the shared attributes of **FACULTY** and **OFFICE_STAFF**. Primary keys are underlined.

- **STUDENT:** Student_ID, Name, Address, Contact_Name.
- **COURSE:** Course_No, Name, Credit_Hours, Min_Level, Min_Term.
- **EMPLOYEE** (Superclass): Emp_ID (Assumed as a unique identifier), Name, Address, Contact_Number.
- **FACULTY** (Subclass of Employee): Designation, Room_Number.
- **OFFICE_STAFF** (Subclass of Employee): Hourly_Rate.

Relationship Sets & their Attributes:

- **Enrols:** Associates **STUDENT** and **COURSE**. (Multiplicity: Many-to-Many).
- **Conducts:** Associates **FACULTY** and **COURSE**. (Multiplicity: Many-to-Many).
 - *Attributes:* **Semester** (label indicating when it is taught), **Is_Coordinator** (Boolean flag indicating if the faculty is the assigned coordinator for that course).

2. Entity-Relationship (ER) Diagram

The following ER diagram uses standard Chen’s notation. The generalization (ISA) is represented by a circle linking the subclasses to the superclass.

The diagram illustrates the following entities and their attributes:

- STUDENT**: Student_ID, Name, Address, Contact_Name
- COURSE**: Course_No, Name, Credit_Hours, Min_Level, Min_Term
- EMPLOYEE**: Emp_ID, Name, Address, Contact_No
- FACULTY**: Designation, Room_No
- OFFICE_STAFF**: Hourly_Rate

Relationships and Generalization:

- Enrols** (M:N): Connects **STUDENT** and **COURSE**.
- Conducts** (M:1): Connects **COURSE** and **FACULTY**. Attributes: Semester, Is_Coordinator.
- ISA** (Generalization): Connects **EMPLOYEE** and **OFFICE_STAFF** to a central circle, indicating that **OFFICE_STAFF** is a specialization of **EMPLOYEE**.

- Q3.** There are many books in central library of BUET in different subjects. Each book must have a ISBN number, title, name of authors, subject and number of copies. A book can have multiple copies and each copy is identified by an accession number. The descriptive attributes of each copy of book are edition, price and data of purchase. A publisher publishes many books and a book is published by ofily one publisher. A publisher is identified by publisher id and described by name, country of origin, address and contact numbers. Book suppliers supply many copies of books to the library by tender in each year.- A tender has an id, name, total price and year. For each copy of books, you must store the record of which copy of book has been supplied by which supplier in which tender. A supplier is identified by supplier id and described by name, address and contact numbers. One copy of a book (book-copy) can be issued to multiple borrowers in different times and a borrower can borrow multiple number of book-copies. Borrowers can be teachers, students or staffs. Each borrower must have a borrower id, name, contact number and address. Teachers are described by joining date and designation. Students are described by session, level and term. Staffs are described by office name. When a book-copy is issued to borrower, some information such as issued data, accession number, borrower id, length of period for issue and return date are needed to enter into the database. At the same time, the status of the borrowed book should be labeled as issued and will be cleared after return by the borrower.
- (a) Draw an ERD for the above library management system.

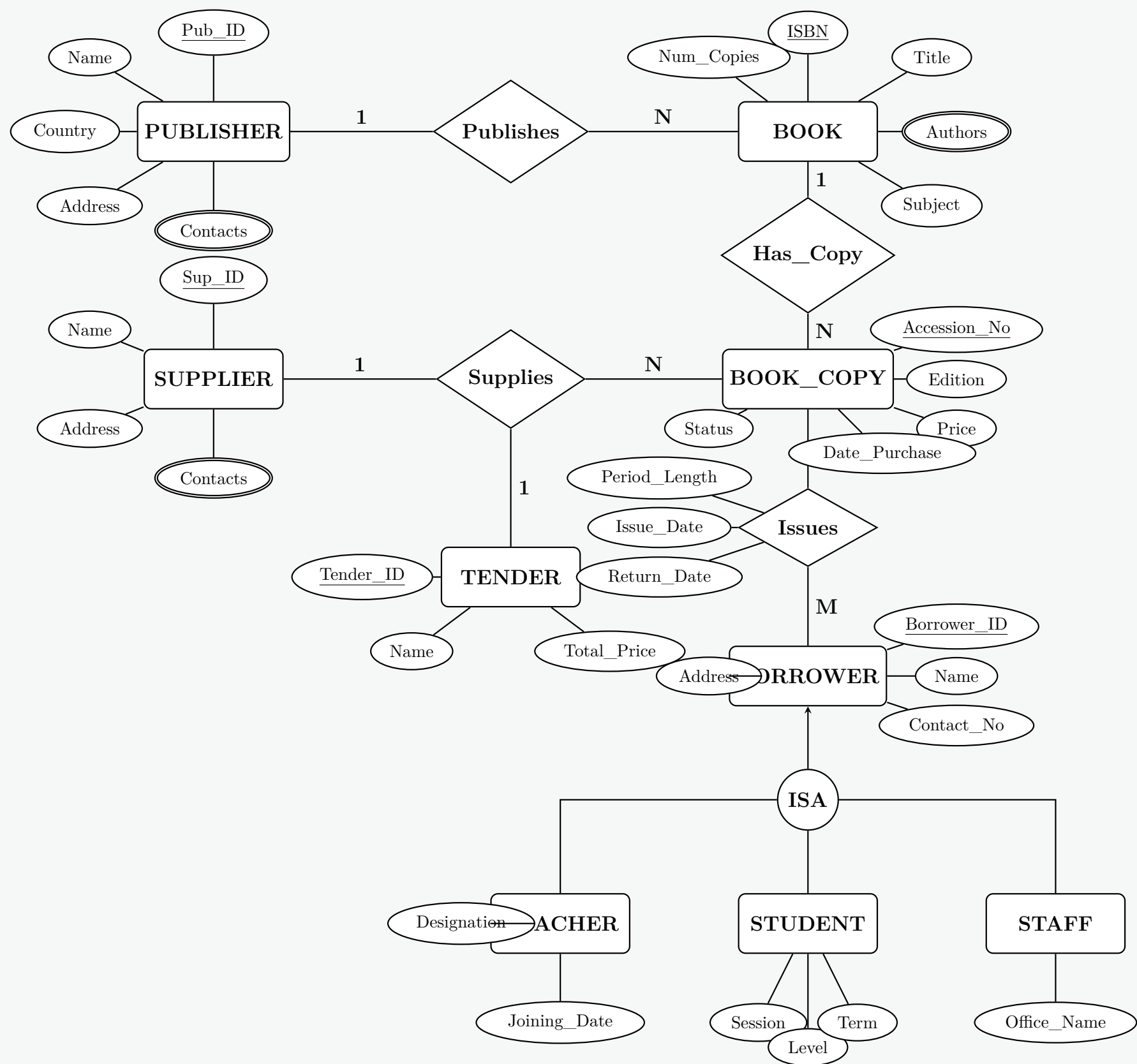
(b) Transform the ERD into tables showing all primary keys and foreign keys.
- (35 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Answer

1. Entity-Relationship Diagram (ERD)

The following Entity-Relationship Diagram models the library management system. A generalized **BORROWER** superclass is used to represent **TEACHER**, **STUDENT**, and **STAFF**. A ternary relationship **Supplies** links **SUPPLIER**, **TENDER**, and **BOOK_COPY**.



2. Relational Schema (Tables)

The ER Diagram is mapped to the following normalized relational tables. **Primary Keys (PK)** are underlined, and **Foreign Keys (FK)** are listed below each relation.

- **PUBLISHER** (Publisher_ID, Name, Country_of_Origin, Address)
- **PUBLISHER_CONTACT** (Publisher_ID, Contact_Number)
 - FK: Publisher_ID references PUBLISHER(Publisher_ID)
- **BOOK** (ISBN, Title, Subject, Number_of_Copies, Publisher_ID)
 - FK: Publisher_ID references PUBLISHER(Publisher_ID)
- **BOOK_AUTHOR** (ISBN, Author_Name)
 - FK: ISBN references BOOK(ISBN)
- **SUPPLIER** (Supplier_ID, Name, Address)
- **SUPPLIER_CONTACT** (Supplier_ID, Contact_Number)
 - FK: Supplier_ID references SUPPLIER(Supplier_ID)

- **TENDER** (Tender_ID, Name, Total_Price, Year)
- **BOOK_COPY** (Accession_Number, Edition, Price, Date_of_Purchase, Status, ISBN, Supplier_ID, Tender_ID)
 - FK: ISBN references BOOK(ISBN)
 - FK: Supplier_ID references SUPPLIER(Supplier_ID)
 - FK: Tender_ID references TENDER(Tender_ID)
- **BORROWER** (Borrower_ID, Name, Contact_Number, Address)
- **TEACHER** (Borrower_ID, Joining_Date, Designation)
 - FK: Borrower_ID references BORROWER(Borrower_ID)
- **STUDENT** (Borrower_ID, Session, Level, Term)
 - FK: Borrower_ID references BORROWER(Borrower_ID)
- **STAFF** (Borrower_ID, Office_Name)
 - FK: Borrower_ID references BORROWER(Borrower_ID)
- **ISSUE** (Borrower_ID, Accession_Number, Issue_Date, Length_of_Period, Return_Date)
 - FK: Borrower_ID references BORROWER(Borrower_ID)
 - FK: Accession_Number references BOOK_COPY(Accession_Number)

Q4. East West Property Development is a construction company of Bashundhara Group with over 1000 employees. A customer can hire the company for more than one project, and employees sometimes work on more than one project at a time. Equipment is assigned to only one project. Draw an E-R diagram for this company indicating cardinality. **(7 marks)** [CSE 303 | L-3/T-1 | 2015–16]

Answer

1. Entity and Relationship Analysis

Based on the scenario provided for East West Property Development, we can identify the following entities, relationships, and their respective cardinalities. Since specific attributes were not detailed in the prompt, standard identifier (primary key) and descriptive attributes have been assumed for completeness.

- **Entities:**

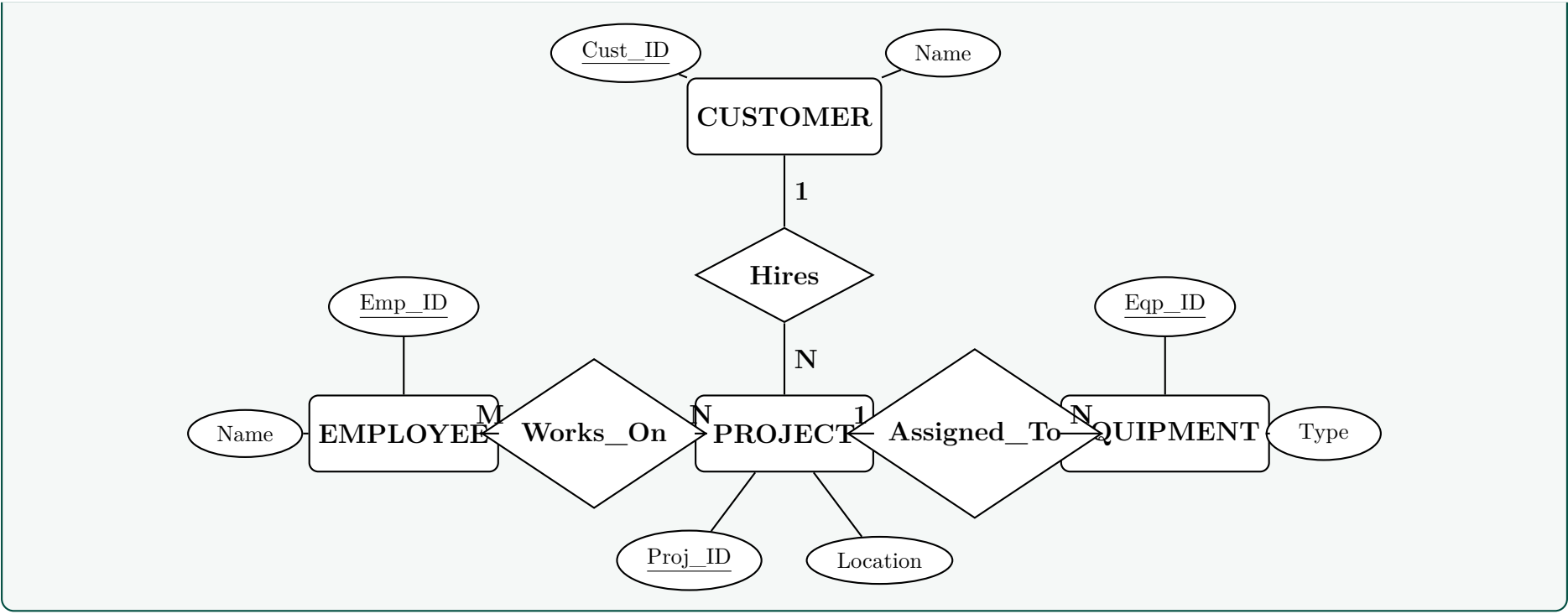
- **CUSTOMER:** Represents the clients who hire the company.
- **PROJECT:** Represents the construction projects undertaken by the company.
- **EMPLOYEE:** Represents the workforce of the company.
- **EQUIPMENT:** Represents the machinery/tools used in construction.

- **Relationships and Cardinalities:**

- **Hires (CUSTOMER and PROJECT):** A **One-to-Many (1:N)** relationship. A customer can hire the company for more than one project (N), but a specific project is typically commissioned by a single customer (1).
- **Works_On (EMPLOYEE and PROJECT):** A **Many-to-Many (M:N)** relationship. Employees can work on more than one project at a time (N), and a construction project naturally requires multiple employees (M).
- **Assigned_To (EQUIPMENT and PROJECT):** A **Many-to-One (N:1)** relationship. Equipment is assigned to only one project at a time (1), but a single project will require multiple pieces of equipment (N).

2. Entity-Relationship (ER) Diagram

The following ER diagram uses standard Chen's notation to visualize the organizational data requirements.



Q5. A university has a large number of courses. Each course may have one or more different courses as prerequisites, or may have no prerequisites. Similarly, a particular course may be a prerequisite of any number of courses or may not be a prerequisite of any other course. Sketch an E-R diagram that includes **COURSE** as the single entity, including cardinality. Each course has a course number and title. Convert the E-R diagram into a relational schema. (7 marks) [CSE 303 | L-3/T-1 | 2015–16]

Answer

1. Entity-Relationship (ER) Diagram

The scenario describes a **Unary (Recursive) Relationship** on a single entity **COURSE**. Since a course can have many prerequisites, and a course can be a prerequisite for many other courses, the relationship between **COURSE** and itself is **Many-to-Many (M:N)**.

To clarify the recursive relationship, *roles* (“Main Course” and “Prerequisite”) are indicated on the edges connecting the entity to the relationship diamond.

The diagram shows a single entity **COURSE** with attributes Course_No (PK) and Title. A recursive relationship **Requires** (diamond) connects the entity to itself. The edge from the entity to the relationship is labeled **N** and *Main Course*. The edge from the relationship back to the entity is labeled **M** and *Is Prerequisite*.

2. Relational Schema

To convert an $M : N$ recursive relationship into a relational schema, we must create two tables: one for the entity itself, and a junction (associative) table to represent the $M : N$ relationship. The primary keys (PK) are underlined.

- COURSE** (Course_No, Title)
- PREREQUISITE** (Course_No, Prereq_No)

Foreign Key Constraints and Dependencies

Relation	Foreign Key Attribute(s)	References
COURSE	None	N/A
PREREQUISITE	Course_No	COURSE (Course_No)
PREREQUISITE	Prereq_No	COURSE (Course_No)

36

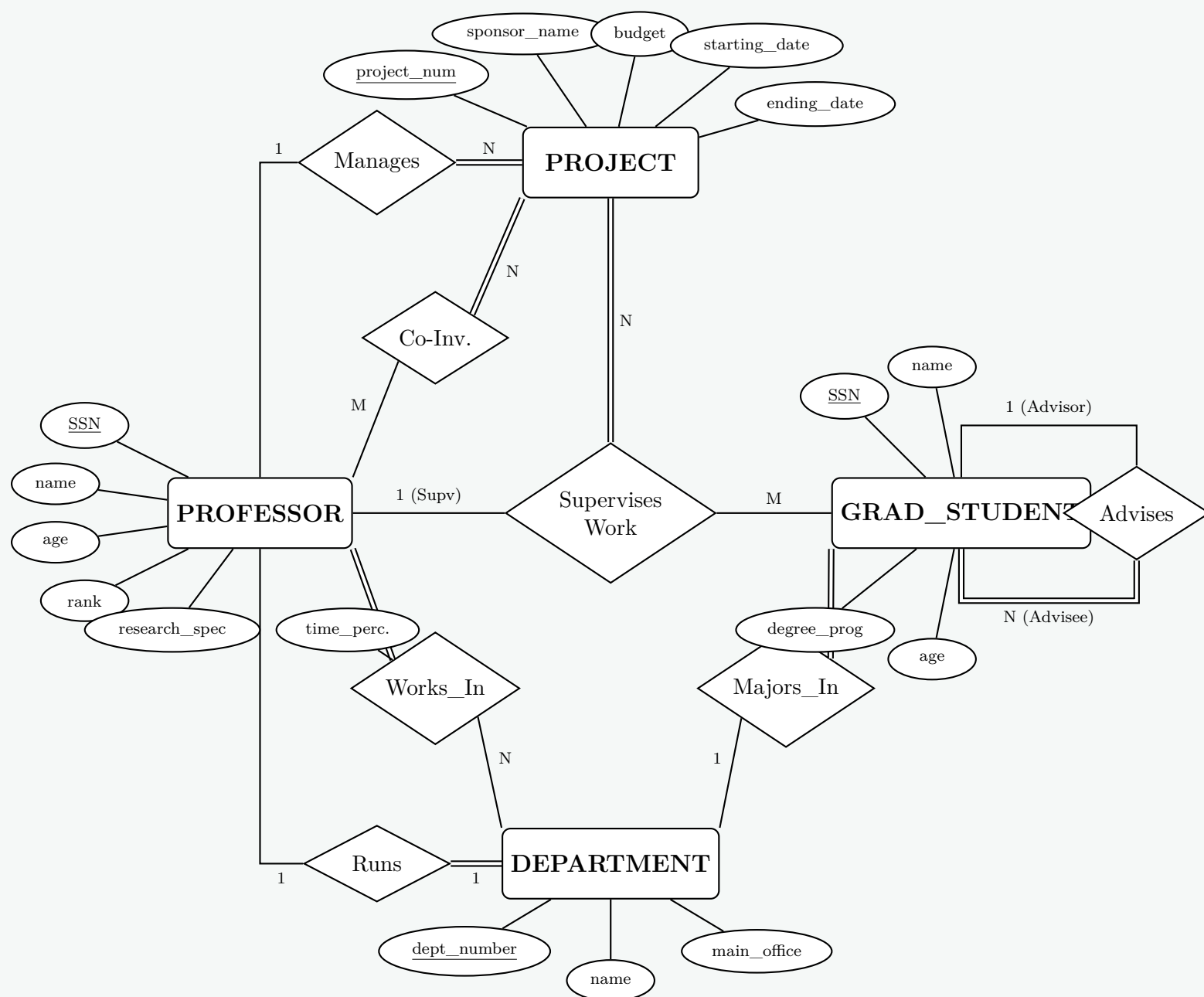
Note: In the **PREREQUISITE** table, both **Course_No** and **Prereq_No** act together as a composite Primary Key to uniquely identify a prerequisite pairing. Simultaneously, each attribute independently acts as a Foreign Key referencing the **COURSE** table.

Q6. Consider the following information about a university database. Professors have an SSN, a name, an age, a rank, and a research speciality. Projects have a project number, a sponsor name (e.g., NSF), a starting date, an ending date, and a budget. Graduate students have an SSN, a name, an age, and a degree program (e.g., M.S. or Ph.D). Each project is managed by one professor (the principal investigator). Each project is worked on by one or more professors (co-investigators). Professors can manage and/or work on multiple projects. Each project is worked on by one or more graduate students (research assistants). When graduate students work on a project, a professor must supervise their work. Graduate students can work on multiple projects, having a (potentially different) supervisor for each. Departments have a department number, name, and main office. A professor (chairman) runs each department. Professors work in one or more departments with a time percentage per department. Graduate students have one major department. Each graduate student has a more senior graduate student (student advisor) who advises on course selection. Draw an Entity-Relationship (ER) diagram to represent the data requirements described above. **(18 marks)** [CSE 303 | L-3/T-1 | 2014–15]

Answer

Entity-Relationship (ER) Diagram

The following ER diagram captures all the specified data requirements, including the ternary relationship for student project supervision, recursive relationship for student advising, and partial/total participation constraints.



Design Notes & Assumptions

- **Ternary Relationship (Supervises_Work):** Associates PROFESSOR, GRAD_STUDENT, and PROJECT. A student working on a specific project must be supervised by one professor.
- **Total Participation (Double Lines):**
 - PROJECT must have a PI (**Manages**) and co-investigators (**Co-Inv.**).
 - DEPARTMENT must be run by a chairman (**Runs**).
 - PROFESSOR must work in at least one department (**Works_In**).
 - GRAD_STUDENT must have a major department (**Majors_In**) and an advisor (**Advises**).
- **Recursive Relationship:** The **Advises** relationship maps senior graduate students (1) to their respective junior advisees (N).

- **Relationship Attributes:** The ‘time percentage’ is modeled as a descriptive attribute of the **Works_In** M:N relationship, as it characterizes the specific assignment between a professor and a department.

Q7. A library keeps records of current loans of books to borrowers. Each borrower is identified by a borrower number and each copy of a book by an accession number (the library may have more than one copy of any given book). The name and address of each borrower is held so that overdue loan reminders can be sent. Information held about books includes the title, author’s name(s), publisher’s name, publication date, ISBN, purchase price, classification (reference or fiction), and number of pages. A given book may be written by a number of authors; the library regards a book as only being published by a single publisher. The library assigns its own unique in-house codes for authors and publishers.

A book may cover a number of different subjects, and borrowers can use an online catalogue to select texts by subject as well as title and author. There is a restriction on the number of books a borrower may have on loan at any time and the loan period — these limits depend on the borrower’s classification (junior, adult, or organisation). When a book is borrowed, the return date is automatically recorded. Other borrowers may reserve books out on loan. A special borrower’s status flag is maintained — borrowers who hold overdue books or have reached their loan limit are flagged to prevent further borrowings.

Draw an Entity-Relationship (ER) diagram to represent the Library data requirements described above. **(18 marks)** [CSE 303 | L-3/T-1 | 2013–14]

Answer

1. Entity-Relationship (ER) Diagram Analysis

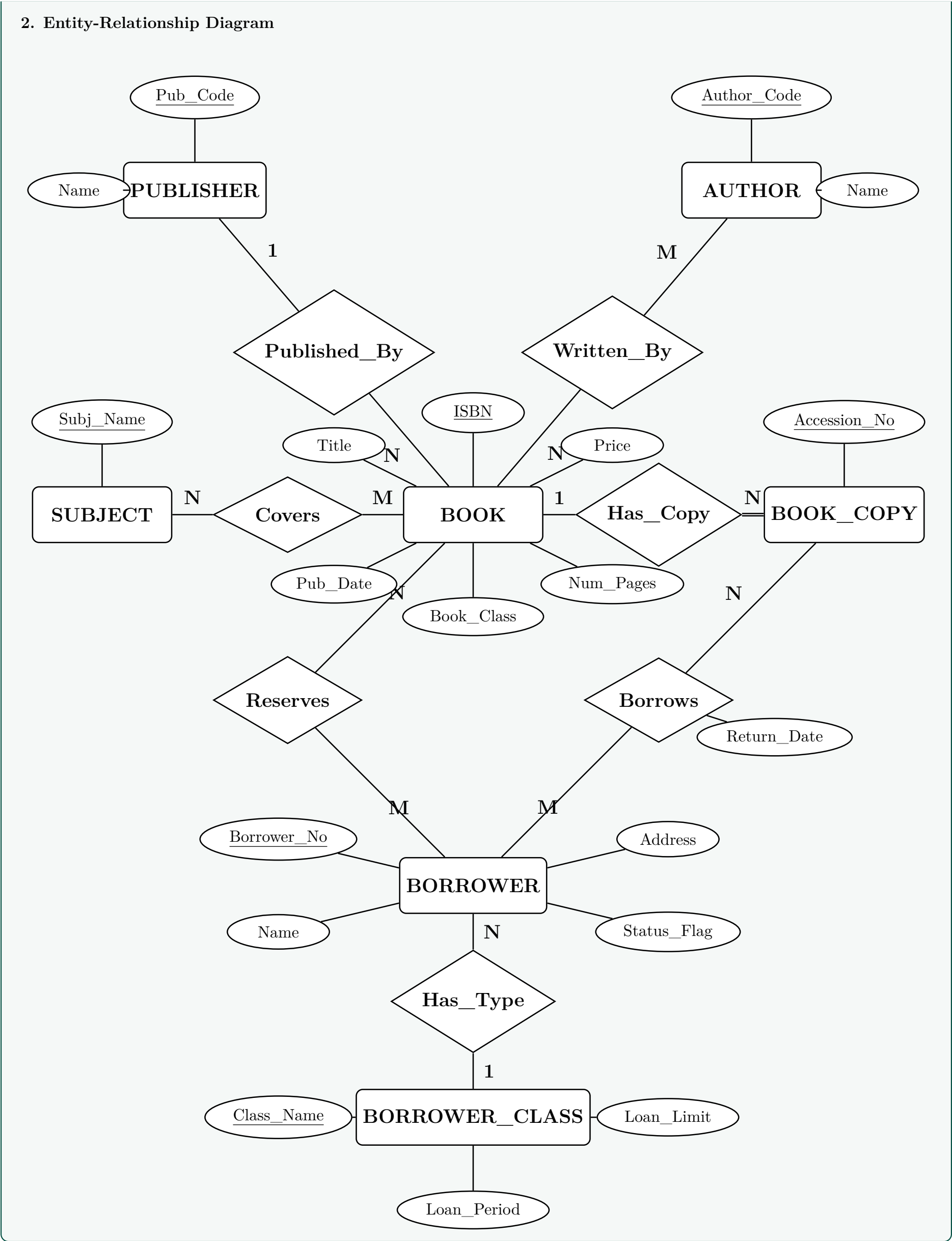
Based on the library data requirements provided, the following elements are identified to form the ER diagram:

- **Entities and Primary Keys:**

- **BOOK:** Identified by ISBN. Attributes include Title, Pub_Date, Price, Book_Class, and Num_Pages.
- **AUTHOR:** Identified by Author_Code. Attributes include Name.
- **PUBLISHER:** Identified by Pub_Code. Attributes include Name.
- **SUBJECT:** Identified by Subj_Name.
- **BOOK_COPY:** Identified by Accession_No. A strong entity as accession numbers are globally unique in a library system.
- **BORROWER:** Identified by Borrower_No. Attributes include Name, Address, and Status_Flag.
- **BORROWER_CLASS:** Identified by Class_Name (e.g., junior, adult, organisation). Attributes include Loan_Limit and Loan_Period. Extracted into a separate entity to enforce the business rule that limits depend strictly on classification.

- **Relationships and Cardinalities:**

- **Published_By:** N:1 between BOOK and PUBLISHER.
- **Written_By:** M:N between BOOK and AUTHOR.
- **Covers:** M:N between BOOK and SUBJECT.
- **Has_Copy:** 1:N between BOOK and BOOK_COPY. BOOK_COPY has total participation.
- **Borrows:** M:N between BORROWER and BOOK_COPY. Incorporates the automatically recorded attribute *Return_Date*.
- **Reserves:** M:N between BORROWER and BOOK.
- **Has_Type:** N:1 between BORROWER and BORROWER_CLASS.



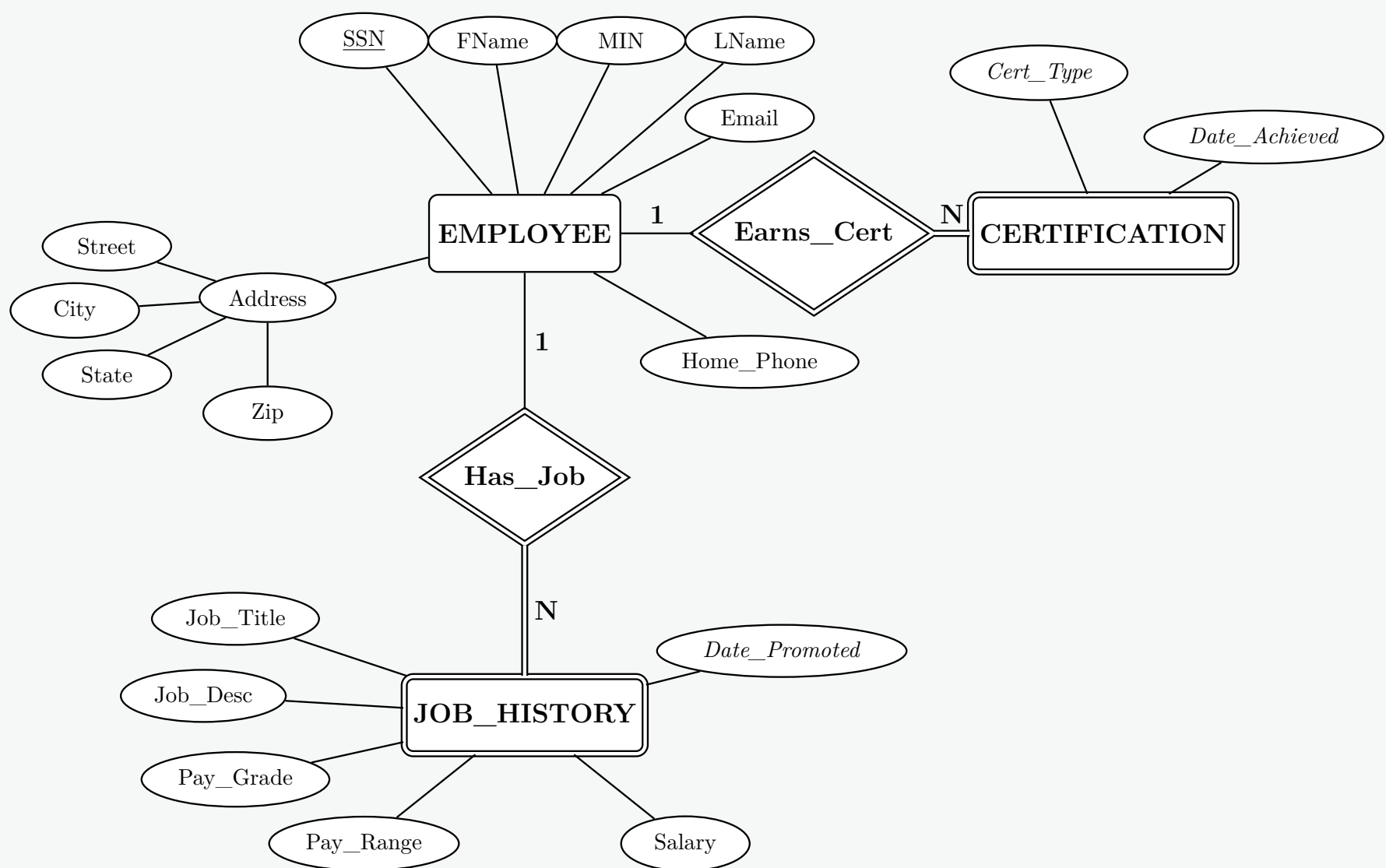
Q8. You have been asked to design an employee tracking database for a company. They want to track information about employees, the employee’s job history, and their certifications. Employee information includes first name, middle initials, last name, social security number, address, city, state, zip, home phone and email address. Job history would include job title, job description, pay grade, pay range, salary and date of promotion. For certifications, they want certification type and date achieved. An employee can have multiple jobs over time (i.e., Analyst, Sr. Analyst, QA Administrator). Employees can also earn certifications necessary for their jobs.

- (a) Draw an ER diagram for this application. Mark the multiplicity of each relationship. Identify key attributes. State all assumptions clearly. **(15 marks)**
- (b) Translate your ER diagram into a relational schema. Select approaches that yield the fewest number of relations; merge relations where appropriate. Specify the key of each relation in your schema. **(10 marks)**

[CSE 303 | L-3/T-1 | 2011–12]

Answer**Part (a): Entity-Relationship (ER) Diagram****Design Assumptions & Rules Applied:**

- **EMPLOYEE** is the core strong entity, uniquely identified by its Primary Key, **SSN**.
- **JOB_HISTORY** is modeled as a **Weak Entity** dependent on **EMPLOYEE**. An employee's job history instances cannot exist without the employee. The partial key is *Date_of_Promotion*.
- **CERTIFICATION** is also modeled as a **Weak Entity** dependent on **EMPLOYEE**. The partial key is a composite of *Cert_Type* and *Date_Achieved* to account for employees potentially renewing or re-earning the same certification over time.
- Identifying relationships (**Has_Job** and **Earns_Cert**) feature **total participation** (represented by double lines) from the weak entities.
- Partial keys of weak entities are denoted using *italics*.

**Part (b): Relational Schema**

To yield the fewest number of relations, we apply the standard mapping rule for weak entities: the weak entity relation absorbs the primary key of its identifying strong owner entity. This eliminates the need for separate tables to represent the identifying relationships. The result is exactly **three** optimally consolidated relations.

Relation	Attributes (Primary Key Underlined)	Foreign (FK)	Keys
EMPLOYEE	<u>SSN</u> , FName, MIN, LName, Street, City, State, Zip, Home_Phone, Email	None	
JOB_HISTORY	<u>SSN</u> , <u>Date_Promoted</u> , Job_Title, Job_Desc, Pay_Grade, Pay_Range, Salary	SSN references EMPLOYEE(SSN)	refer-EM-
CERTIFICATION	<u>SSN</u> , <u>Cert_Type</u> , <u>Date_Achieved</u>	SSN references EMPLOYEE(SSN)	refer-EM-

Schema Properties:

- **EMPLOYEE:** Contains only non-multivalued attributes belonging directly to the employee.
- **JOB_HISTORY:** The composite primary key (SSN, Date_Promoted) ensures that a single employee can have multiple unique job histories recorded over time without collision.
- **CERTIFICATION:** The composite primary key (SSN, Cert_Type, Date_Achieved) allows an employee to hold multiple certifications, as well as repeatedly earn the same certification on different dates.

Q9. Consider the following scenario of a hospital:

- Patients are treated in a single ward by the doctors assigned to them. Usually each patient will be assigned a single doctor, but in rare cases he/she will be assigned two doctors.
- Healthcare assistants also attend to the patients; a number of these are associated with each ward.
- Initially the system will be concerned solely with drug treatment. Each patient is required to take a variety of drugs a certain number of times per day and for varying lengths of time.
- The system must record details concerning patient treatment and staff payment. Some staffs are paid on a part time basis and doctors and care assistants work varying amounts of overtime at varying rates (subject to grade).
- The system will also need to track what treatments are required for which patients and when, and it should be capable of calculating the cost of treatment per week for each patient.

Draw ER diagrams to show the relationships among entities of the hospital management system. (25 marks) [CSE 215 | L-3/T-1 | 2010–11]

Answer

1. Entity-Relationship (ER) Conceptual Design

Based on the hospital scenario provided, the system can be modeled using the following entities, attributes, and relationships.

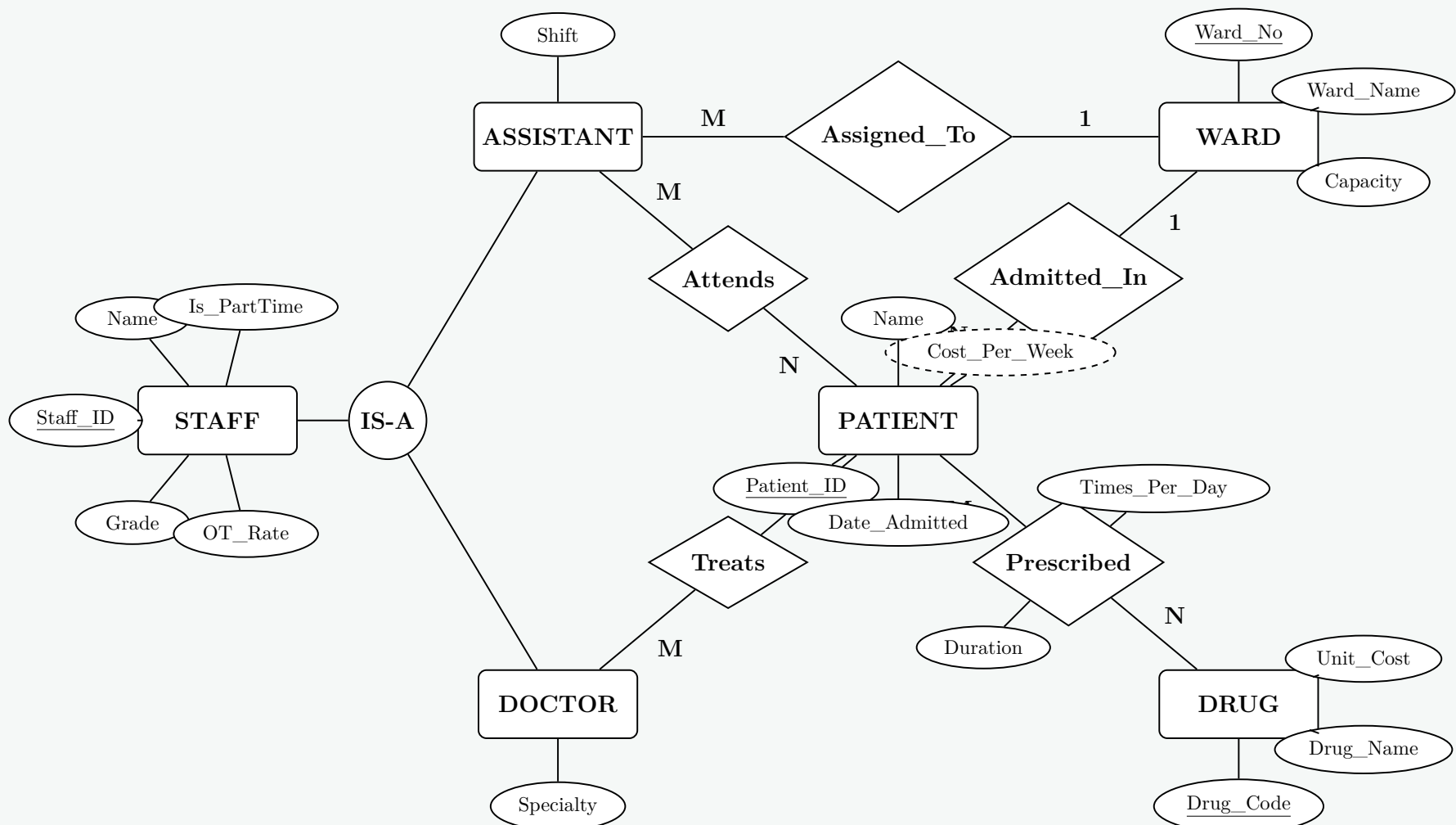
Entities and Attributes

- **STAFF (Superclass):** Generalizes common attributes for all hospital workers.
 - *Attributes:* Staff_ID (Primary Key), Name, Grade, Is_PartTime, OT_Rate.
- **DOCTOR (Subclass of STAFF):** Inherits from STAFF.
 - *Attributes:* Specialty.
- **ASSISTANT (Subclass of STAFF):** Inherits from STAFF. Represents healthcare assistants.
 - *Attributes:* Shift.
- **PATIENT:** Represents individuals admitted for treatment.
 - *Attributes:* Patient_ID (Primary Key), Name, Date_Admitted, **Cost_Per_Week** (Derived Attribute: calculated from drug unit costs and dosage frequencies).
- **WARD:** The physical location where patients are treated.
 - *Attributes:* Ward_No (Primary Key), Ward_Name, Capacity.
- **DRUG:** Information regarding medicinal treatments.
 - *Attributes:* Drug_Code (Primary Key), Drug_Name, Unit_Cost.

Relationships and Cardinalities

- **Admitted_In** (PATIENT → WARD): N:1. Multiple patients are admitted to a single ward.
- **Assigned_To** (ASSISTANT → WARD): M:1. Multiple healthcare assistants are associated with one specific ward.
- **Treats** (DOCTOR → PATIENT): M:N. A doctor treats multiple patients. A patient is usually assigned one doctor, but can have two (hence, inherently M:N).
- **Attends** (ASSISTANT → PATIENT): M:N. Assistants attend to multiple patients, and a patient is attended by multiple assistants.
- **Prescribed** (PATIENT → DRUG): M:N. A patient takes multiple drugs, and a drug is given to multiple patients.
 - *Relationship Attributes:* Times_Per_Day, Duration.

2. Entity-Relationship (ER) Diagram



Database Schema Design

Q1. Write the general procedure to convert a ternary relationship to binary relationships. (8 marks) [CSE 215 | L-2/T-1 | 2024–25]

Answer

Conversion of a Ternary Relationship to Binary Relationships

In database design, a **ternary relationship** exists when three distinct entities participate in a single relationship. However, many implementation data models (and some design tools) restrict relationships to be strictly **binary** (degree 2). To resolve this, a ternary relationship must be decomposed into equivalent binary relationships using an **associative entity** (or weak entity).

General Procedure

The systematic procedure to convert a ternary relationship R connecting three entity types A , B , and C into binary relationships involves the following steps:

- Create an Associative Entity:** Create a new entity type, let's call it E_R , to represent the ternary relationship. This new entity is inherently existence-dependent on the original entities, often making it a weak entity.
- Establish Binary Relationships:** Remove the original ternary relationship R . Create three new binary relationships (e.g., R_A , R_B , and R_C) that connect the newly created associative entity E_R to each of the three original participating entities A , B , and C .
- Assign Cardinalities:** The relationship between the original entities and the new associative entity will always be 1:N (One-to-Many).
 - The original entities (A , B , and C) will have a cardinality of 1.

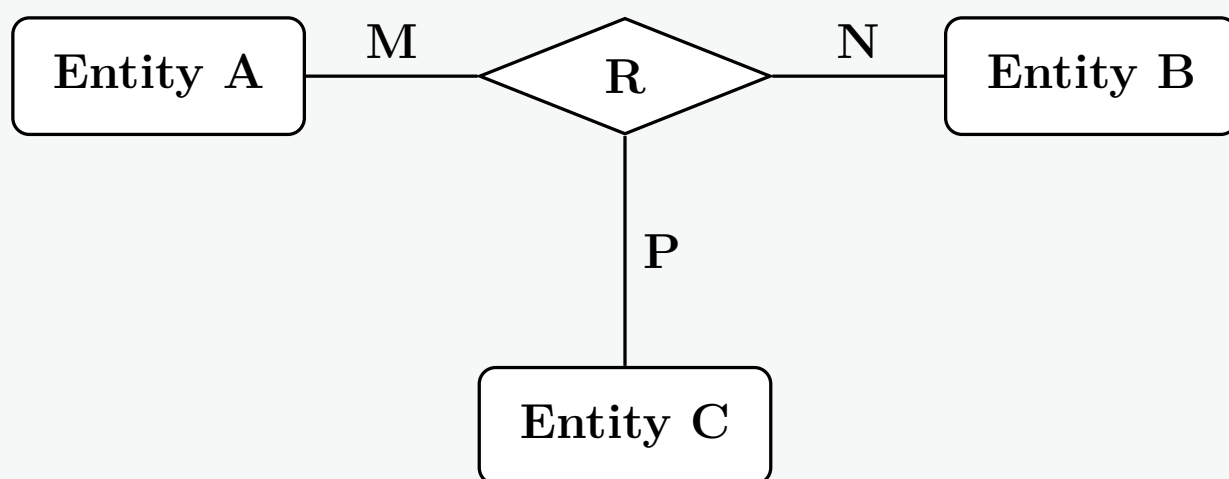
- The associative entity (E_R) will have a cardinality of **N** in all three binary relationships.
- (d) **Migrate Attributes:** Any descriptive attributes that originally belonged to the ternary relationship R must be moved to the new associative entity E_R .
- (e) **Define the Primary Key:** The primary key of the new associative entity E_R is formed by the combination (concatenation) of the primary keys of the participating entities A , B , and C . If R had its own partial key or discriminators, they are also included.

$$PK(E_R) = PK(A) \cup PK(B) \cup PK(C)$$

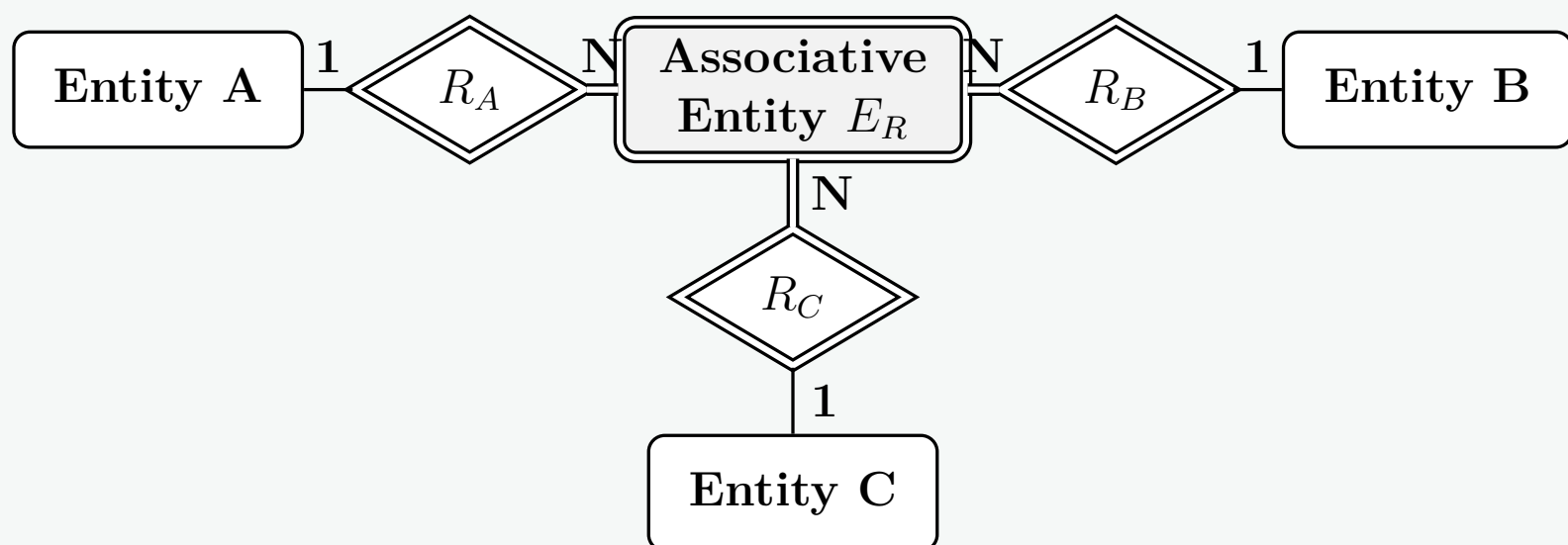
Visual Representation (ER Diagram Conversion)

The following diagram illustrates the transformation from a ternary relationship to three binary relationships linked by an associative entity.

Before: Ternary Relationship



After: Binary Equivalent



Relational Schema Mapping (Example)

If the entities A , B , and C have primary keys A_ID , B_ID , and C_ID respectively, and the ternary relationship R has an attribute *Date*, the resulting relational schema will be:

- **Entity A:** (A_ID, ...)
- **Entity B:** (B_ID, ...)
- **Entity C:** (C_ID, ...)
- **Associative Entity E_R:** (A_ID, B_ID, C_ID, Date)

Note: In the associative entity E_R, the combination of foreign keys (A_ID, B_ID, C_ID) serves as its composite primary key.

Q2. Explain the concept of total and partial participation of entities in a relationship set using an example ER diagram. **(10 marks)**
[CSE 215 | L-2/T-1 | 2024–25]

Answer

Participation Constraints in Entity-Relationship (ER) Models

In an ER model, a **participation constraint** (also known as a minimum cardinality constraint) specifies whether the existence of an entity depends on its being related to another entity via a relationship type. It defines the minimum number of relationship instances that each entity must participate in.

There are two types of participation constraints: **Total** and **Partial**.

1. Total Participation (Existence Dependency)

- **Definition:** An entity set has total participation in a relationship set if **every** entity in the entity set is associated with at least one entity in the other entity set through that relationship.
- **Explanation:** The existence of the entity dictates that it must be related to something else. If an entity does not participate in the relationship, it cannot exist in the database.
- **Notation:** Represented by a **double line** connecting the entity rectangle to the relationship diamond.

2. Partial Participation

- **Definition:** An entity set has partial participation if **only some** entities in the entity set are associated with entities in the other entity set.
- **Explanation:** Participation is optional. An entity can exist in the database without participating in the relationship.
- **Notation:** Represented by a **single line** connecting the entity rectangle to the relationship diamond.

Example Scenario

Consider the entities **EMPLOYEE** and **DEPARTMENT**, and the relationship **Manages**:

- Not every employee is a manager. Therefore, the participation of **EMPLOYEE** in the **Manages** relationship is **partial**.
- However, every department must have a manager to function. Therefore, the participation of **DEPARTMENT** in the **Manages** relationship is **total**.

Feature	Total Participation	Partial Participation
Mandatory/Optional	Mandatory (must participate)	Optional (may or may not participate)
Existence Dependency	Yes. Entity cannot exist without the relationship.	No. Entity can exist independently of the relationship.
ER Notation	Double continuous line	Single continuous line
Minimum Cardinality	Minimum constraint is ≥ 1	Minimum constraint is 0

Q3. How are composite attributes and multivalued attributes handled when an ER model is transformed into a relational database schema? Explain with examples. (10 marks) [CSE 215 | L-2/T-1 | 2024–25]

Answer

Transformation of Composite and Multivalued Attributes

When mapping an Entity-Relationship (ER) model to a relational database schema, different types of attributes require distinct transformation rules to satisfy the principles of normalization (specifically First Normal Form, which requires atomic values).

1. Handling Composite Attributes

Rule: A composite attribute is **flattened**. Only its simple, component sub-attributes are included as columns in the resulting relation. The parent composite attribute itself is discarded and does not appear in the relational schema.

- Working Principle:** The relational model does not support nested structures or attributes containing sub-attributes. By extracting the leaf-level atomic attributes, the relation remains in First Normal Form (1NF).
- Example:** Consider an entity **EMPLOYEE** with a primary key *Emp_ID* and a composite attribute *Name* comprising *Fname*, *Minit*, and *Lname*.

ER Diagram

Relational Schema

<u>Emp_ID</u>	Fname	Minit	Lname
---------------	-------	-------	-------

2. Handling Multivalued Attributes

Rule: A multivalued attribute cannot be stored in a single column because it violates 1NF. It is transformed by creating a **new, separate relation** (table).

- Working Principle:**
 - Create a new relation to hold the multivalued data.
 - Include a column for the multivalued attribute itself.
 - Add the primary key of the parent entity as a **Foreign Key (FK)** in this new relation.
 - The **Primary Key (PK)** of the new relation is formed by the **combination** of the foreign key and the multivalued attribute.
- Example:** Consider an entity **EMPLOYEE** with a primary key *Emp_ID* and a multivalued attribute *Phone*.

ER Diagram

Relational Schema

EMPLOYEE

<u>Emp_ID</u>	EMP_PHONE
---------------	-----------

EMP_PHONE

<u>Emp_ID (FK)</u>	<u>Phone</u>
--------------------	--------------

45

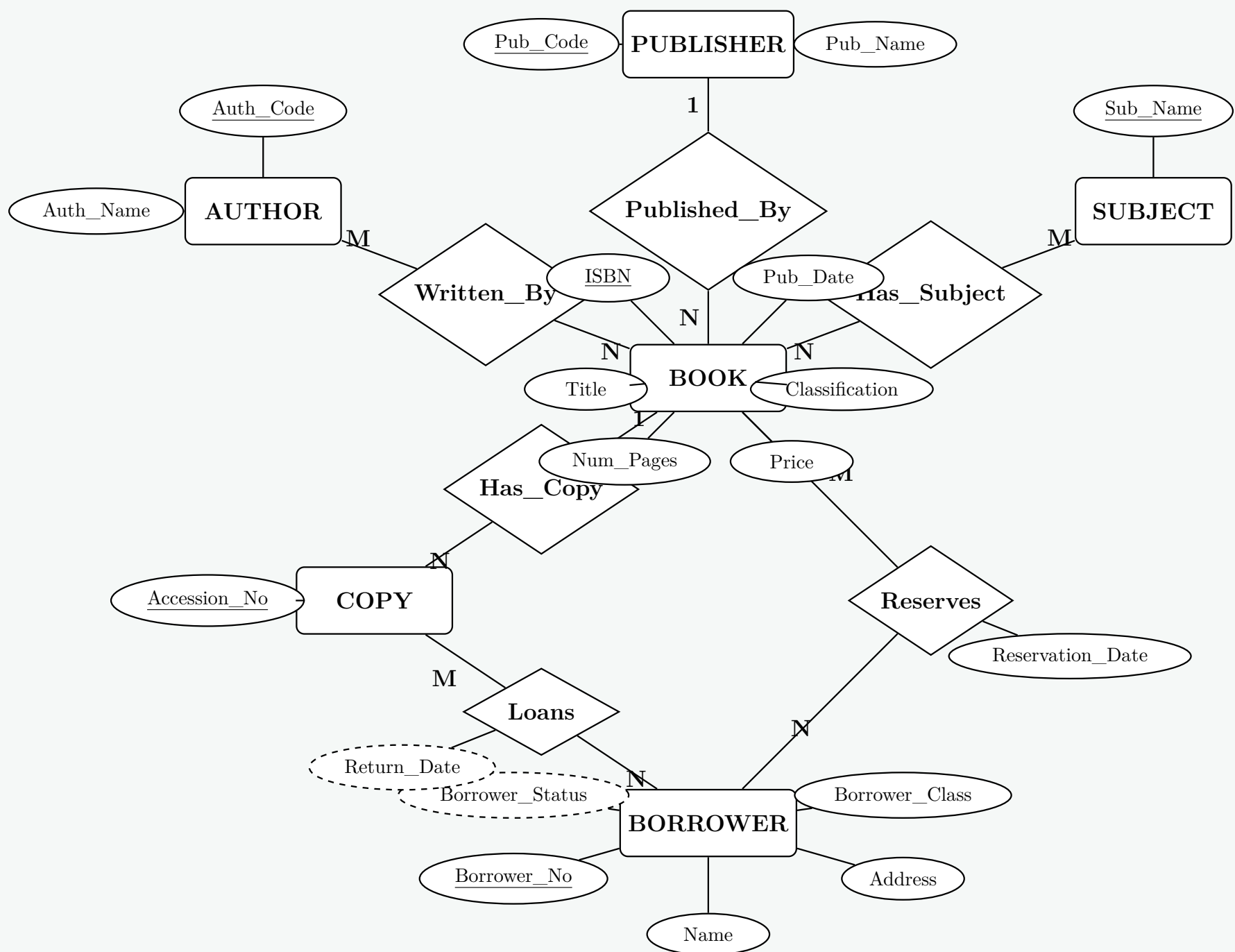
Summary Comparison		
Attribute Type	Transformation Action	Result in Schema
Composite	Flatten the components. Drop the parent attribute.	Additional columns in the <i>same</i> original table.
Multivalued	Create a completely new relation.	A <i>new</i> table linked via a Foreign Key, forming a composite Primary Key.

- Q4.** A library keeps records of current loans of books to borrowers. Each borrower is identified by a borrower number; each copy of a book by an accession number. The library may hold more than one copy of any given book. Borrower name and address are stored for communications. Book information includes: title, author name(s), publisher name, publication date, ISBN, purchase price, classification (reference or fiction), and number of pages. A book may be written by multiple authors but published by a single publisher. The library uses unique in-house codes for authors and publishers. A book may cover multiple subjects; borrowers can use an online catalogue to select texts by subject, title, or author. There are restrictions on the number of books a borrower may have on loan and the loan period, depending on borrower classification (junior, adult, or organization). When a book is borrowed, the return date is automatically recorded. Other borrowers may reserve books currently on loan; the reservation date is recorded. A borrower status flag is maintained — borrowers who hold overdue books or have reached their loan limit are flagged to prevent further borrowings.
- (a) Draw an Entity-Relationship (ER) diagram to represent the library data requirements described above.
 - (b) Identify two attributes in the above scenario which can be derived using business rules. Write the schema description to store these business rules.

(26 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Answer
1. Entity-Relationship (ER) Diagram Based on the library requirements, the conceptual data model involves central entities like BOOK, COPY, and BORROWER. Standard normalization rules are applied to extract AUTHOR, PUBLISHER, and SUBJECT into separate entities to accommodate their respective relationships (M:N for authors and subjects, 1:N for publishers).



2. Derived Attributes and Schema for Business Rules

In the given scenario, some data does not need to be hard-stored as static values but can be logically derived at runtime using established business rules.

Identified Derived Attributes

(a) **Return_Date** (on the *Loans* relationship):

When a book is borrowed, the return date is calculated by adding the designated *Loan_Period* (based on the borrower's classification) to the current checkout date.

(b) **Borrower_Status** (on the *BORROWER* entity):

This is a status flag derived by checking two conditions:

- Has the borrower exceeded their *Max_Loan_Limit*?
- Does the borrower currently hold any loans where the *Return_Date* is strictly less than the current date (overdue)?

Schema Description for Business Rules

To elegantly implement the limits based on the borrower classification (junior, adult, or organization), we create a **lookup schema** (reference table) that stores these constraints.

```
-- Relational Schema definition to store the business rules
CREATE TABLE Borrower_Class_Rules (
  Borrower_Class VARCHAR(20) PRIMARY KEY,
  Max_Loan_Limit INT NOT NULL,
  Loan_Period_Days INT NOT NULL
);
```

Relational Representation:

BORROWER_CLASS_RULES (Borrower_Class, Max_Loan_Limit, Loan_Period_Days)

Working Principle: When processing a loan, the DBMS retrieves the *Loan_Period_Days* for the user's *Borrower_Class* from this rule table to derive the *Return_Date*. Simultaneously, the system compares the user's current active loans against the *Max_Loan_Limit* to compute the *Borrower_Status* flag.

Q5. What is the difference between a view and a materialized view? Draw an example diagram of a disjoint specialization. **(10 marks)**
[CSE 215 | L-2/T-1 | 2023–24]

Answer

1. Difference Between a View and a Materialized View

In database management systems, both **Views** and **Materialized Views** are utilized to simplify complex queries, enhance security, and present a customized schema perspective to users. However, their underlying working principles and storage mechanisms differ significantly.

- View (Virtual View):** A view is a logical, virtual table defined by a SQL query. It does not store any data physically on the disk. Every time a view is queried, the DBMS dynamically executes the underlying SQL query against the base tables to fetch the current data.
- Materialized View:** A materialized view is a database object that physically stores the result set of its defining query on the disk. Because the data is pre-computed and stored, querying a materialized view is significantly faster. However, it requires periodic or triggered refreshing to synchronize its data with the underlying base tables.

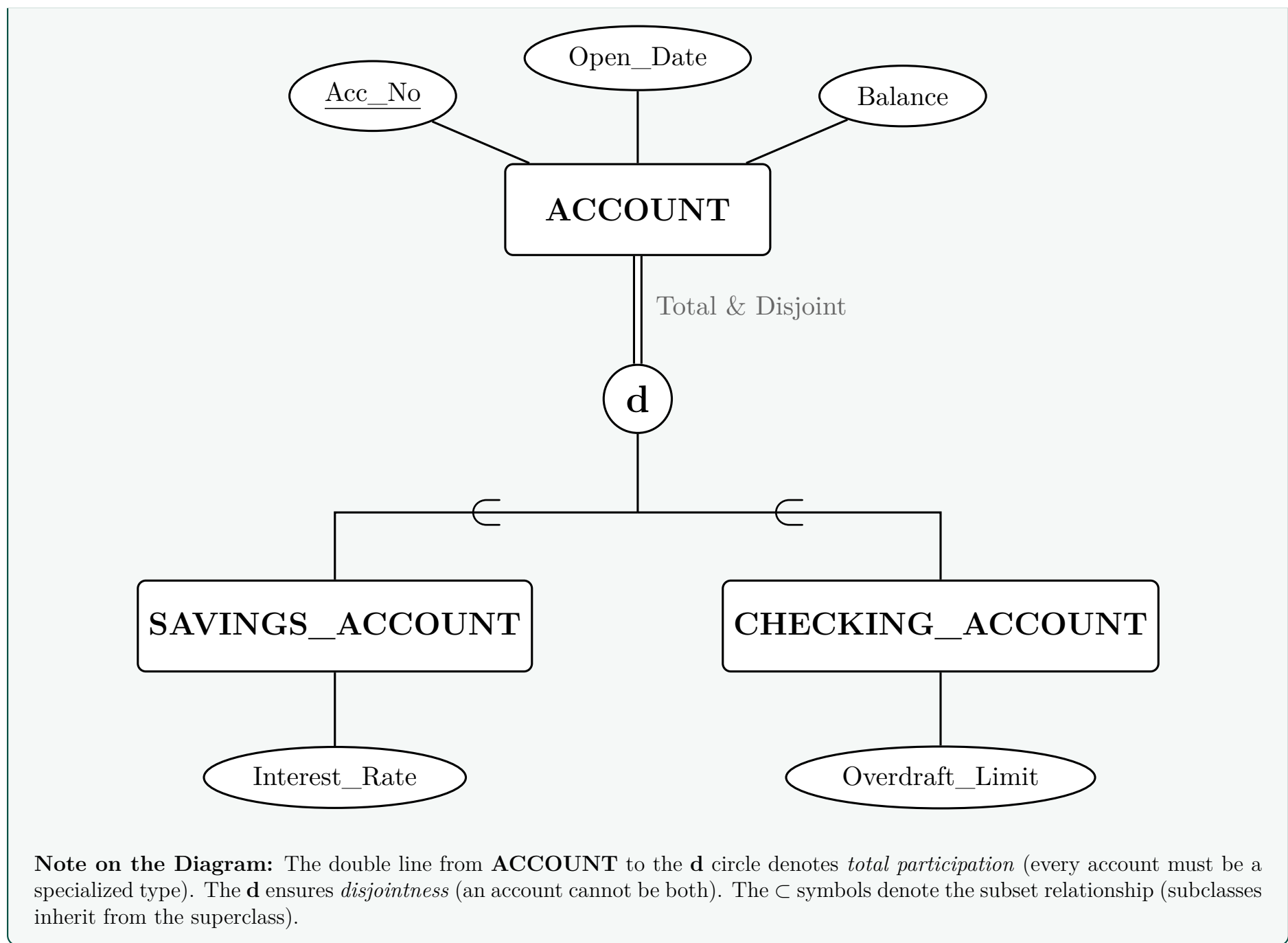
Comparison Table

Feature	Standard View	Materialized View
Storage	Virtual; stores only the query definition, not the data.	Physical; stores both the query definition and the actual data.
Performance	Slower for complex queries (computed on the fly at run-time).	Faster (data is pre-computed and retrieved directly from disk).
Data Freshness	Always up-to-date with the base tables.	May contain stale data unless it is actively refreshed.
Storage Cost	Negligible (only metadata is stored).	High (consumes disk space proportional to the result set).
Primary Use Case	Security (hiding columns/rows), simplifying complex joins for users.	Data warehousing, caching expensive aggregations, business intelligence.

2. Disjoint Specialization in EER Modeling

In the Enhanced Entity-Relationship (EER) model, **specialization** is the process of defining a set of subclasses from a single superclass. A **disjoint specialization** specifies a constraint where an entity from the superclass can belong to **at most one** subclass. It is impossible for an entity to be a member of multiple subclasses simultaneously.

Example Scenario: Consider a banking system where an **ACCOUNT** can be either a **SAVINGS_ACCOUNT** or a **CHECKING_ACCOUNT**. An account cannot be both simultaneously. This requires a disjoint constraint (denoted by a circle containing a **d**). Additionally, if every account *must* be categorized as one of these two (Total Specialization), we use a double line connecting the superclass to the disjoint circle.



Q6. YouTube is a very popular platform for hosting videos. People create channels and upload videos on that. Sometimes the creators can batch up the videos into playlists. Videos can be free to view or behind paywall which requires subscription to YouTube. Premium. A free video can be watched with or without signing into YouTube. However, only signed-in users' views are considered when counting the number of viewers for a video. A signed-in account may or may not have corresponding channels. Each video has a comment section where a signed-in user can comment; and comment to other people's comments. Also, videos contain some tags (given by the uploader or YouTube itself) that help to filter videos. Signed-in users can subscribe to other channels. Now, for each of the mentioned features below, design the corresponding portion of ERD for YouTube only. Add attributes as you see fit and mark primary keys.

- User management:** Users should be able to create accounts, create channels on top of the accounts (one user can create at most one channel), and subscribe to different channels.
- Video hosting:** Creators should be able to upload videos with different tags, and create playlists by batching up multiple videos. Users should be able to view the videos, find them in their history when needed and, also, check the number of viewers to the videos.
- Commenting:** Users should be able to comment on any video any number of times, They should also be able to comment on other peoples comments.
- Subscription system:** Creators should be able to mark some videos as premium (i.e., only subscribed users can view). Users should be able to buy subscriptions up to some end date.

(e) Implement relational schemas for each ERD you designed in this question. **(15 marks)** [CSE 215 | L-2/T-1 | 2022–23]

(20 marks) [CSE 215 | L-2/T-1 | 2022–23]

Q7. Give an example of a Non-Binary Relationship. What issues arise for Non-Binary Relationships? **(5 marks)** [CSE 215 | L-2/T-1 | 2022–23]

Q8. Suppose you are given the following requirements for a simple database for the Bangladesh Premier League (BPL) in a season:

- The BPL has many teams; each team has a name, a city, a coach, a captain, and a set of players.
- Each player belongs to only one team; a player may not be hired by any team.
- Each player has a name, a fielding position (such as slip, mid-wicket, long on, long off etc), a ranking, and a set of injury records.
- A team captain is also a player.
- Injury records must have an id, period (start and end), and a short description.

- A game is played between two teams (host_team and guest_team) with a date (such as May 11th, 2023) and scores (winning and losing team scores such as 143/10 and 148/6).

Construct a clean and concise E/R diagram for the BPL database. List your assumptions and clearly indicate cardinality mappings and any role indicators. **(10 marks)** [CSE 215 | L-2/T-2 | 2021–22]

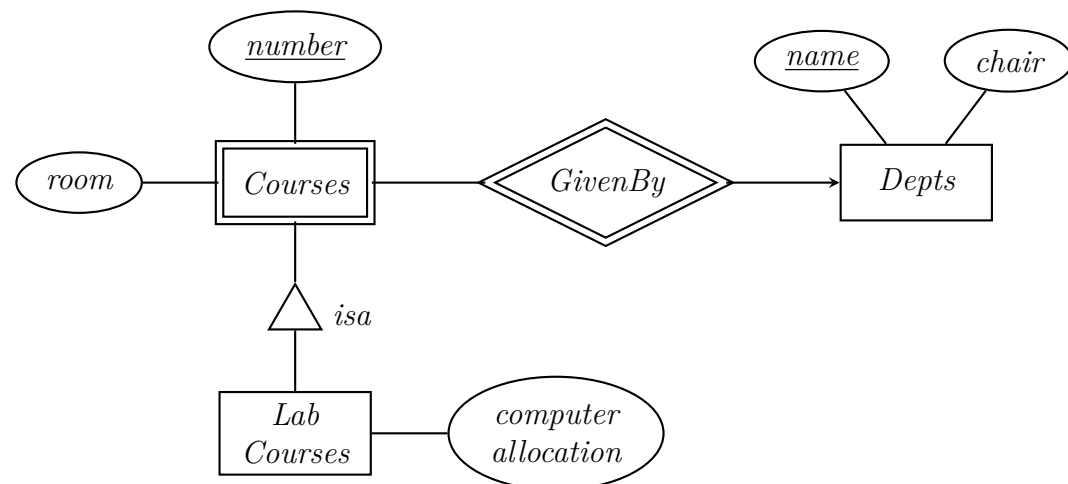
Q9. Consider the following descriptions of an organization having many EMPLOYEES. **(10+10=20)**

- An EMPLOYEE has a name, nid number, data of birth, and address.
 - Based on the EMPLOYEE's Job, EMPLOYEE may be further grouped into: SECRETARY, ENGINEER, TECHNICIAN, and MANAGER
 - Based on the EMPLOYEE's method of pay, EMPLOYEE may also be grouped into: SALARIED_EMPLOYEE, and HOURLY_EMPLOYEE
 - A salaried employee who is also an engineer belongs to the two subclasses: ENGINEER, and SALARIED_EMPLOYEE
 - A salaried employee who is also an engineering manager belongs to the three subclasses: MANAGER, ENGINEER, and SALARIED_EMPLOYEE.
 - We want to record typing speed of each SECRETARY.
 - Each technician has a technical grade (a number) which depends on his skill level
 - Each Engineer has a type (ME, CSE, EEE, etc ...)
 - Each MANAGER manages at least one project, each project has a project id, title, and a project value.
 - Each HOURLY_EMPLOYEE is a member of some TRADE_UNION which has a name and member count.
- Construct an E/R diagram with the above description; clearly indicate the cardinality mappings as well as any role indicators in your E/R diagram.
 - Convert the E/R diagram into a set of relational database schema. You should avoid keeping null values in attributes.

(20 marks)

[CSE 215 | L-2/T-2 | 2021–22]

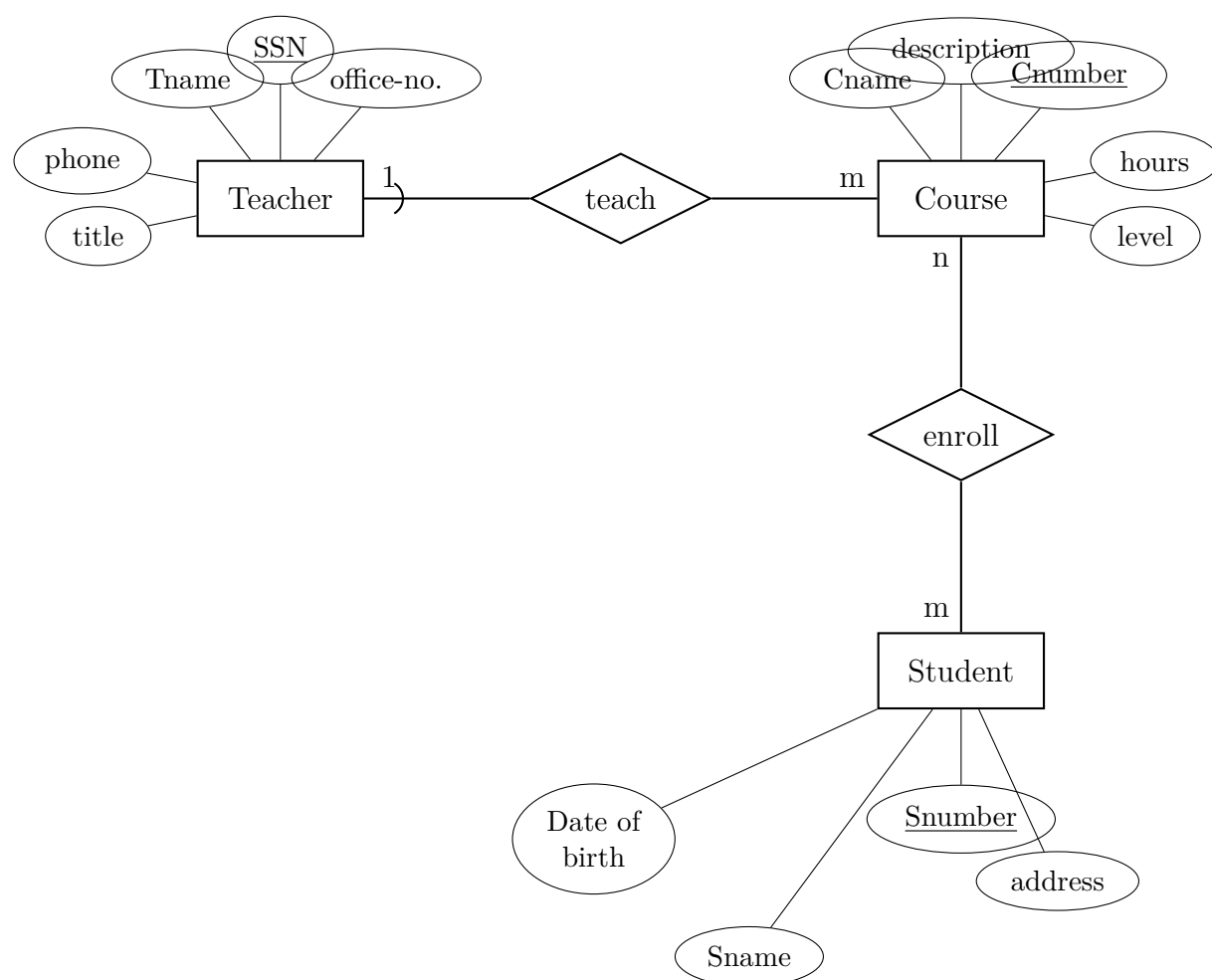
Q10. Convert the E/R diagram of Figure below into a relational database schema.



(5 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Q11. For the given E-R diagram with entities **Teacher**, **Student**, **Course**, and weak entity **Enroll**, determine which of the following statements hold:



- (a) Enroll is a strong entity set.
- (b) All teachers must teach courses.
- (c) All students must enroll in courses.
- (d) All courses must be enrolled by students.
- (e) All courses must be taught by teachers.
- (f) Each course is taught by only one teacher.
- (g) Each course is enrolled by only one student.
- (h) Each teacher may teach many courses.
- (i) Each student can enroll in many courses.
- (j) Teacher is a relationship set.

(10 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Q12. You are running a startup company allowing users to post blogs and comment on other users' blogs.

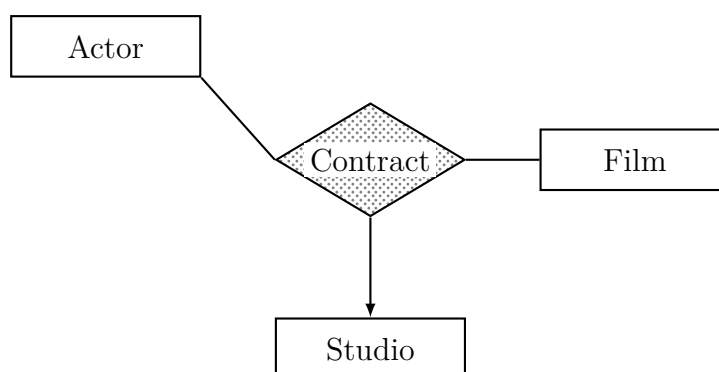
- (a) Design the E/R diagram for your company's data. Store information about users, bloggers, and blogs. Every user has a user ID and a name. Every blog has a blog ID, text content, and one author (a blogger). Every blogger is a user with a rating attribute. Users may comment on blogs. Every comment has optional text content and/or an optional rating.
- (b) Write a sequence of SQL queries that create the tables for this E/R diagram; `uid`, `bid`, and `rating` are integers, all other attributes are text. Include key, foreign key, and not null declarations.

(20 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Q13. Consider a multi-way relationship between **Actor**, **Film** and **Studio**.

- (a) There is an arrow pointing to entity set Studios. What is its interpretation?
- (b) There are no arrows pointing to entity sets Actor or Film. What are their interpretations?



(5 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Q14. Consider the following information about a university database:

- Professors have an SSN, a name, an age, a rank, and a research specialty.
- Projects have a project number, a sponsor name, a starting date, an ending date, and a budget.
- Graduate students have an SSN, a name, an age, and a degree program (e.g., M.S. or Ph.D.).
- Each project is managed by one professor (principal investigator) and worked on by one or more professors (co-investigators).
- Each project is worked on by one or more graduate students (research assistants); a professor must supervise their work. Graduate students can work on multiple projects with a (potentially different) supervisor for each.
- Departments have a department number, name, and main office; a professor (chairman) runs the department.
- Professors work in one or more departments with an associated time percentage.
- Graduate students have one major department and a more senior graduate student advisor.
- Both professors and graduate students have multiple phone numbers and email addresses.

- (a) Draw an ER diagram that captures the above information. Indicate all key and cardinality constraints and any assumptions you make.
- (b) Convert the above ER diagram into relational schema.

(35 marks)

[CSE 215 | L-2/T-2 | 2019–20]

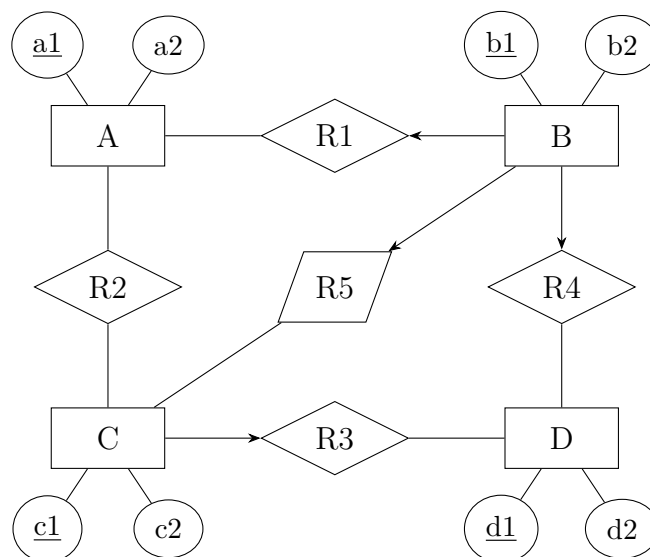
- Q15.** All employees in an organization are identified by employee id. All employees must have name, address, mobile and email. An employee must be one of the following: technical or non-technical. Technical employees have the trade and year of experience. Non-technical employees have the highest degree and year of experience. Each employee is managed by a manager who is also an employee. Draw the complete ERD for this scenario. Convert the ER diagram into two different possible relational schemas. (15 marks)

[CSE 215 | L-2/T-2 | 2019–20]

- Q16.** What do you mean by materialized view? Explain with examples the problems associated with materialized view. What restrictions do most database implementations have to allow updates on materialized views? (10 marks)

[CSE 215 | L-2/T-2 | 2019–20]

- Q17.** Reduce the given ERD to the minimum relational schemas (mention schema names and their attributes).



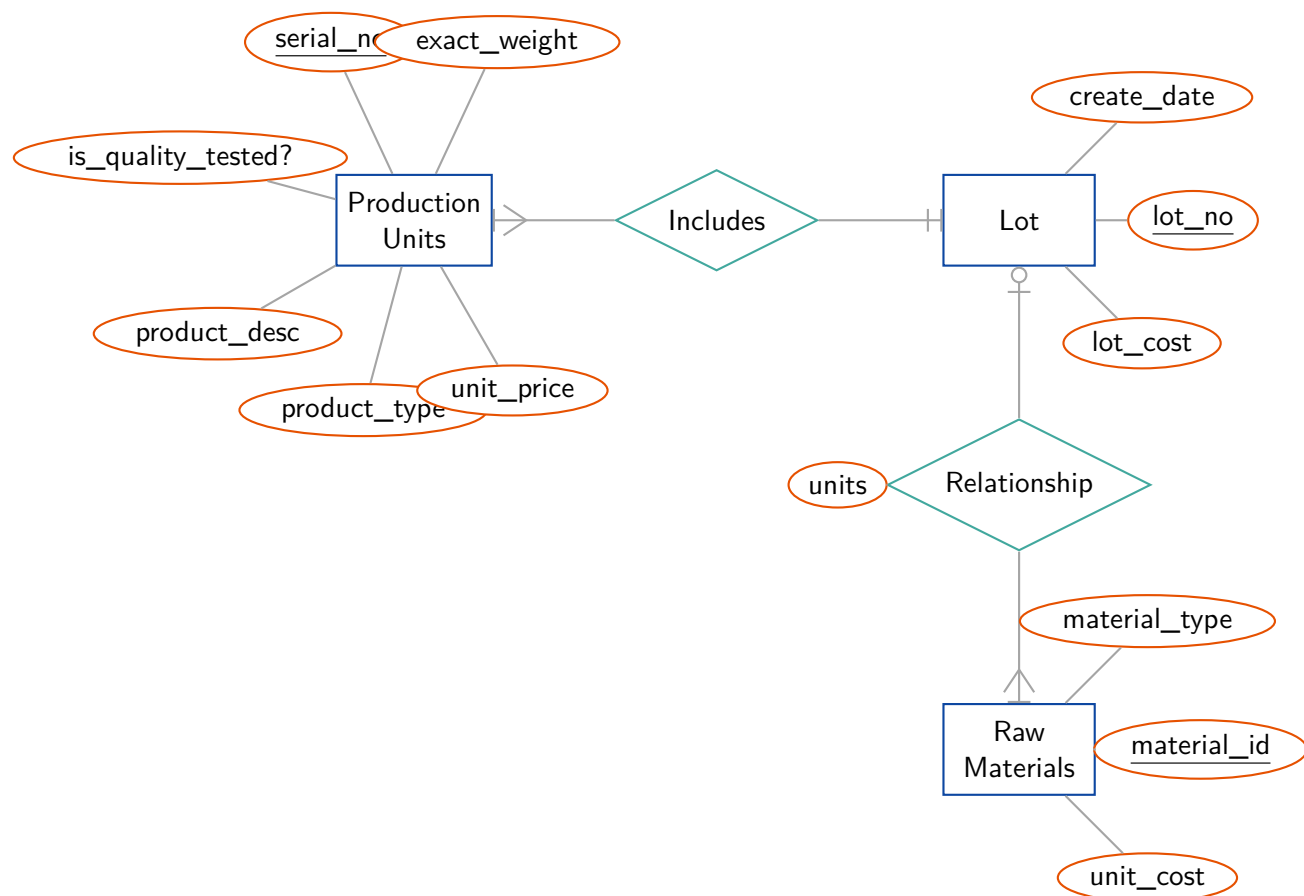
(10 marks)

[CSE 215 | L-2/T-2 | 2019–20]

- Q18.** How can redundancy and incompleteness lead to a bad database design? Explain with appropriate examples. (10 marks)

[CSE 215 | L-2/T-2 | 2018–19]

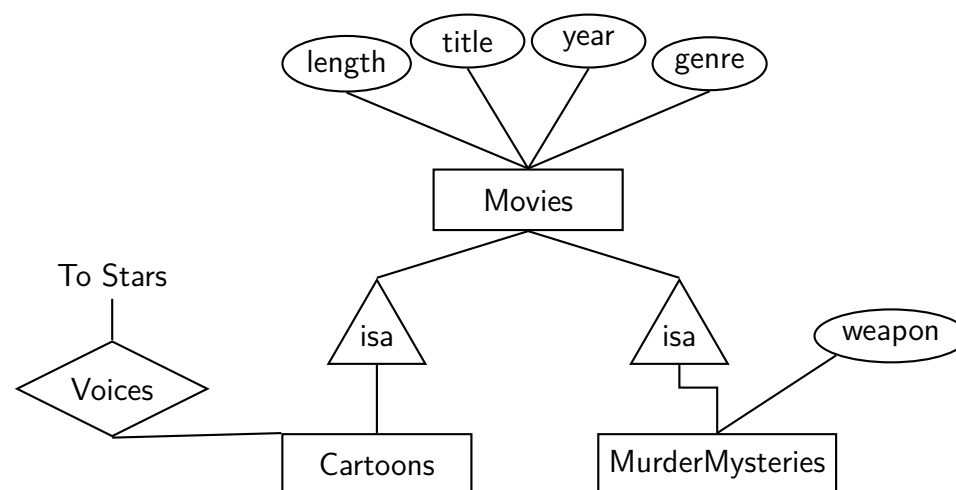
- Q19.** Production tracking is important in many manufacturing environments (e.g., the pharmaceuticals industry, children's toys, etc.). The ER diagram of Figure 3 captures important information of production tracking. Specifically, the ER diagram captures the relationships among production lots (or batches), individual production units, and raw materials.



(10 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Q20. Convert the ISA-hierarchies of the following ER diagram into schemas using three principal conversion strategies.



(10 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Q21. A car rental company maintains a database for all vehicles. For all vehicles: vehicle identification number, license number, manufacturer, size (height, width, length) and color. Special data for certain types:

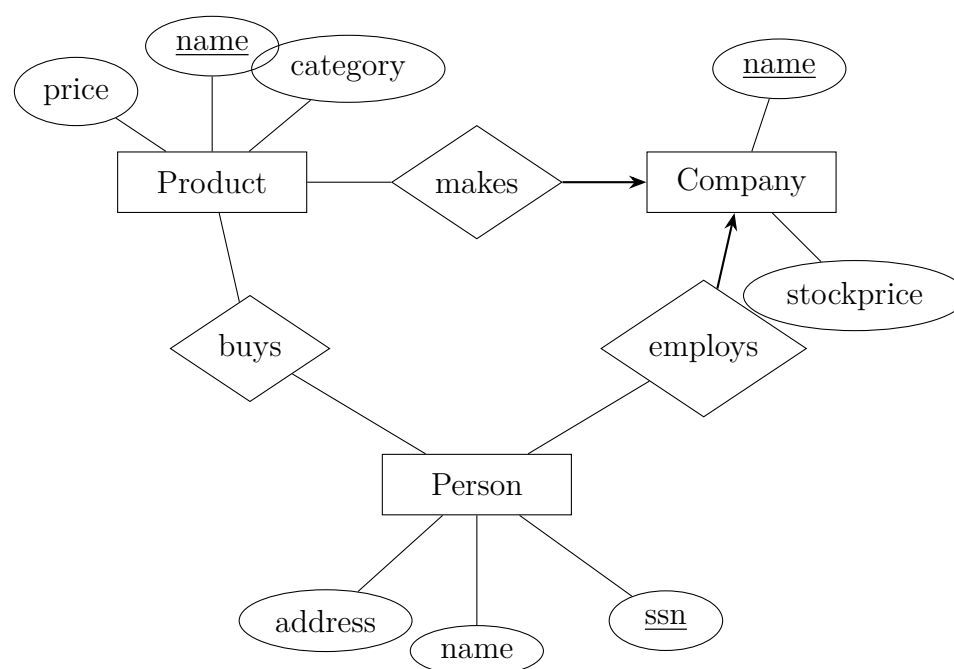
- **Trucks:** load capacity, type
- **Sports car:** power, renter age
- **Bus:** number of passengers, seat-type (array of 40 seats of different types)

- Construct an SQL:99 schema definition for this database using appropriate inheritance.
- Write constructor functions for the above schema and an insert statement to insert data.

(20 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Q22. Convert the following E-R diagram into a relational schema:



(6 marks)

[CSE 303 | L-3/T-1 | 2015–16]

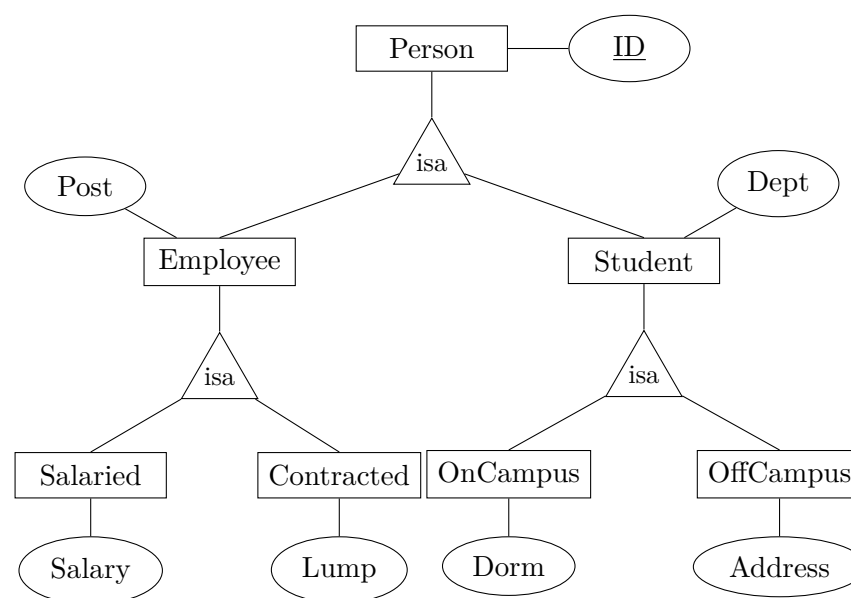
Q23. Discuss with example the difference between strong and weak relationships in the context of relational database modelling. How does security concern arise by the choice of primary key? (10 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Q24. What are the advantages and disadvantages of storing derived attributes in tables? (8 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Q25. Convert the following ER-diagram into relations. Create three separate lists for the three ISA-hierarchy conversion approaches.



(15 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Q26. A database schema consists of four relations:

```
Product(maker, model, type)
PC(model, speed, ram, hd, price)
Laptop(model, speed, ram, hd, screen, price)
Printer(model, color, type, price)
```

Suggest suitable keys and foreign keys of the above relations. (5 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Q27. Consider the E-R diagram (Figure 2) which contains composite, multivalued and derived attributes.

- Give an SQL:2003 schema definition corresponding to the E-R diagram.
- Give constructors for each of the structured types defined above.

(10 marks)

[CSE 215 | L-3/T-1 | 2010–11]

BUET · CSE 215 · Database

Part II

Query Languages

Query Languages

S3 — Query Languages

SQL

Q1. Consider the following relational database schema:

```
Planet(planet_id, name, type, distance_sun_au), — planet-id is the primary key.
Moon(moon_id, name, planet_id, diameter_km), moon_id is the primary key, and planet_id is a foreign key referencing
Planet(planet_id).
```

- Write an SQL function `total_moons()` that returns a single integer representing the total number of moons possessed by all planets. Then write the SQL statement to call the function.
- Find the names of all planets whose name contains a substring that starts with “Ma”, then has exactly two unknown characters, and ends with “s”.

(12 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Q2. Consider the following relational database schema:

```
employee(employee_name, street, city),
works(employee_name, company_name, salary),
company(company_name, city),
manages(employee_name, manager_name)
```

Write SQL statements for the following:

- Give a 20% salary rise to employees who are managers.
- Delete all tuples in the `works` relation for employees of the company “Sonali Bank”. Note that `works.employee_name` is a foreign key referencing `employee(employee_name)`, `manages.employee_name` is a foreign key referencing `employee(employee_name)`, `manages.manager_name` is a foreign key referencing `employee(employee_name)`.
- Design a trigger such that whenever a manager’s salary is updated, all employees managed by that manager receive a 10% salary increase.

(18 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Q3. Create a table in SQL from a real-life scenario where you need to apply all types of integrity constraints. (10 marks) [CSE 215 | L-2/T-1 | 2024–25]

Q4. Why are roles used for assigning privileges to users? Create three users U1, U2, and U3 and assign them INSERT and UPDATE privileges on tables T1 and T2 by assigning them a role of “Admin”. Then transfer these privileges to another role named “Super-Admin”. Finally, revoke the UPDATE privilege from “Admin”. (10 marks) [CSE 215 | L-2/T-1 | 2023–24]

Q5. Consider the following schema:

```
EMPLOYEE(EMPLOYEEID, LASTNAME, FIRSTNAME, PHONE, HIREDATE, JOBID, SALARY, DEPARTMENTID)
JOB(JOBID, JOBTITLE, MINSALARY, MAXSALARY)
DEPARTMENT(DEPARTMENTID, DEPARTMENTNAME, MANAGERID)
JOB_HISTORY(EMPLOYEEID, JOBID, DEPARTMENTID, STARTDATE, ENDDATE)
```

Write SQL to implement the following queries:

- For each employee, find the total number of employees hired before him/her and those hired after him/her. Print employee last name, total employees hired before, and total employees hired after.
- Print the last name of the employees who never changed their job or department.
- Print the last name of the employees and the difference between their salary and the maximum salary of their department, such that the difference is not more than 5000.
- For each department, print the department name, corresponding manager’s last name, and the number of employees working under that manager in that department.
- Print the last name of employees who get a higher salary than the average salary of their department. Also print the difference between their salary and the department average.

(25 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Q6. You have been tasked to prepare a database for a restaurant, The restaurant has provided the following report for you.

Food Item	Containing Items (For Packages)		Ingredients (For Simple Items)			Marketed Price	Sales		
	Simple Item	Qty	Ingredient	Unit Cost	Qty		Buyer	Date	Qty
Chicken Burger			Chicken Patty	60	1	200	C1	01/02/24	2
			Lettuce	30	0.02		C3	02/02/24	4
			Pickle	5	4		C8	02/02/24	1
			Mayonnaise	100	0.02		C1	03/02/24	3
			Sause	70	0.01				
Fried Rice			Raw Rice	50	0.1	80	C4	02/02/24	1
			Egg	7	1		C9	04/02/24	2
			Oil	500	0.005				
Set Menu 1	Fried Rice	1				500	C2	01/02/24	3
	Vegetable	1					C4	03/02/24	1
	Chicken Steak	1					C1	04/02/24	4
							C1	05/02/24	2

In the above table, “Qty” means “quantity of the item”. Now, create a relational schema, find its functional dependencies and normalize the schema. Write down all normalized forms up to BCNF, (1NF, 2NF, 3NF, BCNF).

Using the schema created for the restaurant database in:

- Write a SQL query to create a view that contains the total cost of production of each food item. Cost of production for simple items depends on the total cost of ingredients (unit cost $\times$ quantity used); for packages, it depends on the total cost of the containing simple items.
- Using the schema and view above, find the monthly profit for each food item where profit = (marketed price – cost of production) $\times$ quantity sold in the month.

(20 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Q7. Consider the Toys database of a retail store. It has the following database schema:

Toys (id, name, color, min_age, weight, number_available)

Client (id, name, age, address, city)

Request (client id, toy id)

The underlined attributes are keys for each relation. The tables contain the following information:

- Toys* records information about all of the toys in the store. Each Toy has a unique integer *id*. Attributes *name* and *color* are strings that describe the toy; *min_age*, *weight*, and *number_available* are integers giving the minimum age a child should be to use the toy, the weight of the toy in pounds, and how many are currently available in the store.
- Client* stores information about each of the clients. Each client has a unique integer *id*, strings with the client’s *name*, *address*, and *city*, and an integer *age*.
- Request* has a pair of integers indicating that a particular client has requested a particular toy.

You should assume that all entries in all of the tables are not null. Answer the followings:

- Give the name of the clients who do not have any request at the moment for any toys.
- List the name of the toys that has weight more than the average weight of the toys with the same color.
- List the name of the requesting clients who have to wait because the toys are currently unavailable at the store.
- List all of the requests where *age* of the client is less than the *min-age* listed for the Toy. The answer should include the client *id*, client name, and the toy *id*. The results should be sorted by client name.
- Give the *id* and name of the “most popular” toy. The most popular toy is the one that has more requests than all others. If there is a tie so there is more than one toy with the maximum number of requests, give the *id*’s and names of all of these toys. They do not need to be in any particular order.

(25 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Q8. Consider the following relational database schema:

Product(maker, model, type), PC(model, speed, ram, hd, price), Laptop(model, speed, ram, hd, screen, price)

Write SQL DML queries to reflect the following:

- Insert the fact that for every PC there is a laptop with the same manufacturer, speed, ram, and hard disk, a 17-inch screen, a model number 1100 greater and a price \$400 more.
- Delete all laptops made by a manufacturer that does not make PCs.

(10 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Q9. Show the SQL command to create a duplicate copy of a table named **Student**. What would the command be if we only want to copy the schema and not the data of the **Student** table? (5 marks) [CSE 215 | L-2/T-2 | 2021–22]

Q10. You have been analysing data from a social networking site and have derived the following relation which captures topics discussed by various users:

Discussion(user1, user2, topic)

The relation contains a tuple (u_1, u_2, t) every time a user u_1 discussed a topic t with user u_2 . To avoid duplicate entries, **user1** always precedes **user2** in alphabetical order.

- (a) Write a SQL query that returns all topics discussed by Pori and Himu but not discussed by Pori and Misir Ali.
- (b) Write a SQL query that returns the number of topics discussed by more than 10 pairs of users.

(24 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Q11. You are in charge of managing the program committee for an important conference. The **conference database** stores information about papers submitted to the conference (table Paper), reviewers on the program committee (table Reviewer), and the assignment of reviewers to papers (table Reviews). Each reviewer on the program committee will have to review a set of papers. Each paper will be reviewed by a subset of reviewers. (15)

Paper(pid, title)
Reviewer(rid, name)
Reviews(rid, pid)

- i. Declare the conference database using CREATE TABLE. Include the declaration of the keys and maintain the referential integrities as needed. The details are as follows:
 - pid is a unique paper identifier and the primary key of the Paper table.
 - rid is a unique reviewer identifier and the primary key of the Reviewer table.
 - rid in the Reviews table must be someone mentioned in the Reviewer table. Modifications to Reviewer table that violate this constraint result in:
 - * the rid in Reviews being set to NULL upon deletion.
 - * update of the rid in Reviews table.
 - pid in the Review table must be a paper mentioned in the Paper table. Modifications to Paper table that violate this constraint are **rejected**.
 - A reviewer is assigned zero or more papers.
 - A paper is assigned zero or more reviewers.
- ii. Write a **SQL query** that finds all papers with fewer than three reviewers assigned to them. The output of the query should be a list paper titles. The result should **include papers without any reviewers** assigned to them.
- iii. Write a SQL query that find the reviewers with the most papers assigned to them. There can be more than one such reviewer. The output of the query should be a list of reviewer names. A reviewer should be listed if no other reviewer has strictly more papers to review.

(15 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Q12. Consider the university schema: Classroom(building, room_number, capacity), department(dept_name, building, budget), course(course_id, title, dept_name, credits), instructor(ID, name, dept_name, salary), section(course_id, sec_id, semester, year, building, room_number, time_slot_id), teaches(ID, course_id, sec_id, semester, year), student(ID, name, dept_name, tot_cred), takes(ID, course_id, sec_id, semester, year, grade), advisor(s_ID, i_ID), timeslot(time_slot_id, day, start_time, end_time), prereq(course_id, prereq_id). Write SQL queries for the following:

- (a) Find all instructors whose salary is above average for their department.
- (b) Find all courses taught in both the Fall 2022 semester and in the Spring 2022 semester.
- (c) Find all students who have taken all courses taught by instructor named 'John Doe'.
- (d) List all departments along with the number of students in each department.
- (e) Find the departments that have the lowest average salary.

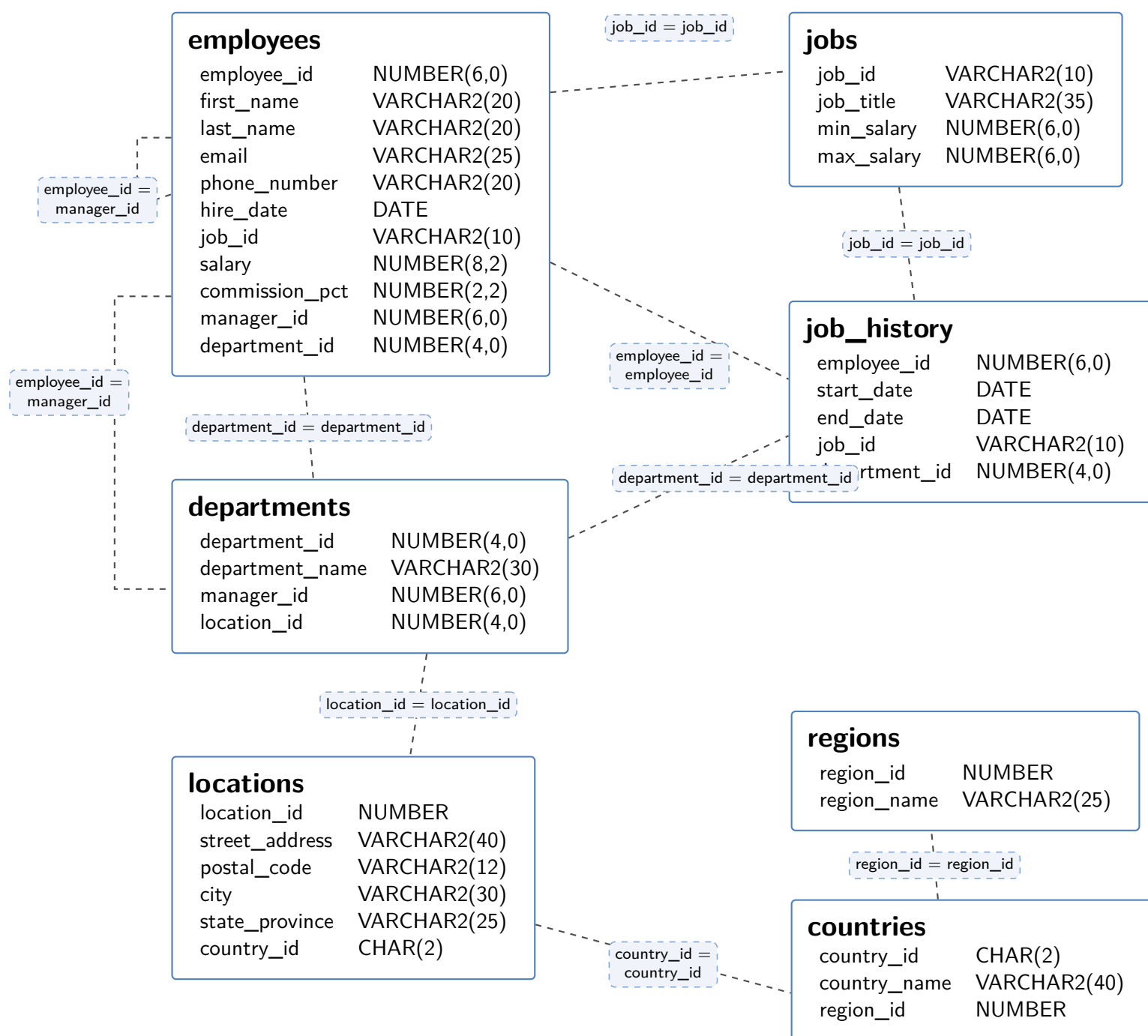
(20 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Q13. (a) The following SQL query intended to show the last name of each manager along with the number of employees he is managing is not giving the correct result. Find the fault and rewrite it correctly:

```
SELECT E1.LAST_NAME, COUNT(*) AS "TOTAL_MANAGED_EMPLOYEES"
FROM EMPLOYEE E1 JOIN E2
ON (E1.MANAGER_ID = E2.EMPLOYEE_ID)
GROUP BY E1.MANAGER_ID
ORDER BY "TOTAL_MANAGED_EMPLOYEES" ASC;
```

- (b) Write an SQL query to show the department id, job id, first hiring date, last hiring date and average salary of employees for each combination of department id and job id, where the average salary is more than 3000. Ensure no null values are printed and order the result by department id.



(10 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Q14. Consider the following schemas:

```
Books(isbn, title, publisher_id, publish_year, price_per_copy, copies_sold)
Author(author_id, name, contact)
Publisher(publisher_id, name, address)
AuthorOfBook(book_id, author_id)
```

Answer the following queries in SQL:

- Find all the books authored by “Ernest Hemingway”.
- Find the revenue earned by each publisher between 1980 and 1990 by selling books.
- Find the names of authors who have published at least 5 books since 1995.
- Find the publisher which published books of most of the authors.
- Find the names of all books which are sold in more copies than any other book published before 1990 by the same publisher.

(25 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Q15. Given the relation schema of CSE department course database:

```
course(course_number, title, credit_hour, level, term, pre_req_course_number)
takes(course_number, student_id)
student(student_id, name, CGPA)
```

Write SQL statements to:

- Find the list of only those students who have taken all the courses of level 3 and term 1.
- Find the list of courses that are not taken by any student.
- Find the student id and the number of courses taken for those students with CGPA higher than 3.5.

(15 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Q16. Given the employee database:


```

Employee(employee_name, house_number, street, city)
Works(employee_name, company_name, salary)
Company(company_name, city)
Manages(employee_name, manager_name)

```

Write SQL DDL for the above employee database with the following constraints:

- A new employee name can be entered only in the employee table and cannot be duplicate.
- Manager name must be one of the employee names.
- City can only be Dhaka, Chittagong, Rajshahi or Khulna.
- No two employees can have the same house number, street and city.
- A company name cannot be duplicate and an employee cannot work in a company whose name is not present in the company table.

(15 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Q17. Given the customer/account/transaction schema, a view is created:

```

customer(customer id, name, street, city)
account(account number, type, balance)
transaction(customer id, account number, date, amount)

```

```

CREATE VIEW cust_balance AS
SELECT a.customer_id, balance
FROM customer AS a, account AS b, transaction AS c
WHERE a.customer_id = c.customer_id
AND b.account_number = c.account_number;

```

Discuss the possible problems if insertion is allowed through the view cust_balance. (5 marks) [CSE 303 | L-3/T-1 | 2016–17]

Q18. Consider the following relational database schema:

```

Product(maker, model, type)
PC(model, speed, ram, hd, price)
Laptop(model, speed, ram, hd, screen, price)
Printer(model, color, type, price)

```

The Product relation gives the manufacturer, model number and type (PC, laptop, or printer) of various products. We assume for convenience that model numbers are unique over all manufacturers and product types; that assumption is not realistic, and a real database would include a code for the manufacturer as part of the model number. The PC relation gives for each model number that is a PC the speed (of the processor, in gigahertz), the amount of RAM (in megabytes), the size of the hard disk (in gigabytes), and the price. The Laptop relation is similar, except that the screen size (in inches) is also included. The Printer relation records for each printer model whether the printer produces color output ('T', if so; 'F' otherwise), the process type (laser or ink-jet, typically), and the price. Write the following SQL queries:

- Find those manufacturers that sell Laptops, but not PCs or Printers.
- Find those hard disk sizes that occur in two or more PCs.
- Find those pairs of PC models that have both the same speed and RAM. A pair should be listed only once.
- Find the manufacturers of PCs with at least three different speeds.
- Find the makers who produce only one type of product.
- Find the maker(s) of the cheapest color printer(s).
- Find the PC model with maximum processing speed without using any aggregate function.

(28 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Q19. Consider three tables $T1$, $T2$ and $T3$, each with a single integer column:

T1	T2	T3
1,2,2,2,3,4,4	2,3,4,4,4,5	2,2,3,3,4,4,5,5

Show the result of the following SQL queries:

- SELECT * FROM T1 UNION SELECT * FROM T2
- SELECT * FROM T1 MINUS SELECT * FROM T3
- (SELECT * FROM T1 UNION ALL SELECT * FROM T2) INTERSECT ALL (SELECT * FROM T3)

(7 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Q20. Consider the following schema:

```

Department(name, headID, yearOfEstablishment)
Employee(id, name, address, designation, salary, gender)

```

- Enforce the rule “Departmental head must be an employee of the university” — show two ways: one using a CHECK constraint and one without any check constraint.
- Write a CHECK constraint to make sure that no two departments have the same person as Head.
- Write an assertion to check: if the Head of the Department’s gender is male then his name must not begin with ‘Ms’.
- Write an assertion to check: the CSE Departmental head’s salary must not be less than 1,50,000 Taka.

(16 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Q21. Answer the following:

- Replace the NOT NULL constraint using a CHECK constraint:

```

CREATE TABLE MyTable (
    ....
    gender CHAR(1) NOT NULL,
    ....
);

```

- Replace the UNIQUE constraint using a CHECK constraint:

```

CREATE TABLE MyTable (
    .....
    headID INT UNIQUE
    .....
);

```

- What are the three different policies for UPDATE and DELETE for maintaining referential integrity? Which is more suitable for UPDATE and which for DELETE? Why?
- Show the DDL to create a copy of a table R including the schema. What is the DDL if we just want to copy the schema and not the tuples?

(19 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Q22. Consider the following schema:

```

Flights(flno: integer, from: string, to: string, distance: interger, departs: integer, arrives: time, price: real)
Aircraft(aid: integer, aname: string, cruisingrange: integer)
Certified(eid: integer, aid: integer)
Employees(eid: integer, ename: string, salary: integer)

```

Write SQL to implement the following queries:

- Find the names of aircraft such that all pilots certified to operate them have salaries more than \$80,000.
- For each pilot certified for more than three aircraft, find the eid and the maximum cruising range of the aircraft for which she or he is certified.
- Find the names of pilots whose salary is less than the price of the cheapest route from ‘Los Angeles’ to ‘Honolulu’.
- Find the aids of all aircraft that can be used on routes from ‘Los Angeles’ to ‘Chicago’.
- Identify the routes that can be piloted by every pilot who makes more than \$100,000.

(25 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Q23. Why are roles used for assigning privileges to users? Create three users U_1 , U_2 and U_3 and assign INSERT and UPDATE privileges to them on tables $T1$ and $T2$ respectively. Do this by assigning the users a role of “Admin”. Then transfer these privileges to another role named “SuperAdmin”. Finally, revoke UPDATE privilege from “Admin”. **(13 marks)**

[CSE 303 | L-3/T-1 | 2014–15]

Q24. Consider the following schema:

```

Patient(ID, FIRSTNAME, SURNAME, ADMISSION_DATE, DR_ID, ward_no)
AILMENT(AILMENTID, NAME, PATIENTID)
Ward(ward_no, ward_name)
Doctor(ID, SURNAME, FIRSTNAME, no_of_patient, AILMENTID)

```

Write SQL to implement the following queries:

- List the surnames of all patients of ‘Dr. Jones’.
- List all the doctors who specialise in the ailment suffered by the patient whose surname is ‘Thomas’.
- List the ward number and ward name of the ward(s) which have the most patients.

- (d) Provide a list of all patients who have not suffered any ailment.
 (e) Show the Doctor's name who has the third highest `no_of_patient`.

(25 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Q25. Create three users U_1 , U_2 and U_3 and assign them `SELECT` and `DELETE` privileges respectively on $T1$ and $T2$ tables. Do this by assigning the users a role of "Admin". Then transfer these privileges to another role named "SuperAdmin". (8 marks) [CSE 303 | L-3/T-1 | 2013–14]

Q26. How do you insert a tuple without causing a constraint violation in the following table?

`Person(id, name, father)` where foreign key `father` references `Person`. (9 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Q27. What are the three types of relations in SQL? (5 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Q28. Consider the following relations:

```
Emp(eno, ename, title, city)
Proj(pno, pname, budget, city)
Works(eno, pno, resp, dur)
Pay(title, salary)
```

where `Emp.title` is a foreign key to `Pay.title`, `Works.eno` is a foreign key to `Emp.eno` and `Works.pno` is a foreign key to `Proj.pno`. Write SQL statements to answer the following queries:

- For each city, how many projects are located in that city and what is the total budget over all projects in the city?
- For each project, what fraction of the budget is spent (in total) on salaries for the people working on that project? Sort your answer by the value of the budget.
- Remove the work assignment of all persons to any projects for which more than 2 persons share the same responsibility.
- List all projects located in Toronto city and include for each one the number of persons working on the project.
- Find the names of projects whose budgets are greater than all projects located in Waterloo city.

(25 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Q29. A database schema consists of four relations:

```
Product(maker, model, type)
PC(model, speed, ram, hd, price)
Laptop(model, speed, ram, hd, screen, price)
Printer(model, color, type, price)
```

- Delete all laptops made by a manufacturer that does not make printers. (5 marks) [CSE 303 | L-3/T-1 | 2011–12]
- Assume that manufacturer A bought manufacturer B. Change all products made by B so that they are now made by A. (5 marks) [CSE 303 | L-3/T-1 | 2011–12]

Q30. Consider the following schema:

```
Product(pid, name, price, category, year, maker_cid)
Purchase(buyer_ssn, seller_ssn, store, pid)
Company(cid, name, stock_price, country)
Person(ssn, name, phone_number, city)
```

Note: In `Purchase`: `buyer_ssn`, `seller_ssn` are foreign keys in `Person`, `pid` is foreign key in `Product`; in `Product` `maker_cid` is a foreign key in `Company`.

Write SQL statements to answer the following queries:

- Find names of people buying telephony products.
- Find products (and their manufacturers) that are more expensive than all products made by the same manufacturer before 1972.
- Find names of people who bought Bangladeshi products and did not buy Indian products.
- Find names of products (and their manufacturers) whose prices are less than the average product price of that company.

(20 marks)

[CSE 215 | L-3/T-1 | 2010–11]

Relational Algebra

Q1. Consider the relations `Instructor` and `Teaches` below. Compute the output of the following:

- $\Pi_{name}(\sigma_{salary < 900000}(\text{Instructor}))$
- $\sigma_{year=2024}(\sigma_{\text{Instructor.id}=\text{Teaches.id}}(\text{Instructor} \times \text{Teaches}))$

(c) $\Pi_{name}(\text{Instructor} \bowtie_{\text{Instructor.id}=\text{Teaches.id}} \text{Teaches})$

Instructor				Teaches			
id	name	dept_name	salary	id	course_id	semester	year
1	Alice	CSE	950000	1	CS101	Fall	2023
2	Bob	Physics	2300000	1	CS102	Spring	2024
3	Carol	CSE	720000	2	PH201	Fall	2023
4	David	Math	2200000	3	CS101	Fall	2023
5	Eve	CSE	2500000	5	CS201	Fall	2023

(9 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Q2. Consider the following schema:

`Suppliers(sid, sname, address), Parts(pid, pname, color), Catalog(sid, pid, cost)`

Write relational algebraic expressions for the following:

- Find the names of suppliers who supply more than 10 parts.
- Find the sids of suppliers who supply some red or green part.
- Find the sids of suppliers who supply some red part or are located at 221 Packer Street.
- Find the sids of suppliers who supply some red part with each part's cost more than 100 taka.
- Find the sids of suppliers who supply every part.

(25 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Q3. A school has prepared a database to manage admission tests. The examinees are given roll numbers and assigned to specific rooms for the exam. The test contains 10 questions. Each question is marked by two examiners to ensure fairness and correctness. An absentee list is kept to ensure no script goes missing. The schema for the database is as follows:

`Room(Room_No, Floor, Capacity)`
`Examinee(Roll_No, Name, Address, Date_of_Birth, Room_No)`
`Evaluation(Roll_No, Question_No, Examiner, Marks)`
`Absentee(Roll_No)`

Write queries using relational algebra to compute the following:

- The students should be assigned to the rooms in a manner that all the “Roll No”s in the room are sequential (e.g., if a room has 10 examinees and the lowest Roll No in that room is 501, then “Roll No”s 502, 503, ..., 510 should be assigned to that room). Find the rooms that violate this condition.
- We need to check if all scripts have been correctly evaluated, i.e., all examinees' answers to all questions have been evaluated by exactly two teachers. Find out the Roll No and Question No of the students that have not got correctly evaluated. Please note that only the examinees that are present in the exam should be considered here.
- The marks given by two examiners can only be at most 2 marks apart (i.e., for a student's answer to a specific question, the two marks by the two examiners should not differ by more than 2). Find the Roll No and Question No for which two examiners' marks differ by more than 2.

(30 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Q4. Suppose we have two relations A and B with the same set of attributes. Relation A has n_a tuples and relation B has n_b tuples. For each of the following relational algebra expressions, give the minimum and maximum number of tuples that can appear in the result. Assume that the produced results are bags, not sets.

- $A \cap B$
- $\sigma_c(A) - B$
- $A \cup B$
- $A \bowtie_L B$, where $\bowtie_L$ is the left outer join

(8 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Q5. Using an example, show that $\pi_a(R \cap S) \neq \pi_a(R) \cap \pi_a(S)$. (5 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Q6. Given the university schema: `Classroom(building, room_number, capacity)`, `department(dept_name, building, budget)`, `course(course_id, title, dept_name, credits)`, `instructor(ID, name, dept_name, salary)`, `section(course_id, sec_id, semester, year, building, room_number, time_slot_id)`, `teaches(ID, course_id, sec_id, semester, year)`, `student(ID, name, dept_name, tot_cred)`, `takes(ID, course_id, sec_id, semester, year, grade)`, `advisor(s_ID, i_ID)`, `timeslot(time_slot_id, day, start_time, end_time)`, `prereq(course_id, prereq_id)`. Write the following queries in relational algebra:

- Find the ID and name of each instructor in the Physics department.
- Find the ID and name of each instructor in a department located in the building ‘Watson’.

- (c) Find all the courses taught in the Fall 2022 semester but not in Spring 2022 semester.
- (d) Find the ID and name of those instructors who earn more than the instructors in the Physics department.
- (e) Find the ID and name of each student who has not taken any course section in the year 2022.

(15 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Q7. Consider the following schema of Sonali Bank Limited:

```
customer(customer_id, customer_name, customer_address)
account(account_id, account_type, account_branch, account_balance)
branch(branch_routing_number, branch_name)
transaction(account_id, transaction_amount, transaction_date)
loan(loan_id, customer_id, loan_amount)
```

Notice that the attribute `account_branch` of table `account` is a foreign key referencing the primary key `branch_routing_number` from table `branch`. On the other hand, the attribute `account_id` of table `account` is a foreign key referencing the primary key `customer_id` from table `customer`.

- (a) Write a relational algebra expression using Cartesian product to find customers' names and their corresponding loan amounts who have a "savings" type account at the branch named "BUET Br."
- (b) Write the equivalent relational algebra expression for the following SQL:

```
SELECT c.customer_name, a.account_id, t.transaction_amount
FROM customer c, account a, transaction t
WHERE c.customer_id = a.account_id
      AND a.account_id = t.account_id
      AND t.transaction_date = '01-JAN-2020'
      AND t.transaction_amount > 5000;
```

(20 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Q8. Consider the following schemas:

```
Manufacturer(man_id, name, address)
Laptop(model, man_id, speed, ram, hd, screen, price)
Printer(model, man_id, is_color, type, price)
```

Write relational algebra expressions for:

- (a) Find the model and speeds of each laptop manufactured by "Dell".
- (b) Find the sorted list of names of the manufacturers that produce at least three different models of laptops.

(10 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Q9. Given three relations:

```
customer(customer_id, name, street, city)
account(account_number, type, balance)
transaction(customer_id, account_number, date, amount)
```

- (a) Write a relational algebra expression using Cartesian product to find `customer_id`, date and amount of all transactions made by customer id "C005".
- (b) Rewrite the expression of (i) using natural join.
- (c) Write the equivalent relational algebra expression for the following SQL:

```
SELECT a.customer_id, b.account_number, name, street, city
FROM customer AS a, account AS b, transaction AS c
WHERE a.customer_id = c.customer_id
      AND b.account_number = c.account_number
      AND date = '01-JAN-2017'
      AND amount > 10000;
```

(20 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Q10. Consider the following schema:

```
Route(FN, DC, AC, DI, SD, SA, PM)
Flight(FN, DT, NP, AD, AA)
Plane(PM, CA, TP)
```

(FN=flight number, DC=departure city, AC=arrival city, DI=distance, SD=sched. depart, SA=sched. arrive, PM=plane model, DT=date, NP=no. of passengers, AD=actual depart, AA=actual arrive, CA=capacity, TP=type)

Translate the following queries into relational algebra:

- (a) Which flights have at least 50 passengers on board and arrived more than 30 minutes late?
- (b) What are the departure and arrival cities of all flights on October 10th that were completely full?
- (c) What are the plane types that travel between BOSTON (BOS) and FRANCISCO (SFO)?
- (d) Which flights departed late but arrived without any delay?

ROUTE	flight number (FN)	from city (FC)	to city (TC)	distance in miles (DI)	scheduled departure time (SD)	scheduled arrival time (SA)	plane (PM)
	1384	ROC	BOS	325	0650	0810	F101
	1205	BOS	ROC	325	1730	1900	F101
	110	BOS	LAX	3250	0900	1040	L1011
	398	LAX	SFO	340	1120	1210	757
	124	SFO	BOS	3180	1400	2310	L1011
	448	BOS	DCA	340	0700	0830	737
	540	DCA	BOS	340	1900	2030	737

FLIGHT	flight number (FN)	date (DT)	number of passengers (NP)	actual departure time (AD)	actual arrival time (AA)
	1384	2000-10-09	40	0650	0800
	1384	2000-10-10	12	0730	0842
	1384	2000-10-11	30	0700	0815
	1205	2000-10-09	42	1735	1900
	1205	2000-10-10	8	1730	1855
	1205	2000-10-11	20	1810	1932
	110	2000-10-10	403	0940	1115
	110	2000-10-11	385	0905	1035
	398	2000-10-09	148	1150	1220
	398	2000-10-10	95	1125	1210
	124	2000-10-09	150	1500	2350
	124	2000-10-10	414	1410	2310

PLANE	model (PM)	capacity (CA)	type (TP)
	737	120	jet
	747	460	jet
	757	200	jet
	L1011	440	jet
	F101	45	prop

(15 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Q11. Consider the following schema:

```
Suppliers(sid: integer, sname: string, address: string)
Parts(pid: integer, pname: string, color: strign)
Catalog(sid: integer, pid: integer, cost: real)
```

Write relational algebra expressions for the following:

- (a) Find the sids of suppliers who supply some red or green part.
- (b) Find the sids of suppliers who supply some red parts and are at the address ‘221 Packer Street’.
- (c) Find the sids of suppliers who supply some red parts and some green parts.
- (d) Find the sids of suppliers who supply every part.

(20 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Q12. Consider the following schema:

```
EMPLOYEE(EMPLOYEEID, LASTNAME, FIRSTNAME, PHONE, HIREDATE, JOBID, SALARY, DEPARTMENT)
JOB(JOBID, JOBTITLE, MINSALARY, MAXSALARY)
DEPARTMENT(DEPARTMENTID, DEPARTMENTNAME, MANAGERID)
```

Write relational algebra expressions for the following:

- (a) List the average salary of each department.

- (b) List the FIRSTNAME and JOBTITLE of employee 'John'.
- (c) Increase all salaries more than 10000 by 5% and less than 10000 by 4%.
- (d) Delete all employees from the EMPLOYEE table who earn less than 1000 as salary.

(20 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Q13. Consider the following relational schema:

```
employee(emp_no, name, office, age)
books(isbn, title, author, publisher)
loan(empno, isbn, date)
```

Answer the following queries in relational algebra:

- (a) Find the names of employees who have borrowed a book published by McGraw-Hill.
- (b) Find the names and ages of employees who have borrowed all books written by Carl Hamacher.
- (c) Find the names of employees who have borrowed more than five different books published by Pearson Education.
- (d) For each publisher, find the names of employees who have borrowed more than five books of that publisher.
- (e) Find the titles and authors of the books that have been borrowed by Peter Hart.

(25 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Q14. Using the schema below, express the following queries using relational algebra:

```
Emp(eno, ename, title, city), Proj(pno, pname, budget, city)
Works(eno, pno, resp, dur), Pay(title, salary)
```

- (a) Find names of employees whose salary is more than \$1000.
- (b) List all projects where at least one person with 'Programmer' title is working on it. (A project name may appear more than once.)

(10 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Q15. Using the schema below, express the following queries using relational algebra:

```
Product(pid, name, price, category, year, maker_cid)
Purchase(buyer_ssn, seller_ssn, store, pid)
Company(cid, name, stock_price, country)
Person(ssn, name, phone_number, city)
```

- (a) Find names of people who bought Indian products and live in Gulshan.
- (b) Find people who bought stuff from Alam or bought products from a company whose stock price is more than BDT 2500.

(8 marks)

[CSE 215 | L-3/T-1 | 2010–11]

Joins

- Q1.** Explain the application of left, right and full outer joins with examples. (15 marks) [CSE 303 | L-3/T-1 | 2016–17]
- Q2.** Distinguish between 'cross join', 'natural join' and 'outer joins'. When (left or right) outer joins are useful? Give an example. (10 marks) [CSE 303 | L-3/T-1 | 2011–12]
- Q3.** Define natural joins and outer joins. (7 marks) [CSE 215 | L-3/T-1 | 2010–11]

Constraints, Assertions & Triggers

Q1. Consider the following schema:

```
Student(Ssn, Name), TakenCourses(Ssn, CourseNo)
Register(Ssn, CourseNo), Course(CourseNo, Name, Year, Term)
PreRequisite(CourseNo, ReqCourseNo)
```

Table **TakenCourses** manages courses a student has previously taken. Write a trigger to check if a student has taken the prerequisite courses when he/she registers for a new course. If a prerequisite course has not been taken, the registration should fail and an appropriate message should be displayed. (10 marks) [CSE 215 | L-2/T-1 | 2023–24]

- Q2.** What is the difference between a UNIQUE key and a PRIMARY KEY? (3 marks) [CSE 215 | L-2/T-2 | 2021–22]

Q3. Consider the following relational database schema:

Product(maker, model, type), PC(model, speed, ram, hd, price),
Laptop(model, speed, ram, hd, screen, price), Printer(model, color, type, price)

The *Product* relation gives the manufacturer, model number and type (PC, laptop, or printer) of various products. model numbers are unique over all manufacturers and product types. The *PC* relation gives for each model number that is a PC the speed (of the processor, in gigahertz), the amount of RAM (in megabytes), the size of the hard disk (in gigabytes), and the price. The *Laptop* relation is similar, except that the screen size (in inches) is also included. The *Printer* relation records for each printer model whether the printer produces color output ('T', if so; 'F' otherwise), the process type (laser or ink-jet, typically), and the price. Write necessary constraints to implement the following (answer each independently):

- (a) speed, ram, hd, screen and price cannot be negative.
- (b) A model of a product must also be the model of a PC, a laptop, or a printer.
- (c) A laptop with a screen size less than 15 inches must have at least a 40 GB hard disk or sell for less than \$1000.
- (d) No manufacturer of PCs may also make laptops.
- (e) If a laptop has a larger main memory than a PC, then the laptop must also have a higher price than the PC.
- (f) A manufacturer of a PC also makes a laptop with at least as great a processor speed.

(24 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Q4. Consider the following table definition:

```
CREATE TABLE MyTable(
    ...
    head INT UNIQUE,
    ...
);
```

Replace the above UNIQUE constraint using a CHECK constraint. **(15 marks)**

[CSE 215 | L-2/T-2 | 2020–21]

Q5. Consider the following two schemas:

Product(maker, model, type), Laptop(model, speed, ram, hd, screen, price)

Write necessary triggers to implement the following constraints:

- (a) speed, ram, hd, screen and price cannot be negative.
- (b) When a Laptop model is deleted, that model must also be deleted from the Product table.

(12 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Q6. What is the difference between referential integrity constraint and foreign key constraint? What are the three different policies for UPDATE and DELETE that can be mentioned for maintaining referential integrity? Among these, which one is more suitable for UPDATE, and which one is more suitable for DELETE? Why? **(10 marks)**

[CSE 215 | L-2/T-2 | 2019–20, 2015–16]

Q7. Consider the schema MovieExec(name, address, cert#, netWorth). We want to prevent the average net worth of movie executives from dropping below \$50,000. Write a trigger that checks this constraint before each update to the netWorth column and, if violated, sets the value of netWorth to the minimum amount required to satisfy the constraint. **(10 marks)**

[CSE 215 | L-2/T-2 | 2017–18]

Q8. What is a Materialized View? Discuss four instances when a trigger can be applied. **(10 marks)**

[CSE 303 | L-3/T-1 | 2013–14]

↔ *Similar: 2014–15: “What is a Materialized View? Discuss the restrictions on triggers in brief.”*

Q9. What are the different policies of maintaining referential integrity? Discuss them with examples. **(10 marks)**

[CSE 303 | L-3/T-1 | 2012–13]

Q10. What are the differences between UNIQUE and PRIMARY KEY constraints in SQL? **(5 marks)**

[CSE 303 | L-3/T-1 | 2012–13]

Q11. Give an example of a CHECK constraint and discuss why it is used. **(5 marks)**

[CSE 303 | L-3/T-1 | 2012–13]

Q12. A database schema consists of four relations:

Product(maker, model, type)
PC(model, speed, ram, hd, price)
Laptop(model, speed, ram, hd, screen, price)
Printer(model, color, type, price)

- (a) Write the following check constraints (using the PC/Laptop/Printer schema):
 - (i) The model number of a product must be a 7-digit number.
 - (ii) A PC with a processor speed less than 2.0 GHz must have at least 512 MB RAM or sell for less than \$800.

(5 marks)

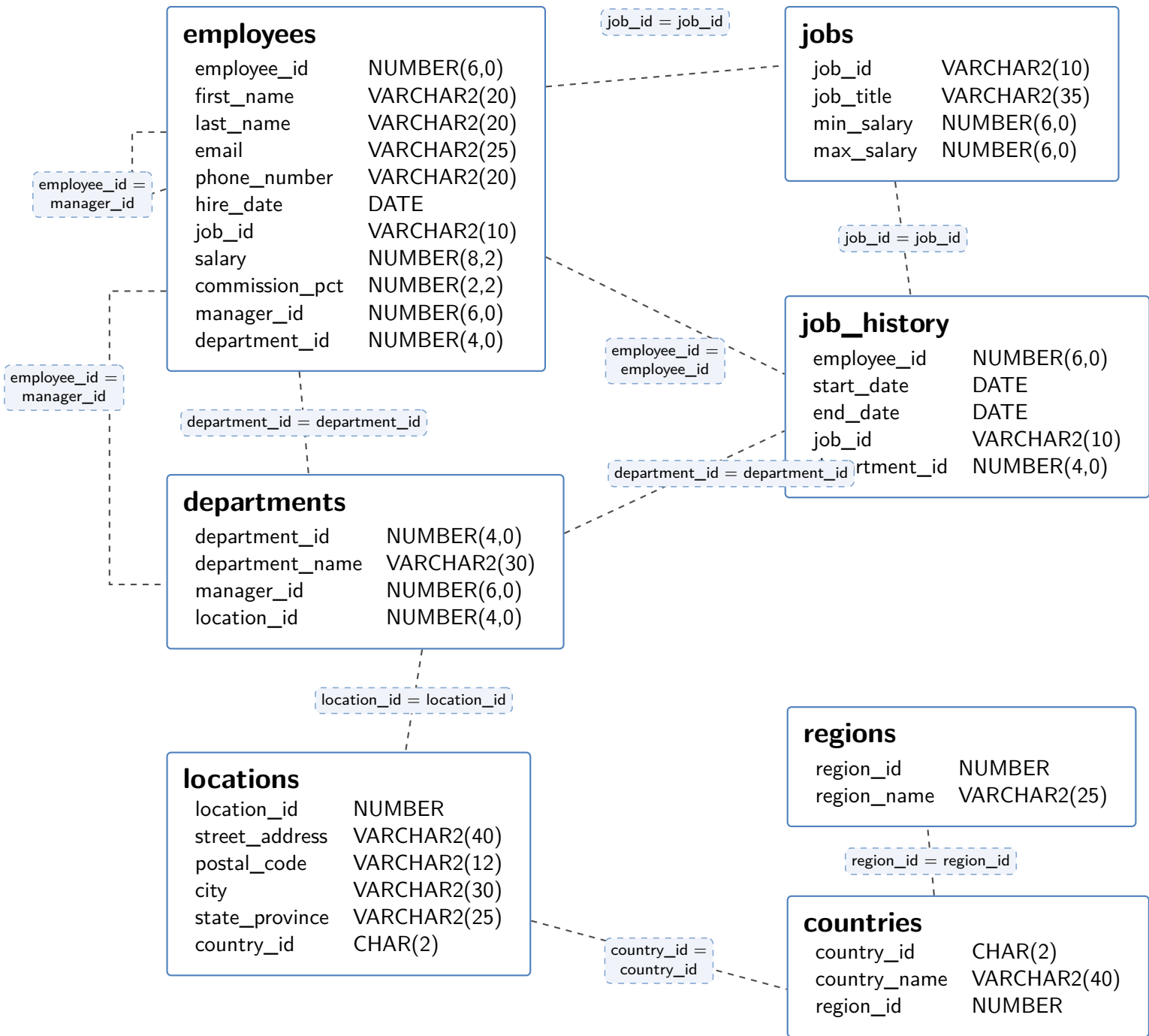
[CSE 303 | L-3/T-1 | 2011–12]

- (b) Write the following assertions:
- (i) The price of a laptop must be larger than the average price of PCs made by the same manufacturer.
 - (ii) A manufacturer of a PC must also make Laptop(s).
 - (iii) If a PC has a greater processor speed than a laptop then the PC must also have a higher price than the laptop.
- (15 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Functions and Procedures

Q1. Write a procedure named `IncreaseOrDecrease` which takes an employee id as input. If the salary of the employee is greater than the average salaries of all departments except their own, but less than the average salary of their own department, the procedure increases the salary by 10%. Otherwise it decreases by 5%. During execution it prints “Increasing Salary” or “Decreasing Salary” accordingly.



(10 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Q2. Consider the following schema:

```
Movie(title, year, filmType, inColor, studioId)
Studio(studioId, name, address)
```

Write a stored procedure that takes as input the name of a studio and a year, then counts and prints the number of black and white movies produced by that studio in the specified year. (10 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Q3. Write constructor functions for the car rental SQL:99 schema (Trucks, Sports car, Bus) and insert statement to insert data into the tables. (10 marks)

[CSE 303 | L-3/T-1 | 2016–17]

BUET · CSE 215 · Database

Part III

Normalization

Normalization

S4 — Normalization

Functional Dependencies

- Q1.** Consider a relation with schema $R(A, B, C, D, E)$ and functional dependencies: $A \rightarrow C$, $BE \rightarrow D$, $CD \rightarrow A$, $BC \rightarrow E$, and $AD \rightarrow B$.
- (a) Determine whether the following functional dependencies follow from the given FDs: $BC \rightarrow AD$, $AC \rightarrow BD$.
 - (b) Determine whether the following are keys of the given relation: $\{A, B\}$, $\{A, E\}$.

(10 marks)

[CSE 215 | L-2/T-1 | 2024–25]

- Q2.** Consider a relation with schema $R(A, B, C, D, E)$ and FDs: $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$, and $D \rightarrow A$.
- (a) What are all the nontrivial FDs that follow from the given FDs? Restrict yourself to FDs with single attributes on the right sides.
 - (b) What are all the keys of R ?
 - (c) What are all the super-keys that are not keys?

(10 marks)

[CSE 215 | L-2/T-2 | 2021–22]

- Q3.** Consider the following table:

FirstName	FirstInitial	LastName	LastInitial	FullName	FullAddress	ZipCode
Alice	A	Levy	L	Alice Levy	45h St, Seattle, 98185	98185
Bob	B	Levy	L	Bob Levy	45h St, Seattle, 98185	98185
Alice	A	Davis	D	Alice Davis	Oak Ave, Seattle, 98185	98185
Carol	C	Davis	D	Carol Davis	Sunnyvale, 94085	94085
Carol	C	Louis	L	Carol Louis	Oak Ave, Seattle, 98185	98185

- (a) Find all functional dependencies that hold on this table. You only need to write a minimal set of functional dependencies that logically imply all others.
- (b) Using the functional dependencies identified , decompose the relation into BCNF. Show your work and the final normalized schema including all keys.

(20 marks)

[CSE 215 | L-2/T-2 | 2020–21]

- Q4.** Consider a relation with schema $R(A, B, C, D)$ and FDs: $AB \rightarrow C$, $C \rightarrow D$, and $D \rightarrow A$.
- (a) What are all the nontrivial FDs that follow from the given FDs? Restrict yourself to FDs with single attributes on the right sides.
 - (b) What are all the keys of R ?
 - (c) What are all the superkeys that are not keys?

(15 marks)

[CSE 215 | L-2/T-2 | 2020–21]

- Q5.** Write down the six rules with examples to find out the closure of a set of functional dependencies. **(5 marks)** [CSE 215 | L-2/T-2 | 2019–20]

- Q6.** Write the steps to compute the closure of a set of attributes with respect to a set of functional dependencies (FD). Consider a relation with attributes A, B, C, D, E, F and the FDs:

$$AB \rightarrow C, \quad BC \rightarrow AD, \quad D \rightarrow E, \quad CF \rightarrow B$$

Find the closure of $\{A, B\}$, that is $\{A, B\}^+$. **(10 marks)** [CSE 215 | L-2/T-2 | 2017–18]

- Q7.** Given a set F of functional dependencies on $R(A, B, C)$:

$$A \rightarrow BC, \quad B \rightarrow C, \quad A \rightarrow B, \quad AB \rightarrow C$$

- (a) Compute the canonical cover for F .
- (b) Decompose R into R_1 and R_2 such that the decomposition is lossless. What are the normal forms of the decomposed relations?

(9 marks)

[CSE 303 | L-3/T-1 | 2016–17]

- Q8.** Given a set F of functional dependencies on $r_1(A, B, C, G, H, I)$:

$$A \rightarrow B, \quad A \rightarrow C, \quad CG \rightarrow H, \quad CG \rightarrow I, \quad B \rightarrow H$$

Compute the attribute closure $(AG)^+$ and show that AG is a candidate key of r_1 . **(6 marks)** [CSE 303 | L-3/T-1 | 2016–17]

Q9. Consider a relation $R(A, B, C, D)$ and FDs: $AB \rightarrow C$, $BC \rightarrow D$, $CD \rightarrow A$, and $AD \rightarrow B$.

- (a) What are all nontrivial FDs that follow from the given FDs? (Restrict yourself to FDs with single attributes on the right side.)
- (b) What are all the keys of R ?
- (c) What are all the superkeys for R that are not keys?

(12 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Q10. Consider a relation with schema $R(A, B, C, D)$ and FDs: $AB \rightarrow C$, $AD \rightarrow B$, $B \rightarrow D$. Find all non-trivial FDs that follow from the given FDs. (10 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Q11. Consider a relation with schema $R(A, B, C, D)$ and FDs: $AB \rightarrow C$, $C \rightarrow D$, and $D \rightarrow A$. What are the non-trivial FDs that follow from the given FDs? (8 marks)

[CSE 215 | L-3/T-1 | 2010–11]

Normal Forms (BCNF, 3NF, 4NF)

Q1. Suppose R is a relation with attributes $A_1, A_2, \dots, A_n$. As a function of n , determine how many superkeys R has for each of the following cases:

- (a) The only key is A_1 .
- (b) The only keys are $\{A_1, A_2\}$ and $\{A_1, A_3\}$.

(8 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Q2. For a relation schema $R(A, B, C, D, E)$, consider the following set of functional dependencies: $ABE \rightarrow C$, $BC \rightarrow D$, $BD \rightarrow E$, $DE \rightarrow A$. Decompose the relation into BCNF. Show the final schemas and the projected functional dependencies for each schema after decomposition. Is your final decomposition dependency-preserving? Justify your answer. (16 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Q3. Discuss an example where a table is in 3NF but not in BCNF. Then, modify the design to satisfy BCNF. (9 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Q4. Why is the normalization technique used in database design? Normalize the following table by drawing a dependency diagram and identifying all dependencies to derive up to 3NF. Show the schema in 3NF.

Attribute	Sample 1	Sample 2	Sample 3	Sample 4	Sample 5
INV_NUM	211347	211347	211347	211348	211349
PROD_NUM	AA-E3422QW	QD-300932X	RU-995748G	AA-E3422QW	GH-778345P
SALE_DATE	15-Jan-2010	15-Jan-2010	15-Jan-2010	15-Jan-2010	16-Jan-2010
PROD_LABEL	Rotary sander	0.25-in. drill bit	Band saw	Rotary sander	Power drill
VEND_CODE	211	211	309	211	157
VEND_NAME	NeverFail Inc.	NeverFail Inc.	BeGood Inc.	NeverFail Inc.	ToughGo. Inc
QUANT_SOLD	1	8	1	2	1
PROD_PRICE	\$49.95	\$3.45	\$39.99	\$49.95	\$87.75

(15 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Q5. Consider a relation with schema $R(A, B, C, D, E)$ and FDs: $AB \rightarrow C$, $DE \rightarrow C$, and $B \rightarrow D$. Decompose the relations, as necessary, into collections of relations that are in BCNF. Show your work and the final normalized schema including all keys. (20 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Q6. What do you know about BCNF and 3NF? What is the major difference between them? Let R be a schema that is not in BCNF and $\alpha \rightarrow \beta$ be the functional dependency that causes a violation of BCNF. How can R be decomposed such that R will be in BCNF? (10 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Q7. Suppose that we decompose the schema $R = (A, B, C, D, E)$ into (A, B, C) and (A, D, E) . Show whether this decomposition is a lossless decomposition if the following set F of functional dependencies holds:

$$A \rightarrow BC, \quad CD \rightarrow E, \quad B \rightarrow D, \quad E \rightarrow A$$

(10 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Q8. “A table in 3NF may fail to meet the criteria of BCNF” — do you agree? Show an appropriate example to validate your claim. (10 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Q9. The following schema has been designed for an address book application:

$$R(\underline{SSN}, Name, PhoneType, PhoneNumber)$$

The following FDs hold: $SSN \rightarrow Name$; $SSN, PhoneType \rightarrow PhoneNumber$; $SSN, PhoneType \rightarrow Name$; $PhoneNumber \rightarrow SSN, Name, PhoneType$.

- (a) What are the keys for R ?
- (b) Is R in BCNF? Why or why not?
- (c) Decompose R into BCNF (if not already in BCNF).

(15 marks)

[CSE 215 | L-2/T-2 | 2017–18]

↪ Similar: 2011–12: same question with additional subpart (iii) “Can you remove any dependency without violating equivalency of F ?”

Q10. Give an example of a relation which is in 3NF but not in BCNF. Define Multivalued Dependency. (10 marks) [CSE 215 | L-2/T-2 | 2017–18]

Q11. Prove that any two-attribute relation is in BCNF. (7 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Q12. For the relation schema $R(A, B, C, D, E)$ with FDs $AB \rightarrow C$, $DE \rightarrow C$, and $B \rightarrow D$:

- (a) Indicate all the BCNF violations.
- (b) Decompose the relations, as necessary, into a collection of relations that are in BCNF.

(10 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Q13. Suppose R is a relation with attributes $A_1, A_2, \dots, A_n$. As a function of n , find how many superkeys R has if the only keys are $\{A_1\}$ and $\{A_2, A_3\}$. (6 marks) [CSE 303 | L-3/T-1 | 2015–16]

Q14. Show an example where a table is in 3NF, but not in BCNF. Then modify the design to satisfy BCNF. (10 marks) [CSE 303 | L-3/T-1 | 2014–15]

↪ Similar: 2013–14: “Show an example where a table is in 3NF, but not in BCNF.” (5 marks)

Q15. Suppose that we have a schema $R = (A, B, C, D)$ with functional dependencies:

$$B \rightarrow C \quad D \rightarrow A$$

List the keys of R . Is R in BCNF? If not, convert R to BCNF. (15 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Q16. The following schema has been designed for an address book application:

$$R(SSN, Name, PhoneType, PhoneNumber)$$

A person can have different types of phone numbers (e.g. Mobile, Home or Office). The following set F of functional dependencies hold on R :

$$SSN \rightarrow Name$$

$$SSN, PhoneType \rightarrow PhoneNumber$$

$$SSN, PhoneType \rightarrow Name$$

$$PhoneNumber \rightarrow SSN, Name, PhoneType$$

- (a) What are the keys for R ?
- (b) Is R in BCNF? Why or why not?
- (c) Can you remove any dependency without violating equivalency of F ?
- (d) Decompose R into BCNF (if it is not already in BCNF).

(15 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Q17. Define BCNF, 3NF, and 4NF. Describe relationships among these normal forms. (17 marks) [CSE 215 | L-3/T-1 | 2010–11]

Q18. Describe the BCNF decomposition algorithm. (10 marks) [CSE 215 | L-3/T-1 | 2010–11]

Anomalies & Normalization

Q1. Suppose we are generating a **StaffPropertyInspection** table to include property inspection by staff members. When the staffs are required to undertake the inspections, they are allocated a company car for use on the day of the inspections. However, a car may be allocated to several staff members on the same day at different time slots. A member of staff may inspect several properties on a given date, but a property is only inspected once by a single staff member on a given date. Normalize the **StaffPropertyInspection** table (Figure 2) up to 2NF. Show only the following steps:

Hint: you have to perform and only show the results of the following steps:

- (a) Find all the candidate keys.
- (b) Determine the primary key.
- (c) Find all the functional dependencies.
- (d) Get the first normal form (1NF).

(e) Get the second normal form (2NF).

StaffPropertyInspection

propertyNo	iDate	iTime	pAddress	comments	staffNo	sName	carReg
PG4	18-Oct-00	10.00	6 Lawrence St, Glasgow	Need to replace crockery	SG37	Ann Beech	M231 JGR
PG4	22-Apr-01	09.00	6 Lawrence St, Glasgow	In good order	SG14	David Ford	M533 HDR
PG4	1-Oct-01	12.00	6 Lawrence St, Glasgow	Damp rot in bathroom	SG14	David Ford	N721 HFR
PG16	22-Apr-01	13.00	5 Novar Dr, Glasgow	Replace living room carpet	SG14	David Ford	M533 HDR
PG16	24-Oct-01	14.00	5 Novar Dr, Glasgow	Good condition	SG37	Ann Beech	N721 HFR

(10 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Q2. Normalize the following booking table up to 3NF by drawing a dependency diagram and identifying all dependencies. Show the resulting 3NF schema.

Date	Time	Customer	Contact	Event	Person	Total	Price
12 Aug, 10	12.30 PM	Maria Indrawan	04222223333	Lunch	Dora Smith	6	60
12 Aug, 10	12.30 PM	John Smith	04113333333	Master Class	Simon Becket	1	120
12 Aug, 10	7.00 PM	Mary Joe	95671290	Dinner	Linda Dee	2	150
13 Aug, 10	12.30 PM	Lindsay Smith	99031001	Lunch	Dora Smith	4	60
13 Aug, 10	12.30 PM	Sunshine Dee	99021900	Lunch	Dora Smith	2	60

(13 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Q3. Normalize the following invoice table up to 3NF by drawing a dependency diagram and identifying all dependencies. Show the resulting 3NF schema.

INV_NUM	PROD_NUM	SALE_DATE	PROD_LABEL	VEND_CODE	VEND_NAME	QTY	PRICE
211347	AA-E3422QW	15-Jan-2010	Rotary Sander	211	NeverFail, Inc	1	\$49.95
211347	QD-300932X	15-Jan-2010	0.25-in drill bit	211	NeverFail, Inc	8	\$3.45
211347	RU-995748G	15-Jan-2010	Band Saw	309	BeGood, Inc.	1	\$39.99
211348	AA-E3422QW	15-Jan-2010	Rotary Sander	211	NeverFail, Inc.	2	\$49.95
211349	GH-7781	16-Jan-2010	Power drill	157	ToughGood	1	\$87.75

(15 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Q4. What are the three anomalies in relational database schemas? Discuss them. (10 marks)

↪ Similar: 2013–14: same question (10 marks)

Q5. What are redundancy, deletion and update anomalies? (10 marks)

[CSE 303 | L-3/T-1 | 2011–12]

BUET · CSE 215 · Database

Part IV

Storage, Indexing & Query Processing

Storage, Indexing & Query Processing

S5 — Storage and Indexing

Data Storage Architecture & RAID

- Q1. Suppose we are using a RAID level 4 scheme with four data disks and one redundant disk, where each block is one byte. Calculate the block of the redundant disk if the corresponding blocks of data disks are: 01010110, 11000000, 00111011, and 11111011. (10 marks) [CSE 215 | L-2/T-1 | 2024–25]
- Q2. What is the purpose of using multiple clustering file organization? Give an example of its usage. Mention the disadvantages of this organization. (10 marks) [CSE 215 | L-2/T-1 | 2024–25]

Q3. The following table stores information about courses offered by BUET:

```
CREATE TABLE course (  
  course_code    CHAR(10) PRIMARY KEY,  
  course_title   VARCHAR(255) NOT NULL,  
  credit_hours   NUMERIC NOT NULL,  
  old_course_code CHAR(10) REFERENCES course(course_code)  
);
```

Here VARCHAR is variable-length, all other data types are fixed length, size of NUMERIC is 8 bytes, and size of CHAR(n) is n bytes. Draw the structure of the records, and the structure of the page when the records of this table are stored using the Slotted Page Structure. (10 marks) [CSE 215 | L-2/T-1 | 2022–23]

- Q4. A relational database table is stored in a database file using fixed-length records and a free list.
- (a) Show the structure of the file after deleting Record 2 (Mohammad Naim) and inserting a new row (Tamim Iqbal, Batter, 34).
- (b) Discuss how the table would be stored in a slotted page structure with a high-level block diagram showing only the records.

Header	Player Name	Role	Age	Free List
Record 0	Shakib Al Hasan	Allrounder	36	-
Record 1				
Record 2	Mohammad Naim	Batter	24	
Record 3	Taskin Ahmed	Bowler	28	
Record 4				
Record 5	Mushfiquir Rahim	Batter	36	
Record 6				
Record 7	Litton Das	Batter	28	
Record 8	Mustafizur Rahman	Bowler	28	

Figure: 5(c)

(10 marks) [CSE 215 | L-2/T-2 | 2021–22]

Q5. Briefly describe the differences between RAID-5 and RAID-6. (5 marks) [CSE 215 | L-2/T-2 | 2021–22]

Q6. A Megatron 800 Disk has the following pending disk requests (listed by track number). If the head is currently at track 71 and moving towards the higher track number, calculate average disk seek times for both the SCAN and C-SCAN algorithms.

Disk Requests: 21, 98, 12, 80, 85, 10

(10 marks) [CSE 215 | L-2/T-2 | 2021–22]

Q7. A table student has been created as follows:

```
CREATE TABLE STUDENT(  
  ID          VARCHAR2(10),  
  NAME        VARCHAR2(50),  
  CGPA        NUMBER(8),  
  ADDRESS     VARCHAR2(932),  
  DESCRIPTION  VARCHAR2(500)  
);
```

There are three blocks 1, 2 and 3 and the size of each block is 4000 bytes. Show the storage of 5 sample records of the student relation in these blocks using fixed-length record structure. No record can overlap the block boundary. (10 marks) [CSE 215 | L-2/T-2 | 2020–21]

Q8. Show and explain the slotted-page structure by inserting two variable length records with size S_1 and S_2 . The block size is 4000 bytes. Show the structure after deletion of the record of size S_1 . (15 marks) [CSE 215 | L-2/T-2 | 2020–21]

- Q9.** The **department** and **instructor** relations are given. Show the multi-table clustering file organization for these relations and explain the types of queries for which this organization will be efficient and the type for which it is inefficient.

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Physics	Watson	70000

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

(10 marks)

[CSE 215 | L-2/T-2 | 2020–21]

- Q10.** You have been given 6 disks of 4TB each and a student relation has 10 blocks: $B_0, B_1, \dots, B_9$. Show the storage of the student relation using:

- RAID level 5 storage system.
- RAID level 6 storage system.
- Compare RAID level 5 with RAID level 6.

(10 marks)

[CSE 215 | L-2/T-2 | 2020–21]

- Q11.** Explain disk access time and its implication in DBMS. (5 marks)

[CSE 215 | L-2/T-2 | 2019–20]

- Q12.** Compare between block-level striping and bit-level striping with examples. Show the storage of blocks $B_0, B_1, B_2, \dots, B_{11}$ of a relation r into a RAID level 5 storage system with 7 disks. (20 marks)

[CSE 215 | L-2/T-2 | 2019–20]

- Q13.** Explain the data recovery mechanism for single disk failure in RAID level 5 with an example. (10 marks)

[CSE 215 | L-2/T-2 | 2019–20]

- Q14.** Describe the storage mechanism, data read and update of data blocks for RAID level 5 storage with 4 disks (D0, D1, D2, D3) and a relation with 8 blocks (B0–B7). Describe the full recovery mechanism of all blocks of D1 if disk D1 fails. (15 marks)

[CSE 215 | L-2/T-2 | 2018–19]

- Q15.** Given the relational schema:

Student(id, name, DOB, cgpa, tot_cred, house_no, street, city, remarks, NID)
 Takes(id, course_no, semester, year, grade)
 Course(course_no, title, credit, pre_req)

Tuple sizes: Student = 400 bytes, Takes = 100 bytes, Course = 80 bytes. Block size = 8000 bytes. Show the organisation of the slotted page structure after insertion of:

- Two tuples of Student.
- Two tuples of Takes.
- One tuple of Course.

(15 marks)

[CSE 215 | L-2/T-2 | 2018–19]

- Q16.** Design a RAID level 6 scheme for six data disks (numbered 1–6) using the minimum number of redundant disks possible. You are not allowed to use any data disk as a redundant disk.

- Describe the recovery process if Disk 1 and Disk 3 fail simultaneously.
- Describe the recovery process if Disks 1, 3 and 6 fail simultaneously.

(16 marks)

[CSE 215 | L-2/T-2 | 2017–18]

- Q17.** Calculate the average time (with explanation) to read one block of data from a Megatron 747 disk (8192 tracks). Given data:

	Seek (ms)	Rot. Latency (ms)	Transfer (ms)	Total (ms)
Minimum	0	0	0.5	0.5
Maximum	17.4	15.6	0.5	33.5

(8 marks)

[CSE 215 | L-2/T-2 | 2017–18]

- Q18.** Schedule the following cylinder requests using the elevator algorithm. Use data from the Megatron 747 disk question. Assume moving the head requires 1 ms (start/stop) + 1 ms per 500 cylinders; head is initially on the 1st cylinder.

Cylinder	Arrival Time (ms)
2000	0
5000	10
1000	15
8000	25
9000	35
4000	50

(9 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Q19. What are the components of disk latency? (2 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Q20. The Megatron 777 disk has the following characteristics: ten surfaces with 10,000 tracks each; tracks hold an average of 1000 sectors of 512 bytes each; disk rotates at 10,000 rpm; time to move head n tracks is $1 + 0.001n$ milliseconds. Answer:

- (a) What is the capacity of the disk?
- (b) What is the maximum seek time?
- (c) What is the average rotational latency?

(5 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Q21. Another Megatron 777 disk incurs average rotational latency of 8.3 ms. Moving the head takes 1 ms to start/stop plus 1 ms per 500 cylinders. The disk controller receives the following requests:

Cylinder	First Available (ms)
1000	0
3000	0
7000	0
2000	20
8000	30
5000	40

Show the finishing time and latency for each request using the elevator algorithm. (10 marks) [CSE 303 | L-3/T-1 | 2016–17]

Q22. In RAID level 4, one redundant disk is used regardless of how many data disks there are. When any block is written, the redundant disk must also be updated. Briefly explain an optimal policy for updating the redundant disk with an example (consider 1 byte of data is written). Explain the logic behind adopting such a policy. (15 marks) [CSE 303 | L-3/T-1 | 2016–17]

Q23. What is stable storage? Briefly explain the idea of reading and writing policy in stable storage. (10 marks) [CSE 303 | L-3/T-1 | 2016–17]

Q24. Figure 3 shows a fixed-length record file structure for a **Customer** relation with fields: record#, CustomerID, Name, Street, City, Balance (records C0001–C0010). Show the file structure after deletion of records with customer IDs C004 and C007 for different file deletion methods. Compare the advantages and disadvantages among the different deletion methods.

Record	CustID	Name	Street	City	Balance
0	C0001	N1	North	Dhaka	1000
1	C0002	N2	North	Dhaka	2000
2	C0003	N3	North	Dhaka	3000
3	C0004	N4	North	Dhaka	4000
4	C0005	N5	North	Dhaka	5000
5	C0006	N6	South	Dhaka	6000
6	C0007	N7	South	Dhaka	7000
7	C0008	N8	South	Dhaka	8000
8	C0009	N9	South	Dhaka	9000
9	C0010	N10	South	Dhaka	10000

(15 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Q25. Discuss the applications where you will use RAID level 1 and RAID level 5. (5 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Q26. A file containing fixed-length records of a **student** relation (attributes: student_id, name, department_code, level, term) is given. Show the file structures after deletion of record 3 for the following cases and give comparative advantages/disadvantages:

- (a) Move record 4 to the space occupied by record 3, record 5 to record 4, and so on.
- (b) Move record 8 to the space occupied by record 3.
- (c) Mark record 3 as deleted and move no records. Use a header and record pointer.

(15 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Q27. The relation **student**(student_id, name, dept_code, level, term) and **department**(dept_code, location, floors, type) are given (dept_code is PK of department and FK in student).

- (a) Show the multitable clustering file structure for the given relations. Explain why executing `SELECT * FROM department` is more expensive in this structure.
- (b) Show the multitable clustering file structure with pointer chain. How is the query in (i) executed faster in this structure?

Record 0	1205001	N11	CSE	3	1
Record 1	1205004	N12	CSE	3	1
Record 2	1205007	N13	CSE	3	1
Record 3	1205010	N14	CSE	3	1
Record 4	1205013	N15	CSE	3	1
Record 5	1205016	N16	CSE	3	1
Record 6	1204001	N21	EEE	3	1
Record 7	1204002	N22	EEE	3	1
Record 8	1204003	N23	EEE	3	1

Figure 1: File containing records of relation $student(\underline{student\ id}, name, department\ code, level, term)$

CSE	ECE BUILDING	12	A
EEE	ME BUILDING	6	B

Figure 2: File containing records of relation $department(\underline{department\ code}, location, floors, type)$

(15 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Q28. Find a RAID level 6 scheme with 15 disks, four of which are redundant. (15 marks) [CSE 303 | L-3/T-1 | 2013–14]

Q29. What are the different ways to improve the performance of a disk-based system? (5 marks) [CSE 303 | L-3/T-1 | 2013–14]

Q30. Write down the pros and cons of an Elevator algorithm. (5 marks) [CSE 303 | L-3/T-1 | 2013–14]

Q31. The Megatron 747 disk has the following characteristics:

- 4 Platters providing 8 surfaces
- 8192 tracks per surface, 256 sectors per track, 512 bytes per sector
- Disk rotates at 3840 rpm (one rotation every 15.6 ms)
- Average seek time: 6.5 ms
- Transfer rate: 10 MB/s

(a) What is the capacity of a single track and what is the size of the entire disk?

(b) What is the average read time of 4096 bytes of data?

(7 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Q32. What are the different types of disk failures? How does a DBMS handle a disk crash? (8 marks) [CSE 303 | L-3/T-1 | 2011–12]

Q33. Find a RAID level 6 scheme with ten disks, such that it is possible to recover from the failure of any three disks simultaneously. You should use as many data disks as you can. (15 marks) [CSE 303 | L-3/T-1 | 2011–12]

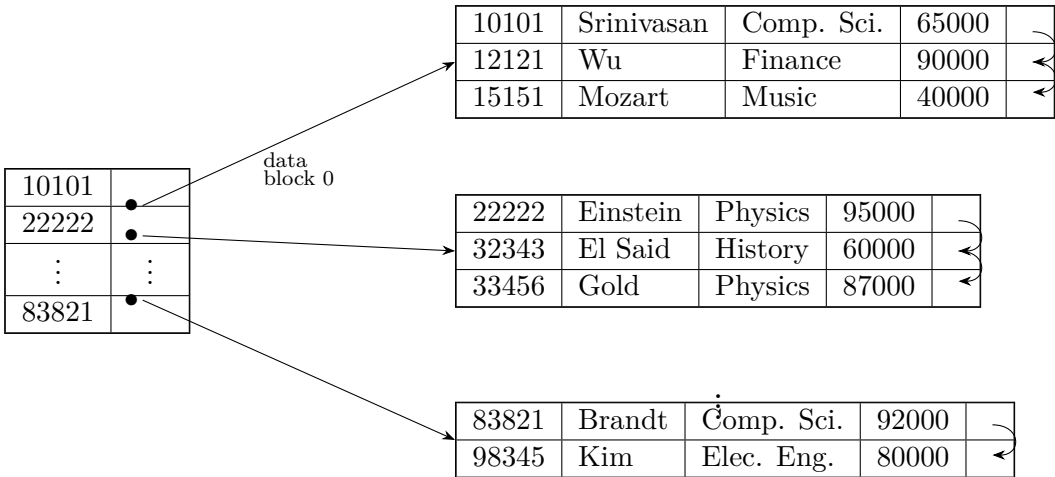
↪ Similar: CSE 215 | L-3/T-1 | 2012–13

Q34. What are the differences between RAID levels 1, 4, 5 and 6? (12 marks) [CSE 303 | L-3/T-1 | 2011–12]

Q35. Explain block level striping with an example of storing 1, 2, . . . , n logical blocks into a disk subsystem consisting of 8 disks. (5 marks) [CSE 215 | L-3/T-1 | 2010–11]

Primary Index

Q1. A primary index structure is given (one block contains only three records, no overflow block allowed). Show the index structure with the concerned blocks after insertion of the following tuples:



ID	Name	Dept_name	Salary
12122	Rafique	History	40000
83820	Abid	Comp. Sci.	90000

(10 marks)

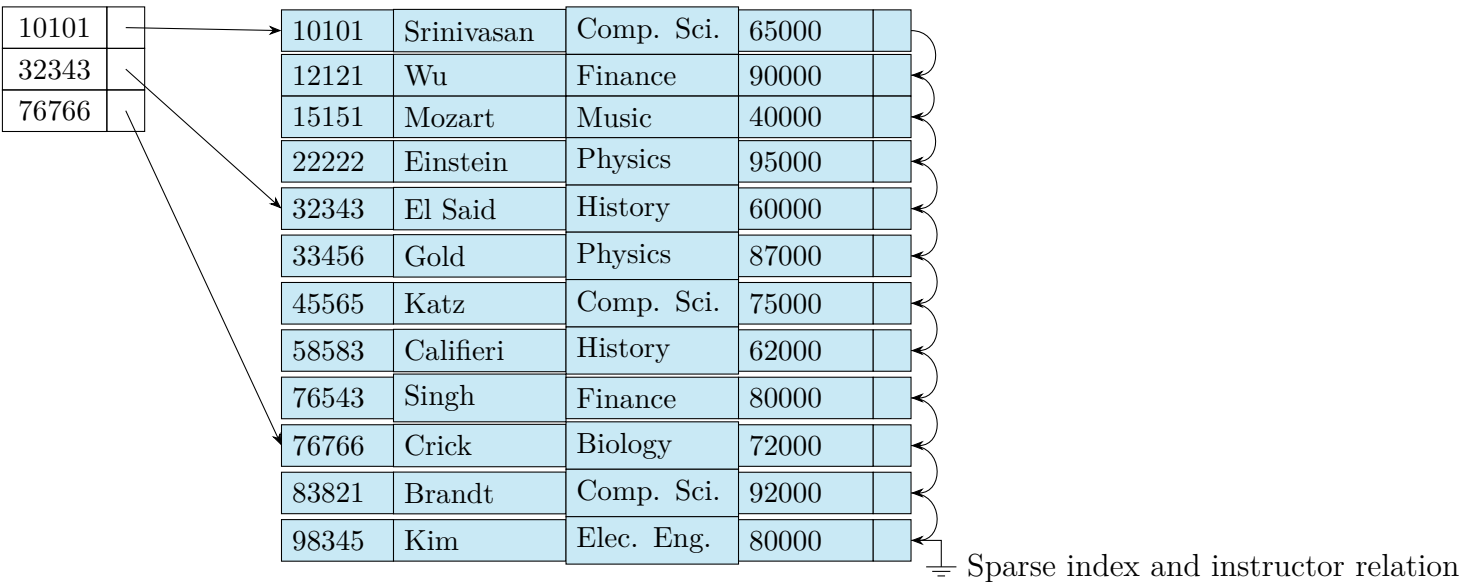
[CSE 215 | L-2/T-2 | 2020–21]

Q2. Compare primary index with secondary index showing representative index structures. (10 marks) [CSE 215 | L-2/T-2 | 2020–21]

Q3. Explain how the following queries will be executed using the sparse index shown in Figure 1:

- (a) SELECT * FROM instructor WHERE id = 22222
- (b) SELECT * FROM instructor WHERE id = 99999

Also explain the deletion of records from 10101 to 22222 from the instructor relation as per the deletion algorithm for a relation with a sparse index.



(15 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Q4. Consider a dictionary where each page contains only a single word along with its description. If we take the word from each page and form an index based on those words, what type of indexing (Dense/Sparse) would it be? Explain your answer. (5 marks) [CSE 215 | L-2/T-2 | 2017–18]

Q5. Create the following index structures for the given student relation:

- (a) A sparse index on `student_id` for search key values {1204001, 1205001, 1205010}.
- (b) A secondary index on `department_code`.
- (c) Delete the record with `student_id` = 1205010 and show the resulting index structures.
- (d) Insert the record {1205011, N114, EEE, 3, 1} and show the resulting index structures.

Record 0	1205001	N11	CSE	3	1
Record 1	1205004	N12	CSE	3	1
Record 2	1205007	N13	CSE	3	1
Record 3	1205010	N14	CSE	3	1
Record 4	1205013	N15	CSE	3	1
Record 5	1205016	N16	CSE	3	1
Record 6	1204001	N21	EEE	3	1
Record 7	1204002	N22	EEE	3	1
Record 8	1204003	N23	EEE	3	1

Figure 1: File containing records of relation *student(student id, name, department code, level, term)*

(20 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Q6. Explain the factors by which an indexing technique can be evaluated. (5 marks) [CSE 215 | L-3/T-1 | 2010–11]

Secondary & Bitmap Index

Q1. “Without bitmap index scan strategy, secondary index scan can sometimes perform worse than linear file scan.” — Justify or criticize this statement with proper explanation. (15 marks) [CSE 215 | L-2/T-2 | 2021–22]

Q2. The student relation has schema `Student(Id, name, f_name, m_name, cgpa, year_admit, division, district, thana, street, house_number, city, NID, DOB, total_credit)`. B⁺ tree secondary indices exist on `name`, `city`, and `year_admit` with heights 4, 2 and 1 respectively. The number of tuples for `name = 'Abid'`, `City = 'Feni'`, and `year_admit = 2015` are 50, 20 and 1000 respectively. Seek cost: 10 ms, block transfer cost: 0.1 ms. Number of blocks of student relation: 2000.

(a) Find the worst-case costs of processing the following conjunctive selection query using the single index method:

```
SELECT * FROM student WHERE name = 'Abid' AND City = 'Feni' AND year_admit = 2015
```

(b) Explain clearly how this query will be executed in this method.

(10 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Q3. A relation schema $R(A, B, C)$ and a relation $r(R)$ is given below. The relation is physically sorted on the primary key A . Construct an ordered secondary index of r on attribute B considering the buckets containing maximum 3 pointers. Explain the cases if you need to insert the tuple $t = \langle a_8, b_2, c_2 \rangle$.

A	B	C
a_1	b_1	c_1
a_2	b_2	c_1
a_3	b_3	c_1
a_4	b_1	c_1
a_5	b_2	c_1
a_7	b_2	c_1
a_8	b_3	c_1

(15 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Q4. The Employee Management System has schema `employee(e_id, name, date_of_birth, salary, street, thana, district, NID)`. Total employees: 40,000; relation size: 4,000 blocks. Primary index: `e_id`; secondary indices: `name`, `district`, `NID` with heights 3, 2, 4 respectively. Transfer time: 0.5 ms/block; average seek time: 8 ms. Tuples with `salary = 50000`: 100; `district = 'Comilla'`: 2000 (each tuple in a different block).

Find the query costs using indices for:

```
SELECT * FROM employee WHERE salary = 50000 OR district = 'Comilla'
```

(8 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Q5. Give an example where the Bit-map index is preferred over other indexing techniques. (5 marks) [CSE 215 | L-2/T-2 | 2017–18]

Q6. Is it possible to implement an unclustered-sparse indexing scheme? If so, give an example. Otherwise, briefly explain the reason. (6 marks) [CSE 303 | L-3/T-1 | 2016–17]

Q7. Write a short note on bitmap indexing. (5 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Q8. Given the relation `employee(emp_id, name, gender, status)`, create bitmap indices on `gender` and `status`.

E001	Abid	Male	Full Time
E002	Arif	Male	Full Time
E003	Sajid	Male	Full Time
E004	Rafiq	Male	Part Time
E005	Shahid	Male	Full Time
E006	Sajid	Male	Full Time
E007	Karim	Male	Part Time
E008	Zahid	Male	Full Time

Figure 3: File containing records of relation `employee(emp_id, name, gender, status)`

(5 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Q9. Give an example where the Bitmap index is preferred over other indexing techniques. (5 marks) [CSE 303 | L-3/T-1 | 2012–13]

Q10. The relation `account` is given in Figure 1. Construct bitmap indices on `type`, `branchname` and `location` attribute. Explain how you can find balance for `branchname = 'B2'` and `location = 'L1'` and `type = 'D'`. (10 marks) [CSE 215 | L-3/T-1 | 2010–11]

Q1. Given a B^+ tree index with 10 million records, node size = 4 kilobytes, search-key size = 8 bytes, disk-pointer size = 4 bytes. Answer the following:

- Given a search-key, how many index blocks need to be read from disk in the best case to find the pointer to the data record?
- Given a range query assuming 1000 records will be retrieved, how many index blocks need to be read in the best case?
- Given a search-key, how many index blocks need to be read in the best and worst cases if we use a hash index instead? Assume the hash index requires at most three overflow blocks.

(15 marks)

[CSE 215 | L-2/T-1 | 2024–25]

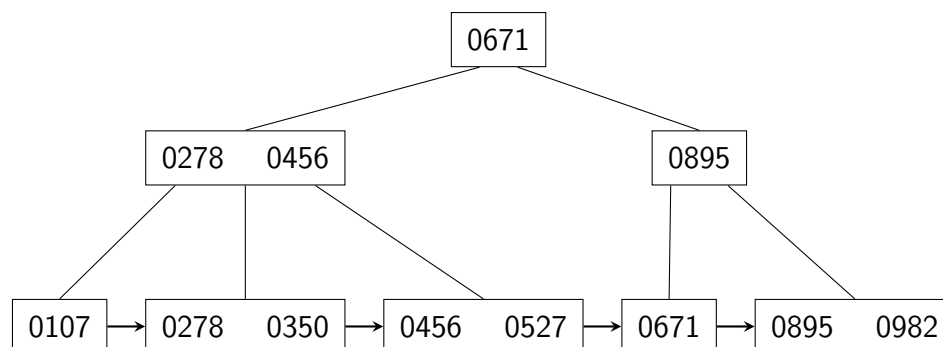
Q2. Assume a B^+ tree index is initially empty and the number of pointers that fit in one node is 4.

- Construct the tree by inserting the following key values in the given order: 2, 3, 5, 7, 11, 17, 19, 23, 29, 31.
- Show the final form of the tree after: Delete 23, Delete 19.

(13 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Q3. The following is a degree-3 B^+ tree created after some insert/delete operations:



Draw the state of this B^+ tree after each of the following operations:

- Insert 315
- Insert 409 and 513
- Delete 456
- Delete 513
- Delete 982 and 895

(20 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Q4. For a degree-5 B^+ tree, what will be the height of the B^+ tree when the numbers from 1 to 100 are inserted sequentially? *Hint:* A tree with l leaves and internal nodes of degree d has height approximately $\log_d l$. (5 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Q5. (a) Construct a B^+ tree index structure with $n = 4$, incrementally inserting the search keys 90, 80, 70, 60, 50, 40, 30, 20. (b) Show the structures after deletion of 20, 30, 40 and 50.

(20 marks)

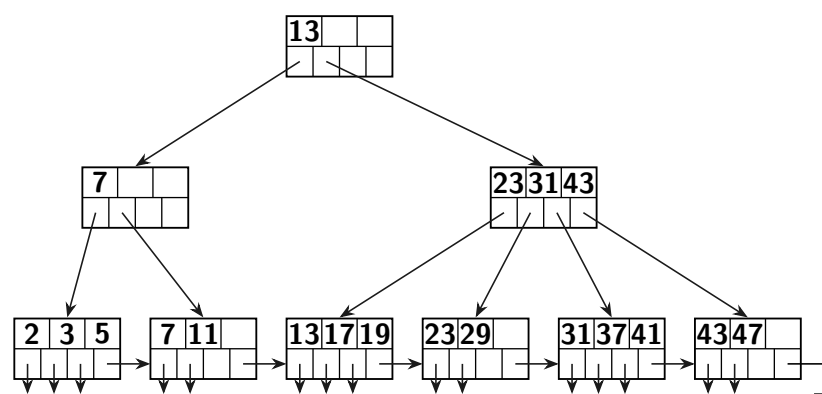
[CSE 215 | L-2/T-2 | 2020–21]

Q6. Construct a B^+ tree primary index for the relation r of question 2(a) (relation $R(A, B, C)$ sorted on primary key A , with keys $a_1, a_2, a_3, a_4, a_5, a_7, a_8$) with $n = 3$ by insertion of search keys incrementally in sorted order and splitting of nodes. After construction of the tree, insert a_6 and show the tree structure. (20 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Q7. Consider a given B^+ tree (Figure for Q4(c)). Modify it incrementally according to the following instructions. For each instruction, draw the final modified state:

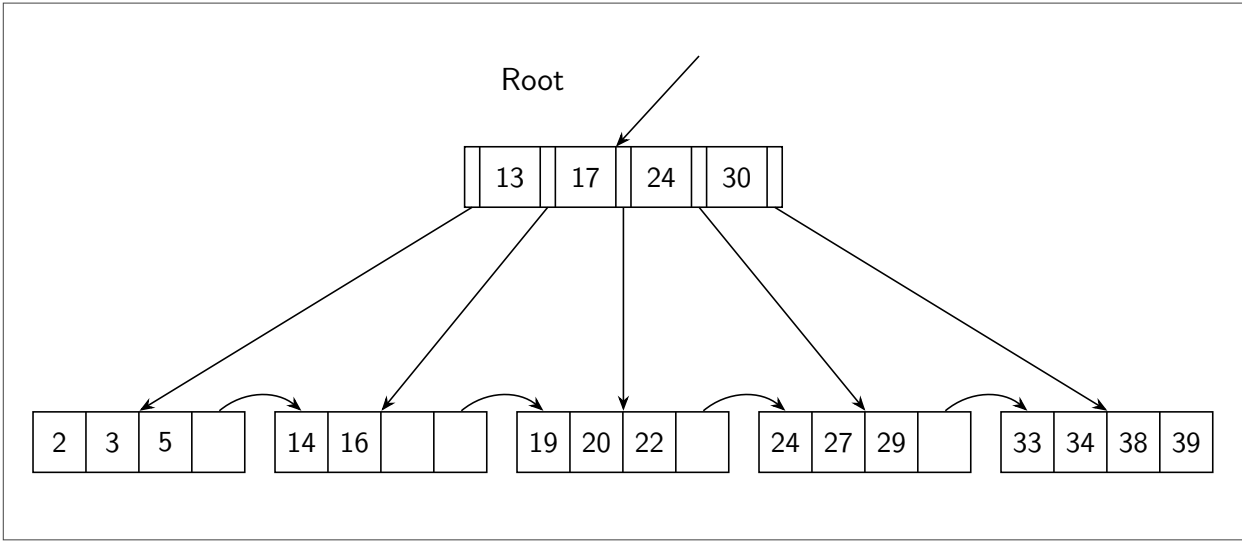
- Insert keys 8, 9
- Insert keys 24, 27
- Remove keys 8, 9
- Remove keys 24, 27



(16 marks)

[CSE 215 | L-2/T-2 | 2017–18]

- Q8.** Consider a B⁺ tree with leaf nodes: {2, 3, 5}, {14, 16}, {19, 20, 22}, {24, 27, 29}, {33, 34, 38, 39}. Show (without explanation) what happens after each of the following successive operations:
- (a) Insert 7
 - (b) Insert 8
 - (c) Delete 19
 - (d) Delete 20



(8 marks)

[CSE 303 | L-3/T-1 | 2016–17]

- Q9.** Write down the key properties of a B⁺ tree. (8 marks) [CSE 303 | L-3/T-1 | 2016–17]
- ↪ Similar: 2017–18: same question (8 marks)

- Q10.** Show the final B⁺ tree index structure for the search key values 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24 with 4 leaf nodes and $n = 4$. (You do not need to create this index step by step.) Then show the index structure after insertion of 15. (20 marks) [CSE 303 | L-3/T-1 | 2015–16]

- Q11.** Discuss the basic principles of B⁺ tree index structure. (10 marks) [CSE 303 | L-3/T-1 | 2014–15]

- Q12.** Compare Hash-based index, B⁺-tree index, and Bitmap index. (6 marks) [CSE 303 | L-3/T-1 | 2013–14]

- Q13.** Draw the B-tree to index the following elements: 2, 3, 5, 13, 17, 19, 23, 29, 31, 37, 41, 43, and 47. Delete elements 31 and 43 and redraw the tree. (17 marks) [CSE 303 | L-3/T-1 | 2012–13]

- Q14.** Write down the key properties of a B⁺-tree. (8 marks) [CSE 303 | L-3/T-1 | 2011–12]

- Q15.** Draw a B⁺-tree for the following keys: 2, 3, 5, 7, 14, 16, 20, 34, 38, and 40. Assume a 2-order B⁺-tree where a node must have 2 to 4 keys. Re-construct the tree after deleting 3, 5, and 7 from the original tree. (15 marks) [CSE 303 | L-3/T-1 | 2011–12]

- Q16.** Suppose in a B⁺-tree, pointers are 4 bytes long and keys are 12 bytes long. How many keys and pointers will a block of 16,384 bytes have? For a one million record table, what would be the expected length of the index? (12 marks) [CSE 303 | L-3/T-1 | 2011–12]
- ↪ Similar: CSE 215 | L-3/T-1 | 2012–13 (same pointer/key sizes, 16384 byte block)

- Q17.** Construct a B⁺ tree for the set of key values {2, 4, 6, 8, 12, 18, 20, 24, 30, 32} with $n = 4$. Assume that the tree is initially empty and values are added in the given order. (10 marks) [CSE 215 | L-3/T-1 | 2010–11]

Hash Index

- Q1.** Suppose that extendable hashing is being used on a database file that contains records with the following search key values: 2, 3, 5, 7, 11, 17, 19, and 23. Construct the extendable hash structure for this file if the hash function is $h(x) = x \bmod 7$ and each bucket can hold at most three records. (15 marks) [CSE 215 | L-2/T-2 | 2021–22]
- Q2.** Insert all the following data incrementally using the Product Id in an initially empty extensible hashing system using hash function $h(x) = x \bmod 1024$. Show the state of the system after each insertion:

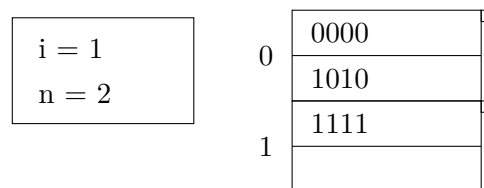
Product Id	Quantity	Price per Qty
03	10	5
05	5	90
09	10	1200
11	12	50
13	11	89
18	2	65

(9 marks)

Insert all the data from the product table above, incrementally using the Product Id in an initially empty linear hashing system. Show the state of the system after each insertion. (12 marks) [CSE 215 | L-2/T-2 | 2017–18]

Q3. What is the key property of a good hash function? How can you choose such a function? (6 marks) [CSE 303 | L-3/T-1 | 2016–17]

Q4. The following figure shows a linear hash table with 3 records. We assume hash function h produces 4 bits and we represent records by the value produced by h when applied to the search key of the record. Here, i is the number of lower order bits of the hash function currently used and n is the current number of buckets. Each bucket can hold maximum of 2 records. We shall adopt the policy of choosing n , so that there are no more than $1.7n$ records in the file.



Now, show (step by step) what happens after successive insertion of 0101, 0001 and 0111. Show the necessary calculation whenever a new bucket is required. Overflow blocks can be used if necessary. (15 marks) [CSE 303 | L-3/T-1 | 2016–17]

Q5. Construct the dynamic hash index structure on the **balance** attribute for the customer relation (records C0001–C0010, balances 1000–10000). The binary representation of the most significant digit of balance is the hash code. Insert records in descending order of balance.

Record	CustID	Name	Street	City	Balance
0	C0001	N1	North	Dhaka	1000
1	C0002	N2	North	Dhaka	2000
2	C0003	N3	North	Dhaka	3000
3	C0004	N4	North	Dhaka	4000
4	C0005	N5	North	Dhaka	5000
5	C0006	N6	South	Dhaka	6000
6	C0007	N7	South	Dhaka	7000
7	C0008	N8	South	Dhaka	8000
8	C0009	N9	South	Dhaka	9000
9	C0010	N10	South	Dhaka	10000

(20 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Q6. Given the **employee**(**emp_id**, **name**, **gender**, **status**) relation with hash values on **name**, create step by step a dynamic hash index structure for **name**. Assume a bucket can contain only one record and an overflow bucket can be used.

E001	Abid	Male	Full Time
E002	Arif	Male	Full Time
E003	Sajid	Male	Full Time
E004	Rafiq	Male	Part Time
E005	Shahid	Male	Full Time
E006	Sajid	Male	Full Time
E007	Karim	Male	Part Time
E008	Zahid	Male	Full Time

Figure 3: File containing records of relation *employee(emp_id, name, gender, status)*

Name	16 bit hash code $h(\text{Name})$
Abid	0001111100001111
Arif	0010111100001111
Sajid	0011111100001111
Rafiq	0100111100001111
Shahid	0101111100001111
Karim	1001111100001111
Zahid	0111111100001111

Figure 4: The list of names of figure 3 and corresponding 16 bit hash codes.

(15 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Q7. Show the state of the linear hashing after inserting items: 0000, 0011, 0010, 0101, 1010, 1011, 1110, and 1111. Then show the state after inserting 1101 and 0111 into the hash table. (14 marks) [CSE 303 | L-3/T-1 | 2013–14]

Q8. Discuss Linear Hashing and Extensible Hashing with examples. How do they differ from each other? (13 marks) [CSE 303 | L-3/T-1 | 2012–13]

Q9. Describe static hash file organization with an example. How can bucket overflows be handled? (10 marks) [CSE 215 | L-3/T-1 | 2010–11]

S6 — Query Processing

Query Processing

Q1. Assume that only one tuple fits in a block and memory holds at most three blocks. Show the runs created on each pass of the sort-merge algorithm when applied to sort the following tuples on the first attribute:

(kangaroo, 17), (wallaby, 21), (emu, 1), (wombat, 13), (platypus, 3), (lion, 8),
(warthog, 4), (zebra, 11), (meerkat, 6), (hyena, 9), (hornbill, 2), (baboon, 12).

(10 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Q2. Describe the two-pass algorithm for grouping and aggregation using sorting. Suppose the number of blocks in relation R is 50,000 and in relation S is 20,000. Compute the memory and I/O cost for the sort-based union operation $R \cup S$. (14 marks) [CSE 215 | L-2/T-1 | 2023–24]

Q3. Describe how aggregation and set operations of an SQL query are processed and whether pipelining can improve their performance. (10 marks) [CSE 215 | L-2/T-2 | 2021–22]

Q4. Memory size is M blocks and the number of blocks of a relation r is b_r . Find the number of runs N , number of merge passes, number of block transfers, and number of seeks to sort the relation r using the external sort-merge algorithm for the following cases:

- (a) Case 1: $N < M$
- (b) Case 2: $N = M$
- (c) Case 3: $N > M$

(15 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Q5. An unsorted relation r_{unsorted} has 36 blocks (B1–B36) and the memory size for external sort-merge is 4 blocks. Show the diagram to sort the relation using the external sort-merge algorithm and store it as r_{sorted} . Using the diagram, explain the total number of block transfers needed to sort the relation and store it. (15 marks) [CSE 215 | L-2/T-2 | 2018–19]

Q6. Explain the algorithm used to process the following SQL query given secondary indices on **salary** and **district**:

`SELECT * FROM employee WHERE salary = 50000 OR district = 'Comilla'`

(7 marks)

[CSE 215 | L-2/T-2 | 2018–19]

Q7. Write the steps of a one-pass algorithm for finding $S - R$, where the number of records in relation R is smaller than the number of records in relation S . (6 marks) [CSE 215 | L-2/T-2 | 2017–18]

Q8. Write down the pseudocode for a sort-based union algorithm that computes $R \cup S$. Also mention its runtime and constraints in terms of $B(R)$, $B(S)$ and M , where $B(R)$ and $B(S)$ are the number of blocks in R and S , and M is the number of blocks of each sublist to be sorted. (12 marks) [CSE 303 | L-3/T-1 | 2016–17]

Q9. Explain external sort-merge with an example. Show that the total number of block transfers for external sorting of a relation r is:

$$b_r \left(2 \left\lceil \log_{M-1} \left(C \frac{b_r}{M} \right) \right\rceil + 1 \right)$$

where b_r is the number of blocks of relation r and M is the size of the main memory buffer. (15 marks) [CSE 303 | L-3/T-1 | 2014–15]

Q10. Let relation $r_1(A, B, C)$ and $r_2(C, D, E)$ have the following properties: r_1 has 20,000 tuples, r_2 has 45,000 tuples, 25 tuples of r_1 fit on one block, and 30 tuples of r_2 fit on one block. Estimate the number of block transfers and seeks required using:

- (a) Nested loop join.
- (b) Block nested loop join.

(10 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Q11. Let t_r and t_s be the time to transfer a block of data and the time to seek a block in a disk subsystem, respectively. Estimate the query processing cost for:

- (a) A primary B⁺ tree index and equality on key.
- (b) A primary B⁺ tree index and comparison on key.

(10 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Q12. Write down the steps of two-phase multiway merge sort. Consider a system where the block size is B bytes, memory size is M bytes, and record size is R bytes. What is the maximum number of records that can be sorted using two-phase merge sort? If the number of records exceeds this maximum, how can you sort them? (15 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Q13. Show the steps of duplicate elimination using a two-pass algorithm. Derive the relationship between number of records and memory size in this algorithm. (15 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Q14. Discuss the subsystems that are related to query processing in a Database Management System (DBMS). (10 marks) [CSE 303 | L-3/T-1 | 2012–13]

Q15. Write the steps of performing a one-pass algorithm for finding the maximum value of an attribute for relation R . If the buffer size is M , what is the relationship between R and M ? Derive the relationship between M and R for a two-pass aggregate operation. (20 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Q16. Derive the total number of disk I/Os for nested loop join of two relations R and S , where the number of records in R is smaller than the number of records in S . (5 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Query Optimization

Q1. Given three relations $R(a, b, c)$, $S(d, e)$, and $T(e, f, g)$, apply query optimization techniques to find the optimized logical plan for the following expression:

$$\sigma_{a>100, b=20, c=d, f<100}((R \times S) \bowtie T)$$

(7 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Q2. For each of the following statements, determine whether it is true. If it is not, provide a counterexample to justify your answer.

- (a) $\delta(\pi_L(R)) = \pi_L(\delta(R))$
- (b) $\delta(\sigma_C(R)) = \sigma_C(\delta(R))$
- (c) $\pi_L(R \cup S) = \pi_L(R) \cup \pi_L(S)$

(9 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Q3. Consider the relations $r_1(A, B, C)$, $r_2(C, D, E)$, and $r_3(E, F)$. Assume that there are no primary keys. Let $V(C, r_1) = 900$, $V(C, r_2) = 1100$, $V(E, r_2) = 50$, and $V(E, r_3) = 100$. Assume that r_1 has 1000 tuples, r_2 has 1500 tuples, and r_3 has 750 tuples. Compute the size of $r_1 \bowtie r_2 \bowtie r_3$. (15 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Q4. Construct an optimized query plan for the following query using equivalence rules. Draw the query plan using an expression tree and justify the plan with explanations.

Schema:

Student (sid, name, major)

Enroll (sid, course, section, term, grade)

Class (course, section, term, instructor, room, time)

Query

```
SELECT S.name, C.instructor, C.term
FROM Student S, Enroll E, Class C
WHERE S.sid = E.sid AND E.course = C.course
AND E.section = C.section AND E.term = C.term
AND C.instructor = 'Hinson' AND S.major = 'ML' ;
```

Equivalence Rules (The notations carry usual meaning)

- (a) $\sigma_\theta(E_1 \times E_2) = E_1 \bowtie_\theta E_2$
- (b) $\sigma_{\theta_0}(E_1 \bowtie_\theta E_2) \equiv (\sigma_{\theta_0}(E_1)) \bowtie_\theta E_2$
- (c) $\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_\theta E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_\theta (\sigma_{\theta_2}(E_2))$
- (d) $\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = \Pi_{L_1}(E_1) \bowtie_\theta \Pi_{L_2}(E_2)$
- (e) $\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = \Pi_{L_1 \cup L_2}(\Pi_{L_1 \cup L_3}(E_1) \bowtie_\theta \Pi_{L_2 \cup L_4}(E_2))$

(15 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Q5. Consider the following database schema:

$R(A, B), S(B, C), T(C, D)$

Write a relational algebra plan for the SQL query below. Give the expression first, then convert it to a tree. Finally find the most optimized tree by pushing down selections and projections as much as possible.

```
SELECT R.A, T.D
FROM T, S, R
WHERE T.C = S.C
      AND S.B = R.B
      AND T.D > 20
```

(10 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Answer

1. Initial Relational Algebra Expression

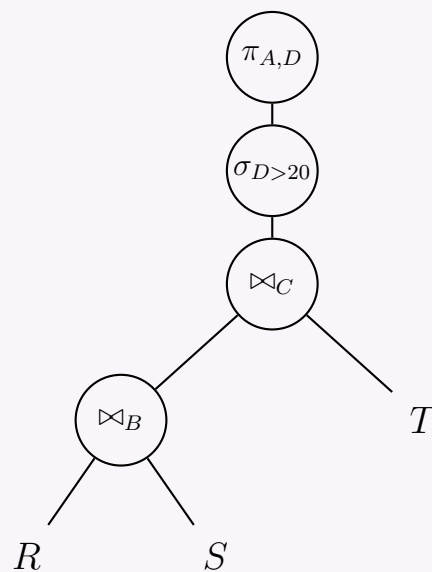
The canonical translation of the provided SQL query maps the **FROM** clause to Cartesian products (or joins), the **WHERE** clause to selections, and the **SELECT** clause to a final projection. Given the explicit equality conditions $T.C = S.C$ and $S.B = R.B$, we use equi-joins (or natural joins, assuming identical attribute names represent the join keys).

The initial, unoptimized relational algebra expression is:

$$\pi_{A,D}(\sigma_{D>20}((R \bowtie_B S) \bowtie_C T))$$

2. Initial Query Tree

The initial query tree translates the expression directly into a logical execution plan, evaluating from the bottom up. At this stage, all rows from R , S , and T participate in the joins before the filter on D is applied.



3. Optimization Strategy (Heuristic Rules)

To achieve the most optimized tree, we systematically apply relational algebra equivalence rules to push down operators:

- **Rule 1: Push Down Selections (σ).**

The selection $\sigma_{D>20}$ only references attribute D , which is strictly native to relation $T(C, D)$. We can commute the selection with the join operations and push it directly onto T . This significantly reduces the cardinality of T before it enters the join.

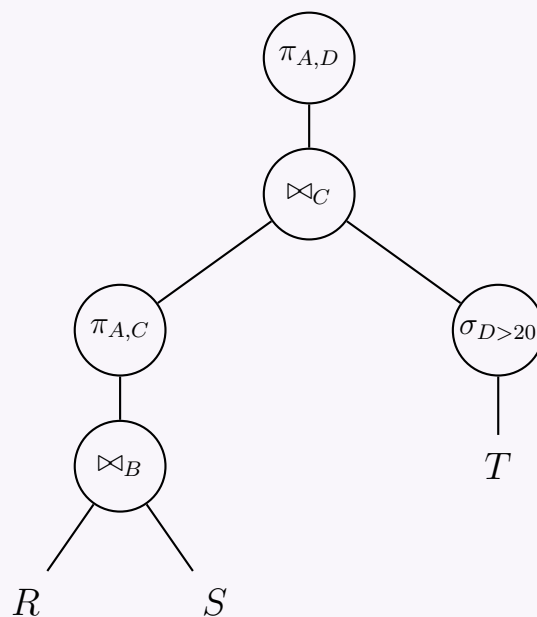
- **Rule 2: Push Down Projections (π).**

We evaluate the required attributes at every node (for both intermediate join conditions and the final output) to discard unnecessary columns as early as possible.

- **From $R(A, B)$:** Attribute A is needed for the final output, and B is needed for the join. Keep both.
- **From $S(B, C)$:** Attribute B is needed to join with R , and C is needed to join with T . Keep both.
- **From $T(C, D)$:** Attribute C is needed to join with S , and D is required for the output and selection. Keep both.
- **Intermediate Result $(R \bowtie_B S)$:** The output of this join contains attributes (A, B, C) . Moving upward, the subsequent join with T requires only C , and the final projection requires only A . Therefore, attribute B is no longer necessary. We can introduce an intermediate projection $\pi_{A,C}$ immediately after the first join to eliminate B early, reducing the memory footprint of the intermediate tuples.

4. Optimized Query Tree

Applying the pushdown heuristics yields the highly optimized execution plan below:



Advantages of this Optimized Tree:

- (a) Applying $\sigma_{D>20}$ natively on T limits disk I/O and prevents processing invalidated tuples during the $\bowtie_C$ operation.
- (b) Splicing $\pi_{A,C}$ above the $\bowtie_B$ relation minimizes the width of the pipelined records streaming into the upper $\bowtie_C$ node.

Exam Tip: Always double-check your attribute tracking when introducing intermediate projections. A common pitfall is discarding an attribute (like C) too early, breaking a join condition located higher up in the tree!

Q6. The following relations are given:

```
Student(id, name, CGPA, street, city)
takes(id, course-id, semester, year)
course(course-id, title, credit)
```

An SQL statement:

```
SELECT id, name, course-id, title
FROM student s, takes t, course c
WHERE s.id = t.id
      AND t.course-id = c.course-id
      AND s.city = 'Dhaka'
      AND c.credit = 3
```

- (a) Construct an expression tree for the given SQL.
- (b) Explain materialization evaluation and pipeline evaluation using the expression tree.

(20 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Q7. Express the following query using relational algebra and draw the equivalent logical query tree:

```
SELECT title
FROM StarsIn
WHERE starsName IN (
    SELECT name
    FROM MovieStar
    WHERE name LIKE '%Khan'
          AND birthdate > '18-MAR-1990'
);
```

(9 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Q8. Show a logical query plan for the following query:

```
SELECT title
FROM StarsIn
WHERE starName IN (
    SELECT name
    FROM MovieStar
    WHERE birthdate LIKE '%1960'
);
```

(8 marks)

[CSE 303 | L-3/T-1 | 2012–13]

- Q1.** Differentiate between the Nested-Loop Join algorithm and the Block Nested-Loop Join algorithm. **(5 marks)** [CSE 215 | L-2/T-1 | 2024–25]
- Q2.** Assuming no indexes exist, which one would you apply for faster operation: a Hash Join or a Sort-Merge Join? Explain why. **(6 marks)** [CSE 215 | L-2/T-1 | 2024–25]
- Q3.** Describe the hybrid hash join algorithm. Compute the memory and I/O cost of the algorithm. **(11 marks)** [CSE 215 | L-2/T-1 | 2023–24]
- Q4.** Construct a one-and-a-half pass algorithm for computing $RLOJ S$ (natural left outer join), where R is the smaller relation. Compute the memory and disk I/O requirements. **(10 marks)** [CSE 215 | L-2/T-1 | 2022–23]
- Q5.** Assume R and S are two database relations with $B(R) = 8000$, $B(S) = 4000$, and the number of available memory buffers $M = 101$. Compute the I/O cost of the natural join operation for the following algorithms:
- (a) Block-based nested loop join.
 - (b) Sort-based nested loop join.
 - (c) Hash-based nested loop join.
- (8 marks)** [CSE 215 | L-2/T-1 | 2022–23]
- Q6.** Give a two-pass hash-based algorithm for computing the set difference $R - S$. Analyse the memory and disk I/O requirements of your algorithm. **(10 marks)** [CSE 215 | L-2/T-1 | 2022–23]
- Q7.** What is the I/O complexity of needed loop join ? **(3 marks)** [CSE 303 | L-3/T-1 | 2013–14]

BUET · CSE 215 · Database

Part V

Transaction Management & Advanced Topics

Transaction Management & Advanced Topics

S7 — Transaction Management

Transactions & ACID Properties

- Q1.** Explain the ACID properties of a transaction with an example. (5 marks) [CSE 215 | L-2/T-2 | 2020–21]
- Q2.** Show the relationship among memory buffer, transaction work area and disk storage. Explain how data A is read from disk for a transaction and data B is written to the disk for a transaction. (10 marks) [CSE 215 | L-2/T-2 | 2020–21]
- Q3.** Two transactions are given:
- (a) Transaction T_1 transfers the sum of 20% of balances of accounts A and B to account C .
 - (b) Transaction T_2 transfers Tk. 2000 from account D to account C .
- Prepare a concurrent schedule for the above two transactions and prove that the schedule is conflict serializable. (15 marks) [CSE 215 | L-2/T-2 | 2018–19]
- Q4.** Briefly explain each of the ACID properties in DBMS. (8 marks) [CSE 303 | L-3/T-1 | 2016–17]
↪ Similar: 2017–18 / 2018–19: “Explain the ACID properties of a transaction with an example. Why are these properties essential for the concurrent execution of transactions?”
- Q5.** What do you mean by the ACID properties of a DBMS? Give real-life examples (one for each property) showing the necessity of these properties. (8 marks) [CSE 303 | L-3/T-1 | 2013–14]
- Q6.** Which two ACID properties are maintained by the Recovery Manager of a DBMS? (5 marks) [CSE 303 | L-3/T-1 | 2012–13]
- Q7.** What do you mean by the ACID properties in a DBMS? Give one example of each property of the ACID. (12 marks) [CSE 303 | L-3/T-1 | 2011–12]
- Q8.** Explain the transaction states with the state diagram. (5 marks) [CSE 215 | L-3/T-1 | 2010–11]

Concurrency Control

- Q1.** Define the Strict 2PL (two-phase locking) protocol. Show that a 2PL schedule is conflict-serializable. (9 marks) [CSE 215 | L-2/T-1 | 2024–25]
- Q2.** Determine whether the following schedules are conflict-serializable by constructing precedence graphs:
- (a) $r_1(A); w_2(A); w_3(A); w_2(B); r_2(C); r_2(B); w_2(C)$
 - (b) $w_1(A); r_2(A); w_2(B); r_2(B); w_3(C); r_4(C); w_4(D); r_3(D); r_3(C)$
- (8 marks) [CSE 215 | L-2/T-1 | 2024–25]
- Q3.** Consider the two transactions given below:
- $$T_1: r_1(A); w_1(B)$$
- $$T_2: r_2(A); w_2(A); w_2(B)$$
- Determine the number of possible schedules, number of serializable schedules, and number of conflict-serializable schedules that can be constructed for the given transactions. (9 marks) [CSE 215 | L-2/T-1 | 2024–25]
- Q4.** Consider the two transactions given below:
- $$T_1: sl_1(A); xl_1(B); u_1(A); u_1(B)$$
- $$T_2: sl_2(B); xl_2(A); u_2(B); u_2(A)$$
- (a) Show a schedule that results in a deadlock but is conflict-serializable to (T_2, T_1) .
 - (b) Does introducing update locks solve the deadlock problem? Justify your answer.
 - (c) Show the lock compatibility matrix for shared, exclusive, and update locks, indicating in each cell whether concurrent locks are allowed (“Yes”) or not allowed (“No”).
- (10 marks) [CSE 215 | L-2/T-1 | 2024–25]
- Q5.** Determine whether each of the following schedules is recoverable or ACR (avoids cascading rollback). Here c_i represents the commit action of transaction T_i .
- (a) $S_1: r_1(A); w_1(A); r_2(A); w_2(A); c_2; r_1(B); w_1(B); c_1$
 - (b) $S_2: r_1(A); w_1(A); r_2(B); w_2(B); c_2; r_1(B); w_1(B); c_1$

(c) $S_3: r_1(A); w_1(A); r_2(B); w_2(B); r_1(B); c_2; w_1(B); c_1$

(12 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Q6. Determine whether each of the following schedules is serializable or conflict-serializable, or both. If a schedule is serializable or conflict-serializable, specify the equivalent serial order. Construct the precedence graph to justify your answer.

(a) $w_1(X); w_2(Y); w_1(Y); w_3(X); w_2(X)$

(b) $w_1(X); w_2(Y); w_1(Y); w_2(X); w_3(X)$

(12 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Q7. Show that a “strict” two-phase locking (2PL) schedule avoids cascading rollback. (5 marks) [CSE 215 | L-2/T-1 | 2023–24]

Q8. Show that if a schedule follows the two-phase locking (2PL) protocol, then it is conflict-serializable. (10 marks) [CSE 215 | L-2/T-1 | 2023–24]

Q9. Consider the following transactions $T_1: l_1(A), r_1(A), w_1(A), u_1(A)$ and $T_2: l_2(A), r_2(A), w_2(A), u_2(A)$.

(a) Do the transactions cause a deadlock when executed concurrently? Justify your answer.

(b) Use shared and exclusive locks with lock upgrades to improve concurrency. Show the resulting transactions.

(c) After incorporating lock upgrades, show a concurrent schedule that causes a deadlock.

(d) Use update locks to solve the deadlock problem in (iii) above. Show the resulting transactions.

(12 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Q10. In which scenario does timestamp-based concurrency control perform better than lock-based protocol? In which scenario are lock-based protocols preferable? Discuss the rules of multi-version concurrency control (MVCC) protocol using a suitable example. (13 marks) [CSE 215 | L-2/T-1 | 2023–24]

Q11. Suppose timestamp-based concurrency control is used in a DBMS. For each read or write action, the DBMS takes one of the following actions:

(a) Wait: Put the transaction to sleep

(b) Abort: Abort the transaction (does not restart)

(c) Execute: Execute the action and continue

(d) Skip: Skip the action and continue

For each of the operations in the schedule below, write down the actions of the DBMS. Timestamps: $TS(T_1) < TS(T_2) < TS(T_3) < TS(T_4) < TS(T_5) < TS(T_6)$.

$r_2(A); w_1(A); w_4(B); r_3(B); w_2(A); r_4(A); w_6(C); w_5(C)$

(12 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Q12. Assume you are given the following schedules involving four transactions T_1, T_2, T_3 , and T_4 :

$S_1: r_1(A); w_2(A); r_2(B); r_4(C); r_3(B); w_2(B); w_3(B); r_4(A); w_4(C); r_1(C)$

$S_2: r_1(A); w_2(A); r_2(B); r_4(C); w_2(B); r_3(B); w_3(B); r_4(A); r_1(C); w_4(C)$

Construct precedence graphs for the given schedules S_1 and S_2 . Are the schedules conflict-serializable? Justify your answer. (12 marks) [CSE 215 | L-2/T-1 | 2022–23]

Q13. Show that a schedule satisfying strict 2PL (two-phase locking) is ACR (avoids cascading rollback). (5 marks) [CSE 215 | L-2/T-1 | 2022–23]

Q14. Consider the following two transactions: $T_1: r_1(A); w_1(A)$ and $T_2: w_2(A)$.

(a) Construct a concurrent schedule of T_1 and T_2 that utilises the “Thomas Write Rule”. Show a serial schedule which is equivalent to your given concurrent schedule.

(b) What is the problem of allowing the Thomas Write Rule?

(18 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Q15. Consider the following two transactions using shared ($sl_i(X)$) and exclusive ($xl_i(X)$) locks:

$T_1: sl_1(A); r_1(A); xl_1(A); w_1(A); u_1(A); c_1$

$T_2: sl_2(A); r_2(A); xl_2(A); w_2(A); u_2(A); c_2$

(a) Construct a concurrent schedule of T_1 and T_2 that results in a deadlock due to lock upgrading.

(b) Modify the given transactions using update locks ($ul_i(X)$) to prevent deadlocks.

(10 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Q16. Consider the following schedule of four transactions with start timestamps $TS(T_1) = 1$, $TS(T_2) = 2$, $TS(T_3) = 3$, $TS(T_4) = 4$ and initial read/write timestamps $RT(X) = WT(X) = 0$ for $X \in \{A, B, C\}$:

$$r_1(A); w_2(A); r_2(B); r_4(C); r_3(B); w_2(B); w_3(B); r_4(A); w_4(C); r_1(C)$$

- (a) For each action, show the timestamp which is updated and show the updated values.
- (b) Show the actions for which the scheduler rolls back the relevant transaction.

(12 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Q17. Consider the following two transactions $T_1: r_1(A); w_1(A); c_1$ and $T_2: r_2(A); c_2$.

- (a) Construct a concurrent schedule of T_1 and T_2 that is recoverable but not ACR (avoids cascading rollback).
- (b) Construct a concurrent schedule of T_1 and T_2 that is serializable but not recoverable.
- (c) Construct a schedule of T_1 and T_2 that is ACR (avoids cascading rollback).

(11 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Q18. With necessary figures and examples, explain how Two-Phase Locking is implemented using a Lock Table. (10 marks) [CSE 215 | L-2/T-2 | 2021–22]

Q19. The SQL standard defines three weak isolation levels: dirty reads, read committed, and repeatable reads. Compare these three levels with illustrative examples. (15 marks) [CSE 215 | L-2/T-2 | 2021–22]

Q20. For the following schedule containing four concurrent transactions shown in chronological order, sketch the precedence graph and determine whether the schedule is conflict serializable. If so, write down the equivalent serial schedule(s).

$$R_1(A), W_2(B), R_3(C), W_3(A), R_1(D), R_4(B), W_4(D), W_2(C)$$

(10 marks)

[CSE 215 | L-2/T-2 | 2021–22]

Q21. The list of instructions of transactions T_1 , T_2 and T_3 are given in chronological order as follows:

$$T_1(\text{READ } A), T_2(\text{WRITE } A), T_3(\text{READ } A), T_1(\text{WRITE } B), T_2(\text{WRITE } B),$$

$$T_1(\text{READ } B), T_1(\text{WRITE } P), T_3(\text{WRITE } Q), T_2(\text{READ } B), T_1(\text{WRITE } A)$$

Put appropriate locks on the transactions and show the status of locks. In appropriate places, you can upgrade or downgrade a lock.

(15 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Q22. Explain conflict serializability with an example. Two transactions T_2 : READ(A), WRITE(A), COMMIT and T_1 : READ(A), READ(B), COMMIT are given. Write down the concurrent schedule and the corresponding precedence graph. Using the precedence graph, prove that your schedule is serializable. Find the equivalent serial schedule. (15 marks) [CSE 215 | L-2/T-2 | 2019–20]

Q23. The hash values of data items A , B and C are 5, 1 and 8 respectively. Show and explain the lock table entries for the following instructions with transactions:

$$\begin{aligned} &\{READ(A), T_1\}, \{READ(B), T_2\}, \{READ(C), T_1\}, \{READ(A), T_2\}, \\ &\{READ(B), T_1\}, \{READ(C), T_2\}, \{WRITE(A), T_3\}, \{WRITE(A), T_4\}, \\ &\{WRITE(B), T_3\}, \{WRITE(C), T_1\} \end{aligned}$$

(15 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Q24. Leonard, Howard, and Raj concurrently hacked a theatre seat reservation system. Leonard read all reservations for the matinee show, made changes and saved. Howard read the same show's reservations and did the same. Leonard then read the night show and made changes. Raj changed the morning show reservations and saved. Howard repeated the same for the same night show. (Different relations exist for morning, matinee and night shows.)

- (a) Construct the schedule of actions using standard notations.
- (b) What is a conflict serializable schedule? Show whether the above schedule is conflict serializable without using a Precedence graph.
- (c) Show whether the schedule is conflict serializable using a Precedence graph. If not, briefly describe the reason. If yes, determine the serializability order using topological sort.

(21 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Q25. “Two-phase locking is efficient for B⁺ tree index structure” — Justify this statement using illustrative examples. (7 marks) [CSE 215 | L-2/T-2 | 2017–18]

Q26. Discuss three problems of non-serializable execution with necessary examples. (9 marks)

[CSE 303 | L-3/T-1 | 2016–17]

Q27. Consider the schedule S : $r_2(A); r_1(B); w_2(A); r_2(B); r_3(A); w_1(B); w_3(A); w_2(B)$. Show that there is no serial schedule conflict-equivalent to S . (7 marks) [CSE 303 | L-3/T-1 | 2016–17]

Q28. Mention the requirements of using shared and exclusive locks. (6 marks) [CSE 303 | L-3/T-1 | 2016–17]

Q29. Consider the following schedule:

$$r_1(A); r_2(B); r_3(C); w_1(B); w_2(C); w_3(D)$$

Insert shared and exclusive locks and place necessary unlocks to maximise concurrency. (9 marks) [CSE 303 | L-3/T-1 | 2016–17]

Q30. Find the equivalent serial schedule for the concurrent schedule of four transactions shown below by swapping non-conflicting instructions. Verify the resulting serial schedule using a precedence graph.

T1	T2	T3	T4
READ(A)			
	READ (C)		
READ(B)			
	WRITE (C)		
		READ(C)	
		WRITE (C)	
			READ(B)
			READ(A)
			READ(D)
WRITE (A)			
			WRITE (D)
READ(B)			
		READ(B)	

Figure 4: Concurrent schedule of four transactions

(20 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Q31. Create a lock table for the following list of data items and their locking status:

Data Item	Hash Value	Transactions with Lock	Transactions Waiting
A	5	T1, T5	T2
S	3	T1, T2	—
D	5	—	T3
F	2	T2	T5
G	2	T2, T3, T4	T1
H	5	T4	—

(15 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Q32. A set of transactions with their timestamps and instructions is given. The timestamp values of data item A are: W-timestamp(A) = 10, R-timestamp(A) = 20. The transactions are:

Transaction	Timestamp	Instruction
T1	5	READ(A)
T2	15	READ(A)
T3	25	WRITE(A)
T4	12	WRITE(A)

Find the status (rollback or executed) of each transaction and the corresponding timestamp values of data item A using the timestamp-based protocol. (15 marks) [CSE 303 | L-3/T-1 | 2015–16]

Q33. Explain the Thomas Write Rule with an example. (5 marks) [CSE 303 | L-3/T-1 | 2015–16]

Q34. A concurrent schedule of transactions T_1 , T_2 and T_3 is given in Figure 5. Find the equivalent serial schedule by using conflict serializability. Show the steps.

Transaction T_1	Transaction T_2	Transaction T_3
READ (A)		
	WRITE (B)	
	READ (B)	
WRITE (A)		
		WRITE (C)
		READ (C)
		READ (B)
	READ (C)	
READ(B)		
		READ (A)

Figure 5: A concurrent schedule of transactions T_1 , T_2 and T_3 .

(10 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Q35. Show and explain the lock compatibility matrix with examples. (5 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Q36. What are the tradeoffs of locking with multiple granularities? (5 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Q37. Why does two-phase locking not work for B^+ tree index structures? Write down the rules for accessing tree-structured data that ensure a serial order of transactions in a schedule. (15 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Q38. Consider the following schedule involving transactions T_1 , T_2 , and T_3 :

$$r_1(A), r_2(B), r_3(C), r_1(B), r_2(C), r_3(D), w_1(A), w_2(B), w_3(C)$$

Insert shared locks, exclusive locks, and unlock actions in appropriate places. Tell what happens when the schedule is run by a scheduler supporting shared and exclusive locks. (10 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Q39. Define the following terms: conflict-equivalent and conflict-serializable. (5 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Q40. Why is conflict-serializability not necessary for serializability? Explain with an example. (5 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Q41. If all transactions are well-formed and two-phase, then any legal history will be serializable. Justify this statement. (6 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Q42. Draw the precedence graph of the following schedule. Is the schedule conflict serializable? If so, what are all the equivalent serial schedules?

$$r_1(A); r_2(A); r_3(B); w_1(A); r_2(C); r_2(B); w_2(B); w_1(C)$$

(10 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Q43. Give a counter-example for: $P(S_1) = P(S_2) \Rightarrow S_1, S_2$ conflict equivalent. (4 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Q44. Consider the following schedule of transactions T_1 , T_2 and T_3 :

$$r_1(A); r_2(B); r_3(C); w_1(B); w_2(C); w_3(D)$$

Insert shared and exclusive locks and place necessary unlocks in appropriate positions. (10 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Q45. Define serial, serializable, conflict-serializable, and two-phase locking in terms of database transactions. (8 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Q46. What is the precedence graph of the following schedule?

$$r_1(A); r_2(A); r_1(B); r_2(B); r_3(A); r_4(B); w_1(A); w_2(B)$$

Is the above schedule conflict serializable? If so, what are all the equivalent serial schedules?

(12 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Q47. A concurrent schedule of three transactions T_1 , T_2 and T_3 is given in Figure 3. Show the steps to transform the above schedule into an equivalent conflict serial schedule. (10 marks)

[CSE 215 | L-3/T-1 | 2010–11]

Q48. Explain with examples, the testing of serializability by using precedence graph. Show the cases of both conflict and non-conflict schedules. (10 marks)

[CSE 215 | L-3/T-1 | 2010–11]

Q49. Explain two-phase locking protocol. (5 marks)

[CSE 215 | L-3/T-1 | 2010–11]

Q50. Describe the implementation of lock manager using a lock table data structure and show the following locks:

- (a) T_1 has granted locks for data items 10, 22, 35 and is waiting for data item 44.
- (b) T_2 has granted locks for data items 44 and 22.
- (c) T_3 has been waiting for data item 22.
- (d) T_4 has been granted locks for data items 8, 10, 22 and is waiting for data item 35.

Also explain how the lock manager processes a lock request using the structure. (10 marks)

[CSE 215 | L-3/T-1 | 2010–11]

Q51. Explain the rules of performing read and write operations in the timestamp ordering protocol. What are the advantages and disadvantages of this protocol? How is the protocol improved by the Thomas Write Rule? (10 marks)

[CSE 215 | L-3/T-1 | 2010–11]

Deadlock

Q1. Consider the following database schedule involving four transactions T_1, T_2, T_3, T_4 with deadlock timestamps 100, 150, 200, 250 respectively:

$S: l_1(A); r_1(A); l_2(B); w_2(B); l_1(B); l_3(C); r_3(C); l_2(C); l_4(C); r_4(C); l_1(C)$

- (a) Determine the actions (wound/wait) taken by the wound-wait deadlock prevention protocol. Assume transactions are not restarted after rollback.
- (b) Determine the actions (wait/die) taken by the wait-die deadlock prevention protocol. Assume transactions are not restarted after rollback.

(12 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Q2. Consider the following schedule involving four transactions T_1, T_2, T_3, T_4 with $TS(T_1) < TS(T_2) < TS(T_3) < TS(T_4)$:

$l_2(A); r_2(A); l_2(B); l_1(A); l_3(C); r_3(C); l_3(B); l_4(C); r_4(C); l_1(C); r_1(C)$

- (a) For each lock action $l_i(X)$, determine what happens under the wait-die scheme.
- (b) For each lock action $l_i(X)$, determine what happens under the wound-wait scheme.

(13 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Q3. Consider the following schedule of actions involving four transactions T_1, T_2, T_3 and T_4 :

$r_1(A); r_2(B); w_1(C); r_3(D); w_3(B); w_2(C); w_4(A); w_1(D); w_1(B)$

Assume that the deadlock-timestamps of T_1, T_2, T_3 and T_4 are 1, 2, 3 and 4, respectively. For each action of the given schedule, determine what happens (“wound” or “wait”) under the wound-wait deadlock avoidance system. If the action is a “wound”, do not restart the transaction. (10 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Q4. Consider the following schedule of actions involving four transactions T_1, T_2, T_3 and T_4 . Shared locks are requested immediately before each read action and exclusive locks immediately before every write action:

$r_1(A); r_2(B); w_1(C); r_3(D); w_3(B); w_2(C); w_4(A); w_1(D)$

- (a) Show the wait-for graph after the execution of all actions.
- (b) Determine if there is a deadlock. In case of deadlock, choose the highest-numbered transaction ($T_4 > T_3 > T_2 > T_1$) to roll back. Show the resulting wait-for graph after the rollback.

(12 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Q5. What is deadlock in database transaction? Give an example. (4 marks)

[CSE 303 | L-3/T-1 | 2016–17]

↔ Similar: 2017–18: same question (5 marks)

Q6. What is deadlock in database transaction? Give an example. (3 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Recovery Systems

Q1. Compare the force policy and steal policy with respect to how modified buffers are written to disk in a database system. Under which policy does the database recovery manager not need to perform any actions during the redo phase? (7 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Q2. The database recovery manager takes a redo or undo action expressed as $\langle X, V \rangle$, meaning database object X is written with value V on disk. Consider the following log records:

$\langle T_0 \text{ start} \rangle; \langle T_0, A, 40, 60 \rangle; \langle T_1 \text{ start} \rangle; \langle T_1, B, 100, 150 \rangle; \langle T_0, C, 50, 100 \rangle; \langle T_2 \text{ start} \rangle; \langle T_2, D, 100, 70 \rangle; \langle T_1, C, 20, 50 \rangle; \langle T_1 \text{ commit} \rangle;$
 $\langle T_2, C, 60, 80 \rangle; \langle T_0 \text{ commit} \rangle; \langle T_2 \text{ commit} \rangle$

- (a) For both redo and undo phases, write down the actions of the recovery manager if a system crash happens between $\langle T_2, C, 60, 80 \rangle$ and $\langle T_0 \text{ commit} \rangle$.
- (b) For both redo and undo phases, write down the actions of the recovery manager if a system crash happens between $\langle T_1, C, 20, 50 \rangle$ and $\langle T_1 \text{ commit} \rangle$.

(11 marks)

[CSE 215 | L-2/T-1 | 2024–25]

Q3. Consider the following sequence of log records:

$\langle \text{START } S \rangle; \langle S, A, 60 \rangle; \langle \text{COMMIT } S \rangle; \langle \text{START } T \rangle; \langle T, A, 10 \rangle; \langle \text{START } U \rangle; \langle U, B, 20 \rangle; \langle T, C, 30 \rangle; \langle \text{START } V \rangle; \langle U, D, 40 \rangle; \langle V, F, 70 \rangle;$
 $\langle \text{COMMIT } U \rangle; \langle T, E, 50 \rangle; \langle \text{COMMIT } V \rangle; \langle V, B, 80 \rangle; \langle \text{COMMIT } T \rangle;$

Assume a system crash happens before $\langle \text{COMMIT } V \rangle$ is recorded in the log.

- (a) Using undo logging, describe the actions the recovery manager will perform after the crash. List each action as $\langle X, A \rangle$ where database element X is written with value A .
- (b) Answer (i) above for redo logging.
- (c) Why is it necessary to write an $\langle \text{ABORT} \rangle$ log record for every uncommitted transaction T after the completion of the recovery operation?

(12 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Q4. Consider the following schedule where c_i represents a commit action for transaction T_i :

$$w_1(A); r_2(A); w_2(A); w_2(B); r_3(B); w_3(C); c_3; c_2; c_1$$

- (a) Does the given schedule preserve database consistency in case of system failure? Justify your answer.
- (b) Rearrange the commit statements so that the schedule preserves database consistency in case of system failure.
- (c) Rearrange the commit statements so that the schedule avoids cascading rollback.

(10 marks)

[CSE 215 | L-2/T-1 | 2023–24]

Q5. The following is a sequence of undo-log records written by two transactions T and U :

$\langle \text{START } T \rangle; \langle T, A, 10 \rangle; \langle \text{START } U \rangle; \langle U, B, 20 \rangle; \langle T, C, 30 \rangle; \langle U, D, 40 \rangle; \langle \text{COMMIT } T \rangle; \langle T, E, 50 \rangle; \langle \text{COMMIT } U \rangle$

The recovery manager takes an action $\langle X, V \rangle$ meaning database element X is restored to value V on disk. Determine the actions that will be taken by the recovery manager if there is a crash and the last log record to appear on disk is one of the following:

- (a) $\langle \text{START } T \rangle$
- (b) $\langle T, E, 50 \rangle$
- (c) $\langle \text{COMMIT } T \rangle$

(9 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Q6. Suppose you are given the following sequence of log records representing the actions of a transaction T :

$\langle \text{START } T \rangle; \langle T, A, 10 \rangle; \langle T, B, 20 \rangle; \langle \text{COMMIT } T \rangle$

- (a) For undo logging, what is the last log record that must be sent to disk before updated A is sent to disk?
- (b) For undo logging, what is the last log record that must be sent to disk before updated B is sent to disk?
- (c) For redo logging, what is the last log record that must be sent to disk before updated A and B are sent to disk?

(7 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Q7. Consider the following sequence of undo log records:

$\langle \text{START } S \rangle; \langle S, A, 60 \rangle; \langle \text{COMMIT } S \rangle; \langle \text{START } T \rangle; \langle T, A, 10 \rangle; \langle \text{START } U \rangle; \langle U, B, 20 \rangle; \langle T, C, 30 \rangle; \langle \text{START } V \rangle; \langle U, D, 40 \rangle; \langle V, F, 70 \rangle;$
 $\langle \text{COMMIT } U \rangle; \langle T, E, 50 \rangle; \langle \text{COMMIT } T \rangle; \langle V, B, 80 \rangle; \langle \text{COMMIT } V \rangle$

Suppose that we begin a nonquiescent checkpoint immediately after the $\langle U, B, 20 \rangle$ log record has been written (in memory). Determine the last log record that must be sent to disk before each of the following events:

- (a) $\langle \text{START CKPT}(\dots) \rangle$ is written to disk.
- (b) $\langle \text{END CKPT} \rangle$ is written to disk.

Also, for a system crash occurring after (i) $\langle V, B, 80 \rangle$ and (ii) $\langle T, E, 50 \rangle$, determine the oldest log record the recovery manager needs to scan to find all possible incomplete transactions. (15 marks)

[CSE 215 | L-2/T-1 | 2022–23]

Q8. The log records for transactions are given as follows:

1. $\langle T_0 \text{ start} \rangle$
2. $\langle T_0, A, 1000, 200 \rangle$
3. $\langle T_1 \text{ start} \rangle$
4. $\langle T_1, B, 1000, 200 \rangle$
5. $\langle T_1 \text{ commit} \rangle$
6. $\langle T_2 \text{ start} \rangle$
7. $\langle T_2, C, 1000, 200 \rangle$
8. $\langle T_3 \text{ start} \rangle$
9. $\langle T_3, D, 1000, 200 \rangle$
10. $\langle T_3 \text{ commit} \rangle$
11. $\langle T_4 \text{ start} \rangle$
12. $\langle T_3, E, 1000, 200 \rangle$
13. $\langle T_4 \text{ commit} \rangle$

- (a) Perform a checkpoint after instruction 7 (list all required log entries).
- (b) Assume a failure occurs after instruction 13. Run the log-based recovery algorithm to recover the transactions from failure, considering the checkpoint.

(15 marks)

[CSE 215 | L-2/T-2 | 2020–21]

Q9. Explain checkpoint in database recovery. (5 marks)

[CSE 215 | L-2/T-2 | 2019–20]

Q10. The following is a sequence of undo log records found on disk after a crash:

$\langle \text{START } T_1 \rangle$; $\langle T_1, A, 10 \rangle$; $\langle \text{START } T_2 \rangle$; $\langle T_2, B, 5 \rangle$; $\langle T_1, C, 7 \rangle$;
 $\langle \text{START } T_3 \rangle$; $\langle T_3, D, 12 \rangle$; $\langle \text{COMMIT } T_1 \rangle$; $\langle \text{START CKPT ?} \rangle$;
 $\langle \text{START } T_4 \rangle$; $\langle T_2, E, 5 \rangle$; $\langle \text{COMMIT } T_2 \rangle$; $\langle T_3, F, 1 \rangle$; $\langle T_4, G, 15 \rangle$;
 $\langle \text{END CKPT} \rangle$; $\langle \text{COMMIT } T_3 \rangle$; $\langle \text{START } T_5 \rangle$; $\langle T_5, H, 3 \rangle$;
 $\langle \text{START CKPT ?} \rangle$; $\langle \text{COMMIT } T_5 \rangle$

- (a) What are the correct values of the two “?”-s in the log record?
- (b) Describe the action of the recovery manager, showing changes to both disk and log.

(12 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Q11. The initial account balances of A_1 , A_2 and A_3 are 1000, 2000 and 3000 respectively. Two serial transactions are given:

T20	T21
READ(A_1)	
$A_1 = A_1 + 100$	
WRITE(A_1)	
READ(A_3)	
$A_3 = A_3 - 100$	
WRITE(A_3)	
COMMIT	
	READ(A_2)
	$A_2 = A_2 + 0.1 \times A_2$
	WRITE(A_2)
	COMMIT

- (a) Write log records for the above transactions.
- (b) Show the log-based recovery if a failure occurs after WRITE(A_3).
- (c) Show the recovery if a failure occurs after WRITE(A_2).

(15 marks)

[CSE 303 | L-3/T-1 | 2015–16]

Q12. After a system crash, the following log records are found:

$\langle T_0, \text{start} \rangle$; $\langle T_0, \text{commit} \rangle$; $\langle T_1, \text{start} \rangle$; $\langle T_1, C, 800, 600 \rangle$

Describe the recovery actions for the above log records. (5 marks)

[CSE 303 | L-3/T-1 | 2014–15]

Q13. The following is a sequence of undo/redo-log records written by two transactions T and U :

$\langle T, \text{start} \rangle$; $\langle T, A, 10, 11 \rangle$; $\langle U, \text{start} \rangle$; $\langle U, B, 20, 21 \rangle$; $\langle T, C, 30, 31 \rangle$;
 $\langle U, D, 40, 41 \rangle$; $\langle U, \text{commit} \rangle$; $\langle T, E, 50, 51 \rangle$; $\langle T, \text{commit} \rangle$

Describe the action of the recovery manager, including changes to both disk and the log, if there is a crash and the last log record to appear on disk is:

- (a) $\langle U, \text{start} \rangle$

- (b) $\langle U, \text{commit} \rangle$
- (c) $\langle T, E, 50, 51 \rangle$
- (d) $\langle T, \text{commit} \rangle$

(12 marks)

[CSE 303 | L-3/T-1 | 2013–14]

Q14. What properties of a recovery system allow us to recover from a crash while recovering a previous crash? What are the purposes of checkpointing and nonquiescent checkpointing? (10 marks)

[CSE 303 | L-3/T-1 | 2012–13]

Q15. The following is a sequence of undo/redo-log records written by two transactions T and U :

$\langle T, \text{start} \rangle$; $\langle T, A, 10, 11 \rangle$; $\langle U, \text{start} \rangle$; $\langle U, B, 20, 21 \rangle$; $\langle T, C, 30, 31 \rangle$;
 $\langle U, D, 40, 41 \rangle$; $\langle U, \text{commit} \rangle$; $\langle T, E, 50, 51 \rangle$; $\langle T, \text{commit} \rangle$

Describe the action of the recovery manager, including changes to both the disk and log, when the last log record to appear on disk is:

- (a) $\langle U, \text{commit} \rangle$
- (b) $\langle T, E, 50, 51 \rangle$

(12 marks)

[CSE 303 | L-3/T-1 | 2011–12]

Q16. Describe log-based recovery methodology. For the given transactions $\langle T_1, T_2, T_3 \rangle$, write the log records.

T1	T3	T2
Read(A)	Read(A)	Read(B)
$A = A - 50$	DISPLAY(A)	Read(A)
Write(A)		$B = B + 50$
		Write(B)
		DISPLAY(A+B)

(10 marks)

[CSE 215 | L-3/T-1 | 2010–11]

S8 — Advanced Topics

Distributed & Parallel Databases

Q1. A distributed database system consists of five sites (S_1 – S_5). A transaction T_1 initiated from site S_1 reads and writes data items A (fragmented in S_2 and S_4) and B (fragmented in S_2 , S_3 and S_5). Any write operation must be performed to all related sites.

- (a) Show all the messages (e.g. $\langle \text{Prepare } T_1 \rangle$, etc.) passing between S_1 and the related sites (S_2, S_3, S_4, S_5) for Phase 1 and Phase 2 to **COMMIT** the transaction T_1 using the two-phase commit protocol.
- (b) Show all the message passing for Phase 1 and Phase 2 to **ABORT** the transaction T_1 because of an $\langle \text{Abort } T_1 \rangle$ message sent from S_5 to S_1 .

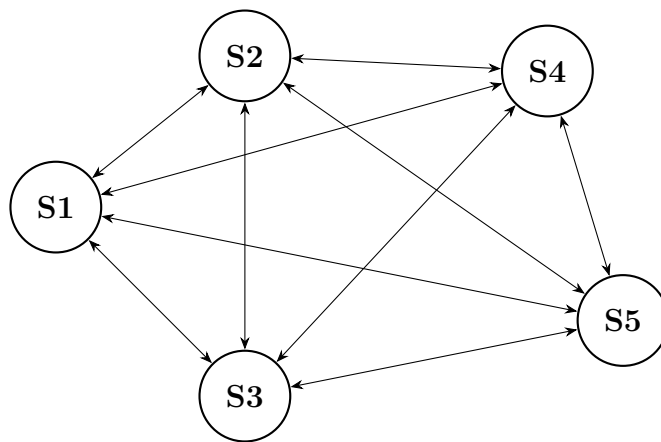


Figure 1: Distributed Database System consisting of five sites

T1	Data replication
READ (A)	Data item A is fragmented to S2, S4
READ (B)	
WRITE (A)	Data item B is fragmented to S2, S3, S5
WRITE (B)	
COMMIT	

Figure 2: A transaction T_1 initiated from S_1 of Figure 1.

(20 marks)

[CSE 303 | L-3/T-1 | 2015–16]

- Q2.** Given relations $R_1(A, B, C, D)$ and $R_2(C, D, E)$ stored in sites S_1 and S_2 . Describe the join operation of R_1 and R_2 using the semi-join strategy. Give an example of parallel join. (10 marks) [CSE 215 | L-3/T-1 | 2010–11]

Big Data & XML

- Q1.** From the following Document Type Definition (DTD), describe the structure of the data in XML and write a sample XML file using this DTD:

```
<!DOCTYPE STARS [
  <!ELEMENT STARS (STAR*)>
  <!ELEMENT STAR (NAME, ADDRESS+, MOVIES)>
  <!ELEMENT NAME (#PCDATA)>
  <!ELEMENT ADDRESS (STREET, CITY)>
  <!ELEMENT STREET (#PCDATA)>
  <!ELEMENT CITY (#PCDATA)>
  <!ELEMENT MOVIES (MOVIE*)>
  <!ELEMENT MOVIE (TITLE, YEAR)>
  <!ELEMENT TITLE (#PCDATA)>
  <!ELEMENT YEAR (#PCDATA)>
]>
```

(10 marks)

[CSE 215 | L-2/T-2 | 2017–18]

- Q2.** Briefly describe the Map-Reduce framework. (5 marks)

[CSE 215 | L-2/T-2 | 2017–18]

Table Partitioning

- Q1.** Suppose BIIS has a large number of records in its Course Registration table, accumulated over the last 20 years. Explain how table partitioning can be done on this table, and what benefits and drawbacks it could offer. (10 marks) [CSE 215 | L-2/T-2 | 2021–22]